

数据结构

第七章 图

图的定义和术语

图的存储结构

图的遍历

生成树

拓扑排序

关键路径

最短路径

6.1 图的定义和术语

图(Graph)是一种比线性表和树更为复杂的数据结构。

线性结构：是研究数据元素之间的一对一关系。在这种结构中，除第一个和最后一个元素外，任何一个元素都有唯一的一个直接前驱和直接后继。

树结构：研究数据元素之间的一对多的关系。在这种结构中，每个元素对下(层)可以有0个或多个元素相联系，对上(层)只有唯一的一个元素相关，数据元素之间有明显层次关系。

图结构：是研究数据元素之间的多对多的关系。在这种结构中，任意两个元素之间可能存在关系。即结点之间的关系可以是任意的，图中任意元素之间都可能相关。

图的应用极为广泛，已渗入到诸如语言学、逻辑学、物理、化学、电讯、计算机科学以及数学的其它分支。

6.1 图的定义和术语

□ 图(Graph)——图 G 是由两个集合 $V(G)$ 和 $E(G)$ 组成的,记为 $G=(V,E)$ 。其中:

$V(G)$ 是顶点的非空有限集

$E(G)$ 是边的有限集合,边是顶点的无序对或有序对

□ 有向图——有向图 G 是由两个集合 $V(G)$ 和 $E(G)$ 组成的
其中: $V(G)$ 是顶点的非空有限集

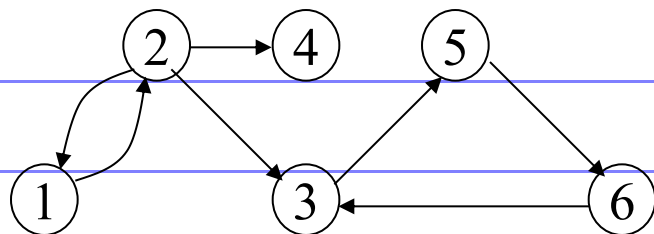
$E(G)$ 是有向边 (也称弧) 的有限集合,弧是顶点的有序对,记为 $\langle v,w \rangle$, v,w 是顶点, v 为弧尾, w 为弧头

□ 无向图——无向图 G 是由两个集合 $V(G)$ 和 $E(G)$ 组成的
其中: $V(G)$ 是顶点的非空有限集

$E(G)$ 是边的有限集合,边是顶点的无序对,记为 (v,w) 或 (w,v) ,并且 $(v,w)=(w,v)$

图的例子

例1

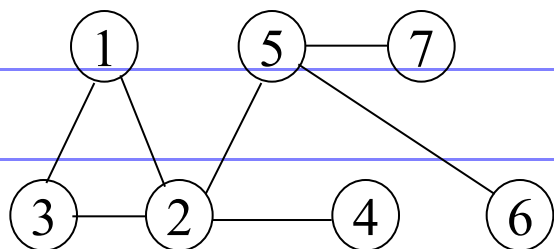


G1

图G1中: $V(G1)=\{1,2,3,4,5,6\}$

$E(G1)=\{<1,2>, <2,1>, <2,3>, <2,4>, <3,5>, <5,6>, <6,3>\}$

例2



G2

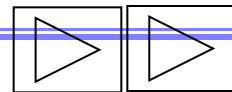
图G2中: $V(G2)=\{1,2,3,4,5,6,7\}$

$E(G1)=\{(1,2), (1,3), (2,3), (2,4), (2,5), (5,6), (5,7)\}$

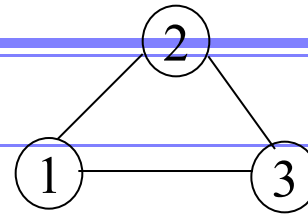
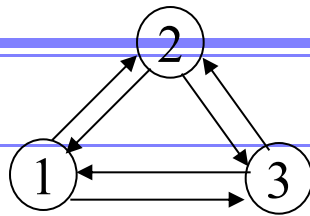
图的定义和术语

- 有向完备图—— n 个顶点的有向图最大边数是 $n(n-1)$
- 无向完备图—— n 个顶点的无向图最大边数是 $n(n-1)/2$
- 权——与图的边或弧相关的数叫权(权可以表示从一个顶点到另一个顶点的距离或耗费)
- 网——带权的图叫网
- 子图——如果图 $G(V, E)$ 和图 $G'(V', E')$, 满足:
 - $V' \subseteq V$
 - $E' \subseteq E$ 则称 G' 为 G 的子图
- 顶点的度
 - 无向图中, 顶点的度为与每个顶点相连的边数
 - 有向图中, 顶点的度分成入度与出度
 - ◇ 入度: 以该顶点为头的弧的数目
 - ◇ 出度: 以该顶点为尾的弧的数目

- 路径：路径是顶点的序列 $V=\{V_{i0}, V_{i1}, \dots, V_{in}\}$ ，满足 $(V_{ij-1}, V_{ij}) \in E$ 或 $\langle V_{ij-1}, V_{ij} \rangle \in E, (1 \leq j \leq n)$
- 路径长度：沿路径边的数目或沿路径各边权值之和
- 回路：第一个顶点和最后一个顶点相同的路径叫~
- 简单路径：序列中顶点不重复出现的路径叫~
- 简单回路：除了第一个顶点和最后一个顶点外，其余顶点不重复出现的回路叫~
- 连通：从顶点 V 到顶点 W 有一条路径，则说 V 和 W 是连通的
- 连通图：图中任意两个顶点都是连通的叫~
- 连通分量：非连通图的每一个连通部分叫~
- 强连通图：有向图中，如果对每一对 $V_i, V_j \in V, V_i \neq V_j$ ，从 V_i 到 V_j 和从 V_j 到 V_i 都存在路径，则称 G 是~

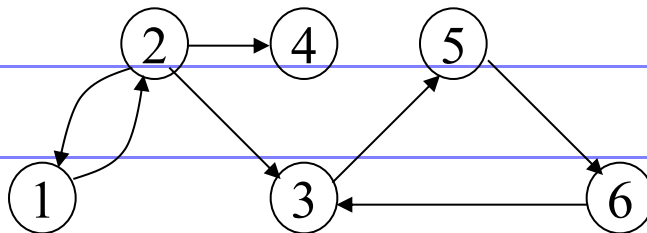


例

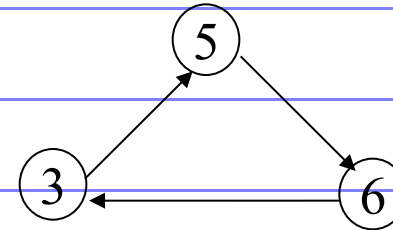


例

有向完备图



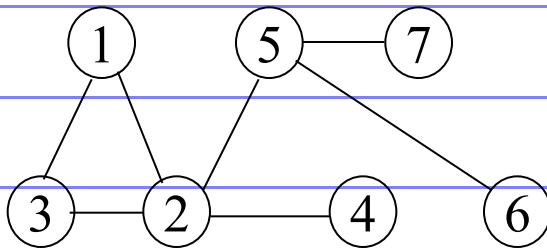
无向完备图



例

图与子图

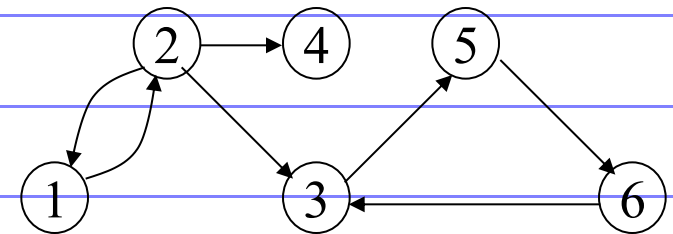
例



G2

顶点5的度: 3

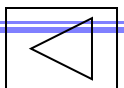
顶点2的度: 4



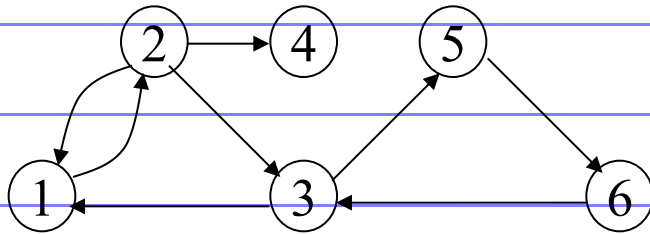
G1

顶点2入度: 1 出度: 3

顶点4入度: 1 出度: 0



例



G1

路径: 1, 2, 3, 5, 6, 3

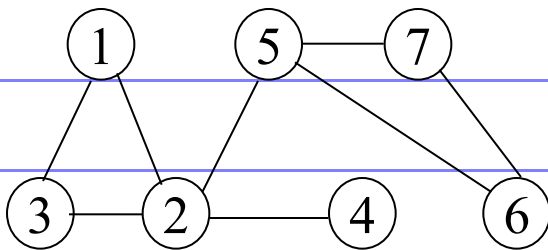
路径长度: 5

简单路径: 1, 2, 3, 5

回路: 1, 2, 3, 5, 6, 3, 1

简单回路: 3, 5, 6, 3

例



G2

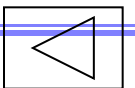
路径: 1, 2, 5, 7, 6, 5, 2, 3

路径长度: 7

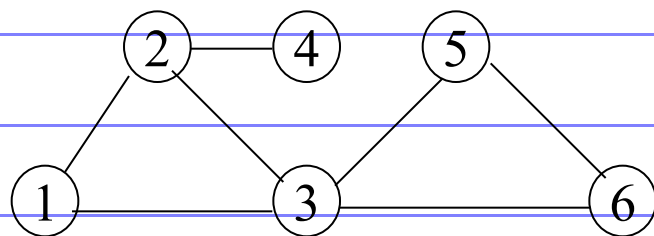
简单路径: 1, 2, 5, 7, 6

回路: 1, 2, 5, 7, 6, 5, 2, 1

简单回路: 1, 2, 3, 1

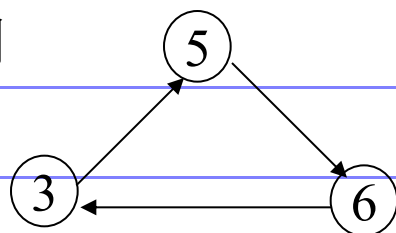


例



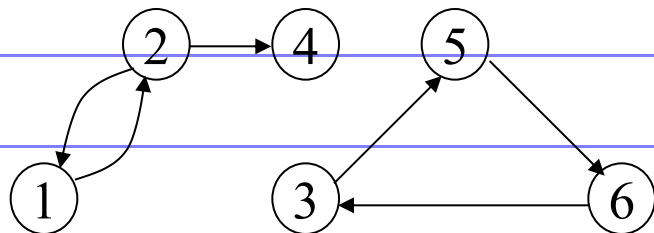
连通图

例

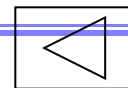


强连通图

例



非连通图
连通分量



图的抽象数据类型定义

图是一种数据结构，加上一组基本操作就构成了图的抽象数据类型。

图的抽象数据类型定义如下：

ADT Graph{

- 数据对象V：具有相同特性的数据元素的集合，称为顶点集。
- 数据关系R： $R = \{VR\}$

$VR = \{ \langle v, w \rangle \mid \langle v, w \rangle \mid v, w \in V \wedge p(v, w), \langle v, w \rangle \text{表示从} v \text{到} w \text{的弧, } P(v, w) \text{定义了弧} \langle v, w \rangle \text{的信息} \}$

- 基本操作P：

Create_Graph()：图的创建操作。

初始条件：无。

操作结果：生成一个没有顶点的空图G。**GetVex(G, v)**：求图中的顶点v的值。

初始条件：图G存在，v是图中的一个顶点。

操作结果：生成一个没有顶点的空图G。

... ..

DFStraver(G, V)：从v出发对图G深度优先遍历。

初始条件：图G存在。

操作结果：对图G深度优先遍历，每个顶点访问且只访问一次。

BFStraver(G, V)：从v出发对图G广度优先遍历。

初始条件：图G存在。

操作结果：对图G广度优先遍历，每个顶点访问且只访问一次。

} ADT Graph

详见p_{156~157}。

6.2 图的存储结构

图的存储结构比较复杂，其复杂性主要表现在：

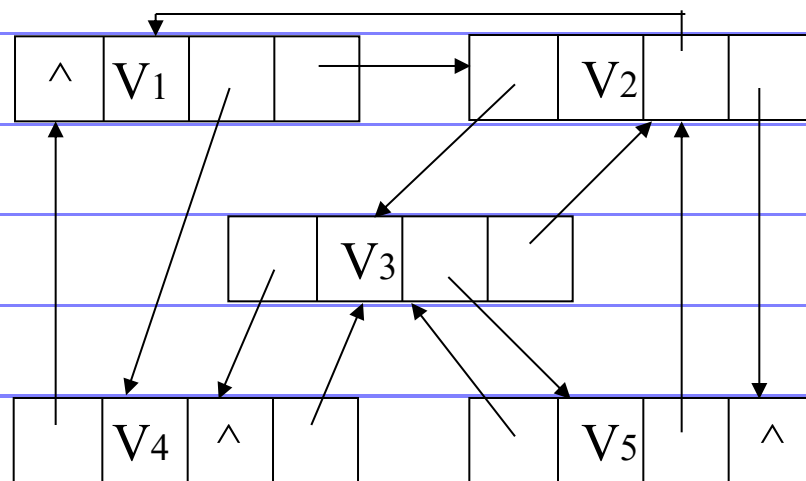
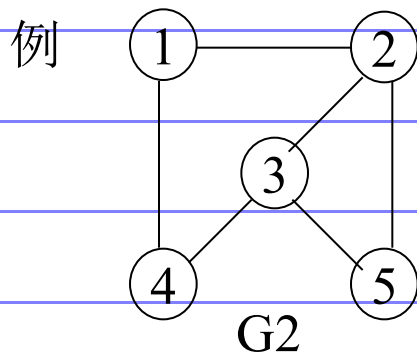
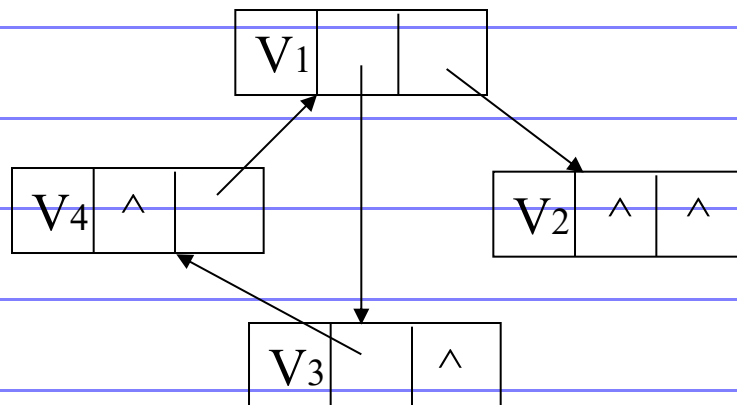
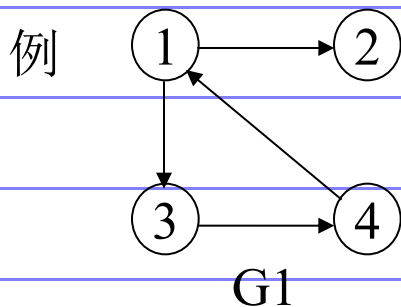
- ◆ 任意顶点之间可能存在联系，无法以数据元素在存储区中的物理位置来表示元素之间的关系。
- ◆ 图中顶点的度不一样，有的可能相差很大，若按度数最大的顶点设计结构，则会浪费很多存储单元，反之按每个顶点自己的度设计不同的结构，又会影响操作。

图的常用的存储结构有：

- ☐ 多重链表
- ☐ 邻接矩阵
- ☐ 关联矩阵
- ☐ 邻接表

图的存储结构-多重链表

□ 多重链表



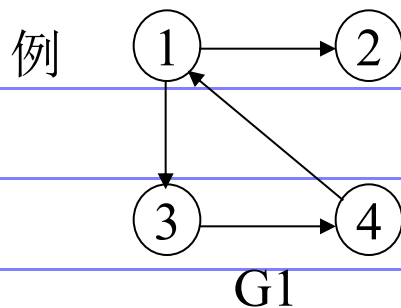
数据结构

图的存储结构-邻接矩阵

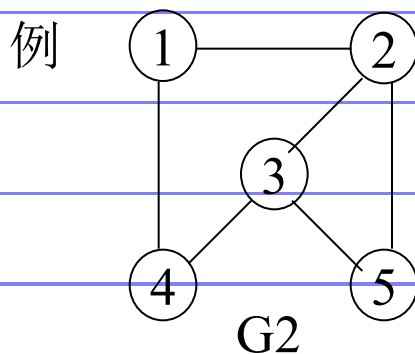
□ 邻接矩阵——表示顶点间相联关系的矩阵

□ 定义：设 $G=(V,E)$ 是有 $n \geq 1$ 个顶点的图， G 的邻接矩阵 A 是具有以下性质的 n 阶方阵

$$A[i, j] = \begin{cases} 1, & \text{若}(v_i, v_j) \text{ 或 } \langle v_i, v_j \rangle \in E(G) \\ 0, & \text{其它} \end{cases}$$



	①	②	③	④
①	0	1	1	0
②	0	0	0	0
③	0	0	0	1
④	1	0	0	0



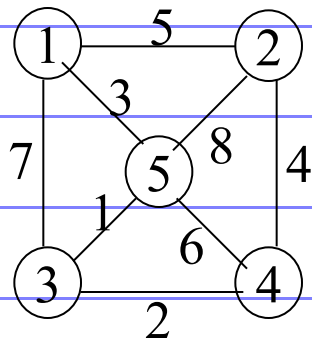
	①	②	③	④	⑤
①	0	1	0	1	0
②	1	0	1	0	1
③	0	1	0	1	1
④	1	0	1	0	0
⑤	0	1	1	0	0

■ 特点:

- ◇ 无向图的邻接矩阵对称, 可压缩存储; 有 n 个顶点的无向图需存储空间为 $n(n+1)/2$
- ◇ 有向图邻接矩阵不一定对称; 有 n 个顶点的有向图需存储空间为 n^2
- ◇ 无向图中顶点 V_i 的度 $TD(V_i)$ 是邻接矩阵 A 中第 i 行元素之和
- ◇ 有向图中,
 - 顶点 V_i 的出度是 A 中第 i 行元素之和
 - 顶点 V_i 的入度是 A 中第 i 列元素之和
- ◇ 网络的邻接矩阵可定义为:

$$A[i, j] = \begin{cases} \omega_{ij}, & \text{若 } (v_i, v_j) \text{ 或 } \langle v_i, v_j \rangle \in E(G) \\ 0, & \text{其它} \end{cases}$$

例



	①	②	③	④	⑤
①	0	5	7	0	3
②	5	0	0	4	8
③	7	0	0	2	1
④	0	4	2	0	6
⑤	3	8	1	6	0

图的存储结构-关联矩阵

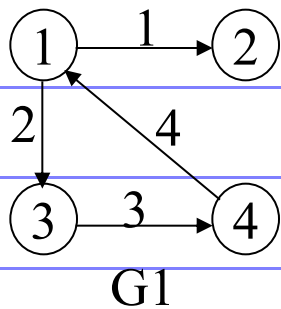
□ 关联矩阵——表示顶点与边的关联关系的矩阵

■ 定义：设 $G=(V,E)$ 是有 $n \geq 1$ 个顶点， $e \geq 0$ 条边的图， G 的关联矩阵 A 是具有以下性质的 $n \times e$ 阶矩阵

$$\text{有向图: } A[i, j] = \begin{cases} 1, & i \text{ 顶点与 } j \text{ 边相连, 且 } i \text{ 为尾} \\ 0, & i \text{ 顶点与 } j \text{ 边不相连} \\ -1, & i \text{ 顶点与 } j \text{ 边相连, 且 } i \text{ 为头} \end{cases}$$

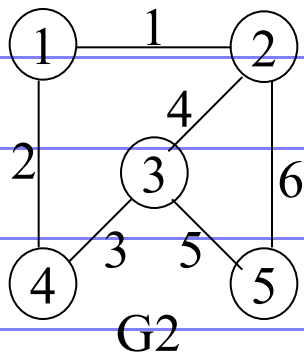
$$\text{无向图: } A[i, j] = \begin{cases} 1, & i \text{ 顶点与 } j \text{ 边相连} \\ 0, & i \text{ 顶点与 } j \text{ 边不相连} \end{cases}$$

例



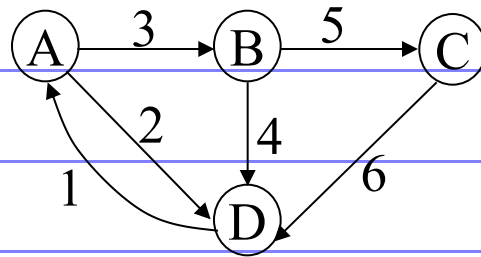
	1	2	3	4
①	1	1	0	-1
②	-1	0	0	0
③	0	-1	1	0
④	0	0	-1	1

例



	1	2	3	4	5	6
①	1	1	0	0	0	0
②	1	0	0	1	0	1
③	0	0	1	1	1	0
④	0	1	1	0	0	0
⑤	0	0	0	0	1	1

例



	1	2	3	4	5	6
A	-1	1	1	0	0	0
B	0	0	-1	1	1	0
C	0	0	0	0	-1	1
D	1	-1	0	-1	0	-1

□ 特点

- 关联矩阵每列只有两个非零元素，是稀疏矩阵； n 越大，零元素比率越大
- 无向图中顶点 V_i 的度 $TD(V_i)$ 是关联矩阵 A 中第 i 行元素之和
- 有向图中，
 - ◇ 顶点 V_i 的出度是 A 中第 i 行中“1”的个数
 - ◇ 顶点 V_i 的入度是 A 中第 i 行中“-1”的个数

邻接表

- 实现：为图中每个顶点建立一个单链表，第 i 个单链表中的结点表示依附于顶点 V_i 的边（有向图中指以 V_i 为尾的弧）

```
#define MAX_VERTEX_NUM 20
```

```
typedef struct ArcNode {
```

```
    int adjvex;    //邻接点域，存放与 $V_i$ 邻接的点在表头数组中的位置
```

```
    struct ArcNode *nextarc; //链域，指示下一条边或弧
```

```
    InfoType *info;
```

adjvex	nextarc	info
--------	---------	------

```
} ArcNode;
```

```
typedef struct VNode { //表头接点
```

```
    VertexType data;    //存放顶点信息
```

```
    ArcNode *firstarc; //指示第一个邻接点
```

vexdata	firstarc
---------	----------

```
} VNode, AdjList[MAX_VERTEX_NUM];
```

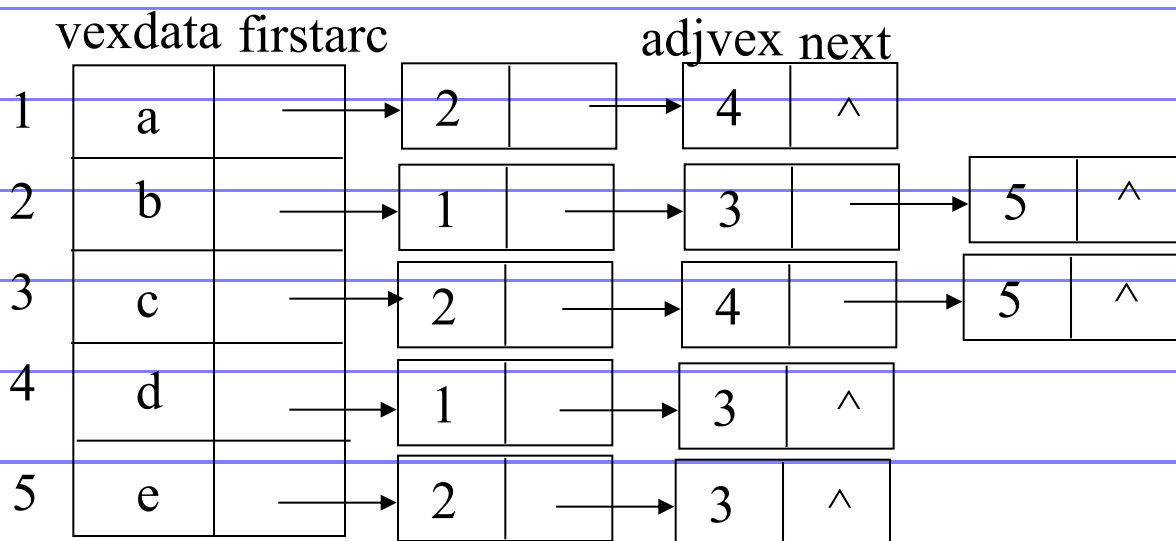
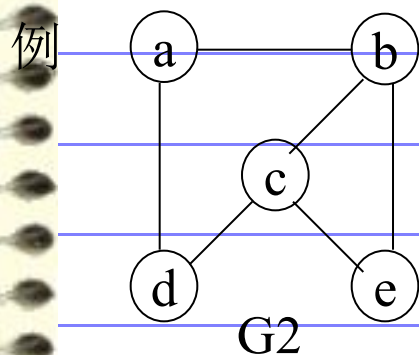
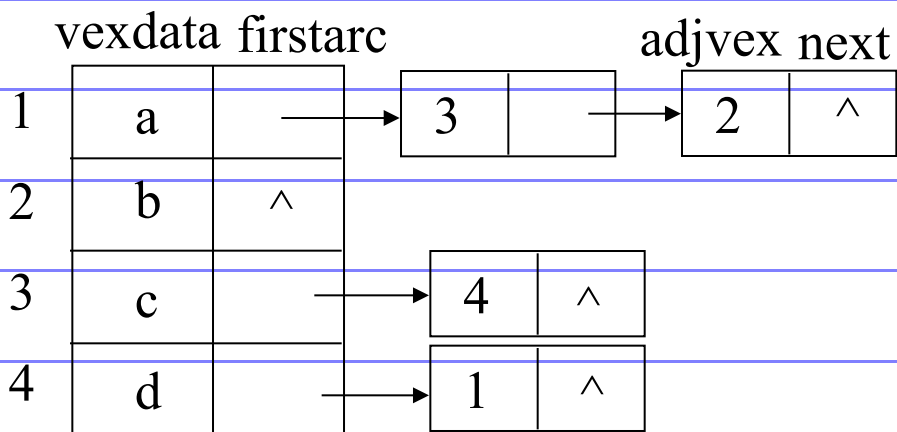
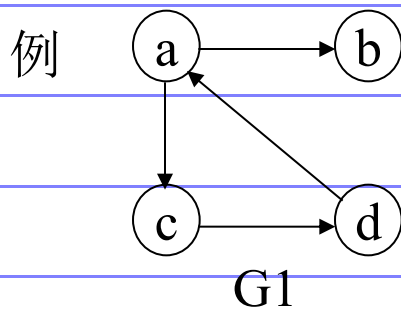
```
typedef struct {
```

```
    AdjList vertices;
```

```
    int vexnum, arcnum;
```

```
    int kind;    //图的种类标识
```

```
} ALGraph;
```



特点

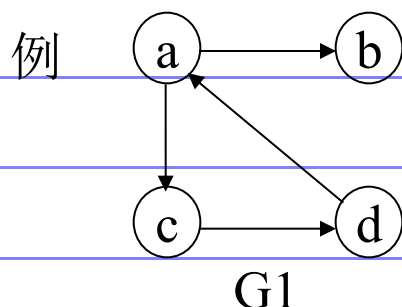
■ 无向图中顶点 V_i 的度为第 i 个单链表中的结点数

■ 有向图中

◇ 顶点 V_i 的出度为第 i 个单链表中的结点数

◇ 顶点 V_i 的入度为整个单链表中邻接点域值是 i 的结点数

◇ 逆邻接表：有向图中对每个结点建立以 V_i 为头的弧的单链表



	vexdata	firstarc	adjvex	next
1	a	→	4	^
2	b	→	1	^
3	c	→	1	^
4	d	→	3	^

数据结构

有向图的十字链表表示法

十字链表(Orthogonal List)是有向图的另一种链式存储结构，是将有向图的正邻接表和逆邻接表结合起来得到的一种链表。

在这种结构中，每条弧的弧头结点和弧尾结点都存放在链表中，并将弧结点分别组织到以弧尾结点为头(顶点)结点和以弧头结点为头(顶点)结点的链表中。

有向图的十字链表表示法

```
#define MAX_VERTEX_NUM 20
```

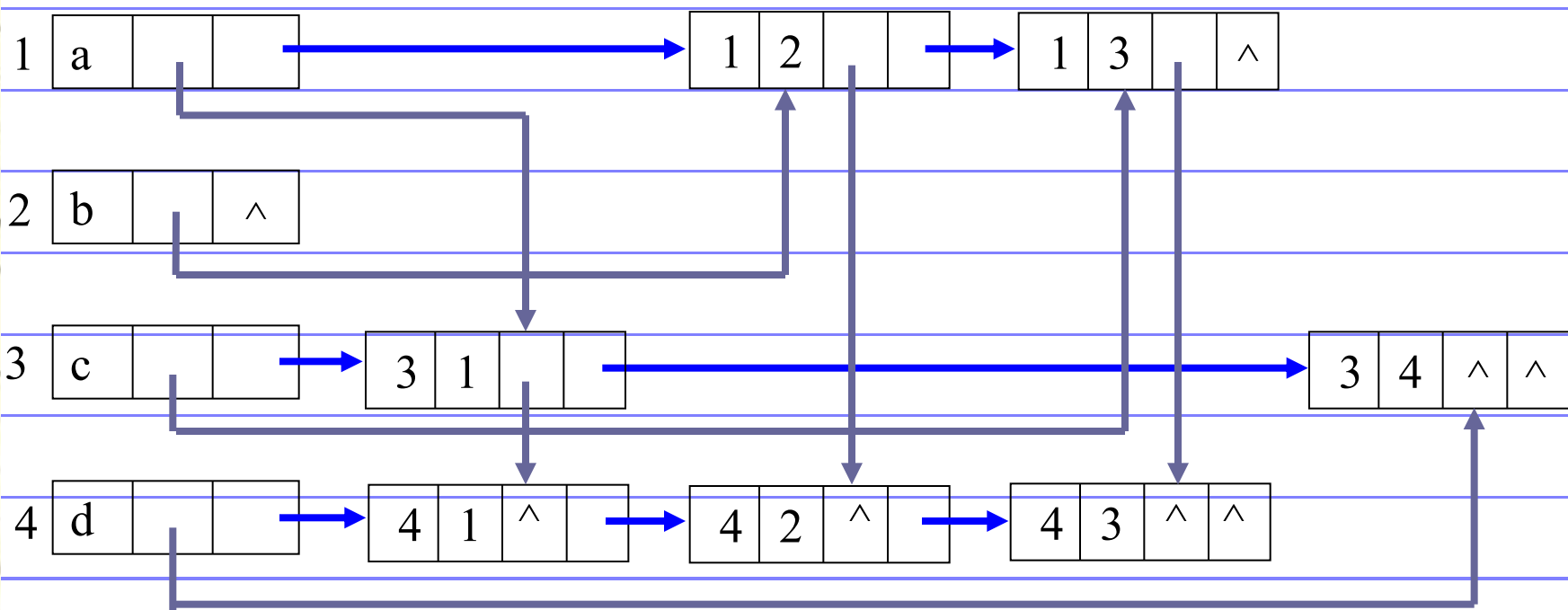
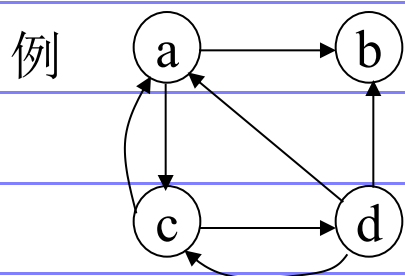
```
typedef struct ArcBox { //弧结点  
    int tailvex, headvex; //弧尾、弧头在表头数组中位置  
    struct ArcBox *hlink, *tlink; //分别指向弧头、弧尾相同的下一条弧  
    InfoType *info;  
} ArcBox;
```

tailvex	headvex	hlink	tlink	info
---------	---------	-------	-------	------

```
typedef struct VexNode { //顶点结点  
    VertexType data; //存与顶点有关信息  
    ArcBox *firstin, *firstout; //分别指向该顶点第一条入弧和出弧  
} VexNode;
```

data	firstin	firstout
------	---------	----------

```
typedef struct {  
    VexNode xlist[MAX_VERTEX_NUM];  
    int vexnum, arcnum;  
} OLGraph;
```



无向图的邻接多重表表示法

```
#define MAX_VERTEX_NUM 20
#define enum {unvisited, visited} VisitIf;
typedef struct EBox {    //边结点
    VisitIf mark;    //标志域
    int ivex, jvex; //该边依附的两个顶点在表头数组中位置
    struct EBox *ilink, *jlink; //分别指向依附于ivex和jvex的下一条边
    InfoType *info;
```

```
}EBox;
```

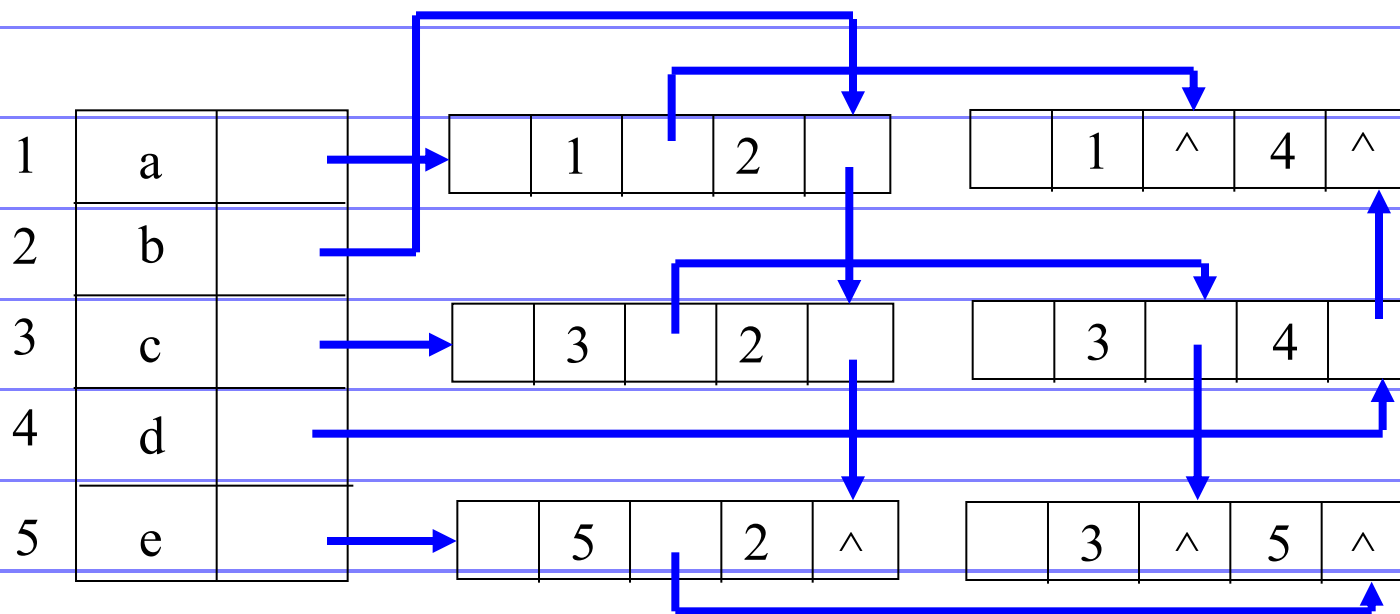
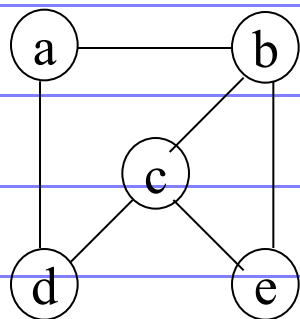
mark	ivex	ilink	jvex	jlink
------	------	-------	------	-------

```
typedef struct VexNode {    //顶点结点
    VertexType data; //存与顶点有关的信息
    EBox *firstedge; //指向第一条依附于该顶点的边
}VexBox;
```

data	firstedge
------	-----------

```
typedef struct {
    VexBox adjmulist[MAX_VERTEX_NUM];
    int vexnum, edgenum;
} AMLGraph;
```

例



6.3 图的遍历

图的遍历(Travering Graph): 从图的某一顶点出发, 访遍图中的其余顶点, 且每个顶点仅被访问一次。图的遍历算法是各种图的操作的基础。

◆ **复杂性:** 图的任意顶点可能和其余的顶点相邻接, 可能在访问了某个顶点后, 沿某条路径搜索后又回到原顶点。

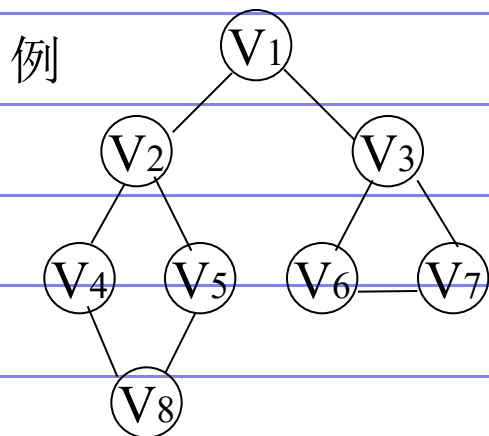
◆ **解决办法:** 在遍历过程中记下已被访问过的顶点。设置一个辅助向量 **Visited[1...n]** (n 为顶点数), 其初值为 0, 一旦访问了顶点 v_i 后, 使 **Visited[i]** 为 1 或为访问的次序号。

图的遍历算法有**深度优先搜索算法**和**广度优先搜索算法**。采用的数据结构是**(正)邻接链表**。

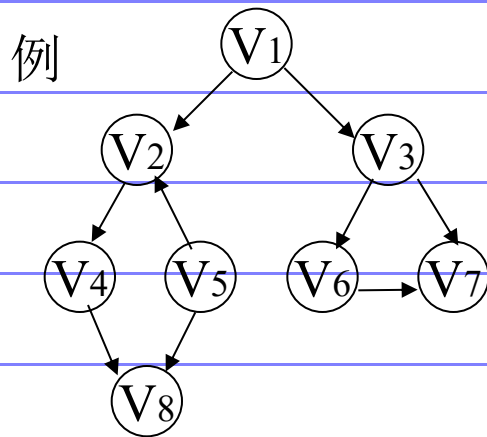
6.3 图的遍历

□ 深度优先遍历(DFS)

- 从图的某一顶点 V_0 出发，访问此顶点；然后依次从 V_0 的未被访问的邻接点出发，深度优先遍历图，直至图中所有和 V_0 相通的顶点都被访问到；若此时图中尚有顶点未被访问，则另选图中一个未被访问的顶点作起点，重复上述过程，直至图中所有顶点都被访问为止



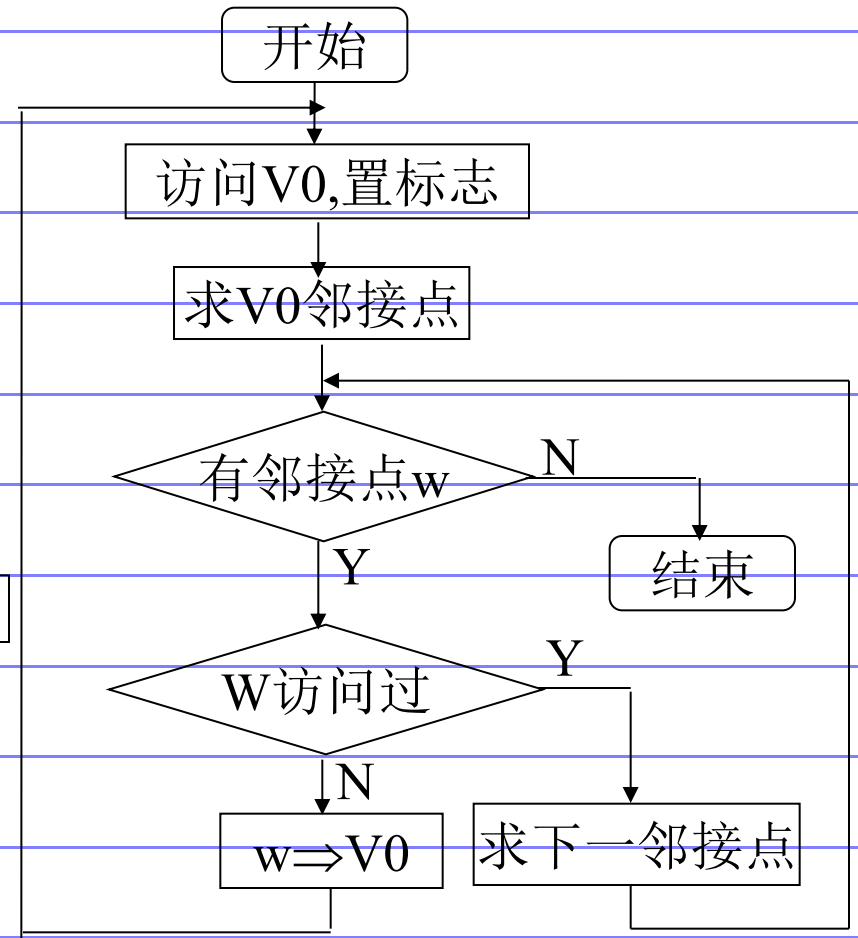
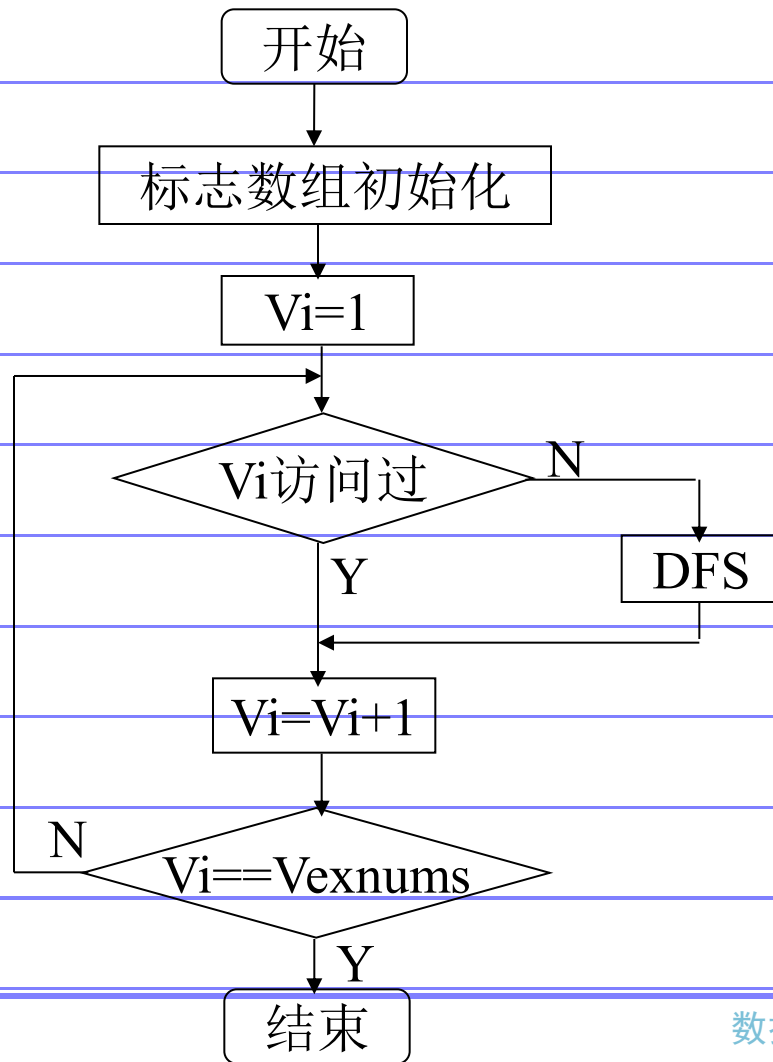
深度遍历: $V_1 \Rightarrow V_2 \Rightarrow V_4 \Rightarrow V_8 \Rightarrow V_5 \Rightarrow V_3 \Rightarrow V_6 \Rightarrow V_7$



深度遍历: $V1 \Rightarrow V2 \Rightarrow V4 \Rightarrow V8 \Rightarrow V3 \Rightarrow V6 \Rightarrow V7 \Rightarrow V5$

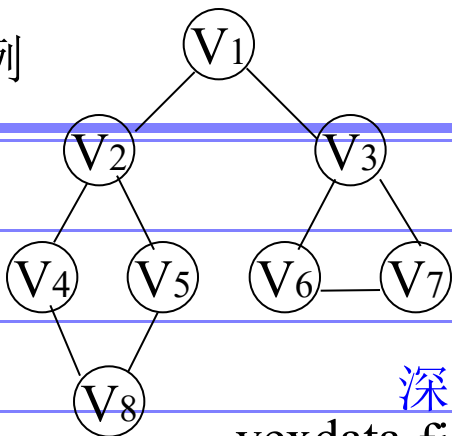
深度优先遍历算法

□ 递归算法



DFS

例

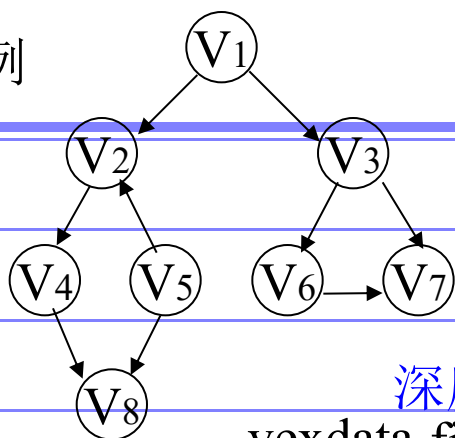


深度遍历: $V1 \Rightarrow V3 \Rightarrow V7 \Rightarrow V6 \Rightarrow V2 \Rightarrow V5 \Rightarrow V8 \Rightarrow V4$

	vexdata	firstarc		adjvex	next
1	1	→	3	→	2 ^
2	2	→	5	→	4 → 1 ^
3	3	→	7	→	6 → 1 ^
4	4	→	8	→	2 ^
5	5	→	8	→	2 ^
6	6	→	7	→	3 ^
7	7	→	6	→	3 ^
8	8	→	5	→	4 ^

数据结构

例



深度遍历: $V1 \Rightarrow V3 \Rightarrow V7 \Rightarrow V6 \Rightarrow V2 \Rightarrow V4 \Rightarrow V8 \Rightarrow V5$

	vexdata	firstarc	adjvex	next
1	1	→	3	→ 2 ^
2	2	→	4	^
3	3	→	7	→ 6 ^
4	4	→	8	^
5	5	→	8	→ 2 ^
6	6	→	7	^
7	7	^		
8	8	^		

基于邻接表的图的深度优先遍历

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTEX_NUM 100 // 图中最大节点数
typedef char VertexType;

// 边表节点
typedef struct node {
    VertexType adjvex; // 与顶点相连的邻接点下标(adjoin: 邻接)
    struct node* next; // 指向顶点的下一个邻接点
} EdgeNode;

// 顶点结构
typedef struct vnode {
    VertexType vex; // 存储顶点名
    EdgeNode* firstedge; // 边表头指针, 指向顶点第一个邻接点
} VertexNode, AdjList[MAX_VERTEX_NUM];

typedef struct {
    AdjList adjlist; // 描述图结构的邻接表
    int vexnum; // 节点的数目
    int edgenum; // 边的数目
} ALGraph; // adjacency list: 邻接表

int main(void)
{
    ALGraph g;

    CreateALG(&g);
    printf("-----\n");
    printf("vexnum = %d ; edgenum = %d\n", g.vexnum, g.edgenum);
    printf("-----\n");
    TraverseALG(g);
    printf("-----\n");
    DFSTraverseALG(g);

    return 0;
}

void CreateALG(ALGraph* ALG); // 邻接表法创建图
void TraverseALG(ALGraph ALG); // 输出图ALG的邻接表
void DFSTraverseALG(ALGraph ALG); // 深度优先遍历以邻接表存储的图ALG
void DFSALG(ALGraph ALG, int i); // 以vi为出发点对邻接表存储的图ALG开始DFS搜索
// 定位节点vertex, 并将其下标赋给index
void LocateVex(ALGraph ALG, VertexType vertex, int* index);
int visited[MAX_VERTEX_NUM]; // 标志数组
```

基于邻接表的图的深度优先遍历

```
void CreateALG(ALGraph* ALG)
{
    VertexType ch;
    int i = 0, count = 0;
    EdgeNode* temp;

    printf("请输入图的顶点: ");
    // 建立顶点表
    while ((ch = getchar()) != '\n') {
        ALG->adjlist[i].vex = ch;
        ALG->adjlist[i].firstedge = NULL;
        i++;
    }
    ALG->vexnum = i; // 顶点数

    // 头插法建立顶点的邻接边表
    for (i = 0; i < ALG->vexnum; i++) {
        printf("请输入顶点 %c 的邻接顶点: ", ALG->adjlist[i].vex);
        // 按下回车结束邻接点的创建
        while ((ch = getchar()) != '\n') {
            temp = (EdgeNode*)malloc(sizeof(EdgeNode));
            temp->adjvex = ch;
            temp->next = ALG->adjlist[i].firstedge;
            ALG->adjlist[i].firstedge = temp;
            count++;
        }
    }

    // 无向图中每条边连接两个顶点, 故: 节点总度数 = 边数 * 2
    ALG->edgenum = count / 2;
}
```

```
void TraverseALG(ALGraph ALG)
{
    int i;
    EdgeNode* temp;

    if (ALG.vexnum == 0) {
        printf("图为空\n");
        return;
    }

    // 遍历图
    for (i = 0; i < ALG.vexnum; i++) {
        printf("顶点 %c : ", ALG.adjlist[i].vex);
        temp = ALG.adjlist[i].firstedge;
        // 输出图的信息
        while (temp) {
            printf("[ %c ] -> ", temp->adjvex);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}
```

```
// 深度优先遍历以邻接表存储的图ALG
```

```
void DFSTraverseALG(ALGraph ALG)
```

```
{
```

```
    int i;
```

```
    // 初始化标志数组
```

```
    for (i = 0; i < ALG.vexnum; i++) {  
        visited[i] = 0;  
    }
```

```
    printf("图的深度优先遍历序列: ");
```

```
    // 从第一个节点开始DFS搜索
```

```
    for (i = 0; i < ALG.vexnum; i++) {  
        if (!visited[i]) {  
            DFSALG(ALG, i);  
        }  
    }
```

```
    // 以下标为i的节点为出发点对图ALG开始DFS搜索
```

```
void DFSALG(ALGraph ALG, int i)
```

```
{
```

```
    EdgeNode* temp;
```

```
    int index;
```

```
    printf("%c, ", ALG.adjlist[i].vex);
```

```
    visited[i] = 1; // 标记节点i已被访问
```

```
    temp = ALG.adjlist[i].firstedge;
```

```
    while (temp) {
```

```
        LocateVex(ALG, temp->adjvex, &index);
```

```
        // 若以index为下标的节点未被遍历, 则遍历, 并从该节点开始进行下一轮DFS搜索
```

```
        if (!visited[index]) {
```

```
            DFSALG(ALG, index);
```

```
        }
```

```
        // 若以index为下标的节点被遍历, 则寻找节点的下一个邻接点
```

```
        temp = temp->next;
```

```
    }
```

```
}
```

```
void LocateVex(ALGraph ALG, VertexType vertex, int* index)
```

```
{
```

```
    int i;
```

```
    for (i = 0; i < ALG.vexnum; i++) {
```

```
        if (ALG.adjlist[i].vex == vertex) {
```

```
            *index = i; // 将节点vertex的下标赋给index
```

```
            return;
```

```
        }
```

```
    }
```

```
}
```

```
请输入图的顶点: 12345678
```

```
请输入顶点 1 的邻接顶点: 32
```

```
请输入顶点 2 的邻接顶点: 541
```

```
请输入顶点 3 的邻接顶点: 761
```

```
请输入顶点 4 的邻接顶点: 82
```

```
请输入顶点 5 的邻接顶点: 82
```

```
请输入顶点 6 的邻接顶点: 73
```

```
请输入顶点 7 的邻接顶点: 63
```

```
请输入顶点 8 的邻接顶点: 54
```

```
-----  
vexnum = 8 ; edgenum = 9
```

```
-----  
顶点 1 : [ 2 ] -> [ 3 ] -> NULL
```

```
顶点 2 : [ 1 ] -> [ 4 ] -> [ 5 ] -> NULL
```

```
顶点 3 : [ 1 ] -> [ 6 ] -> [ 7 ] -> NULL
```

```
顶点 4 : [ 2 ] -> [ 8 ] -> NULL
```

```
顶点 5 : [ 2 ] -> [ 8 ] -> NULL
```

```
顶点 6 : [ 3 ] -> [ 7 ] -> NULL
```

```
顶点 7 : [ 3 ] -> [ 6 ] -> NULL
```

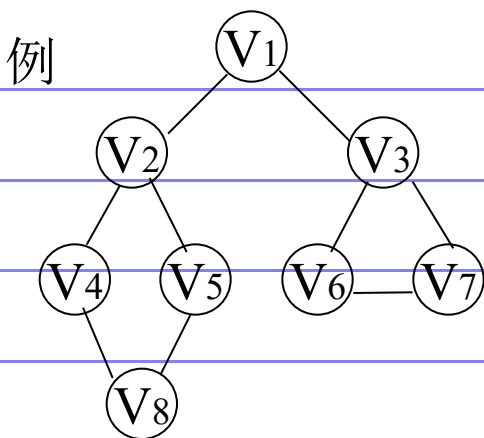
```
顶点 8 : [ 4 ] -> [ 5 ] -> NULL
```

```
-----  
图的深度优先遍历序列: 1, 2, 4, 8, 5, 3, 6, 7, Hit any key to close this window...
```

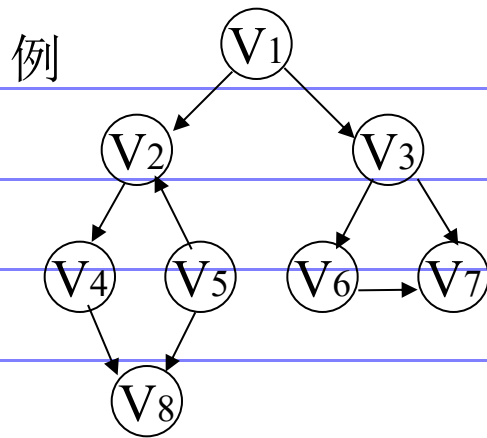
图的遍历

□ 广度优先遍历(BFS)

- 从图的某一顶点 V_0 出发，访问此顶点后，依次访问 V_0 的各个未曾访问过的邻接点；然后分别从这些邻接点出发，广度优先遍历图，直至图中所有已被访问的顶点的邻接点都被访问到；若此时图中尚有顶点未被访问，则另选图中一个未被访问的顶点作起点，重复上述过程，直至图中所有顶点都被访问为止

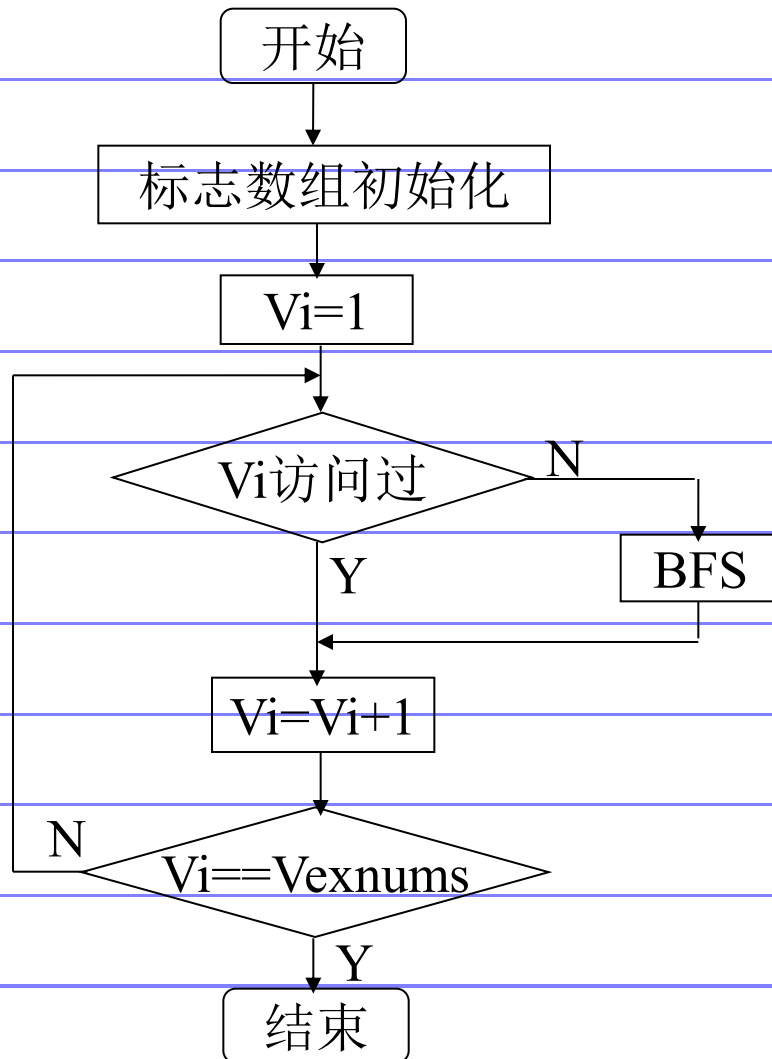


广度遍历: $V1 \Rightarrow V2 \Rightarrow V3 \Rightarrow V4 \Rightarrow V5 \Rightarrow V6 \Rightarrow V7 \Rightarrow V8$

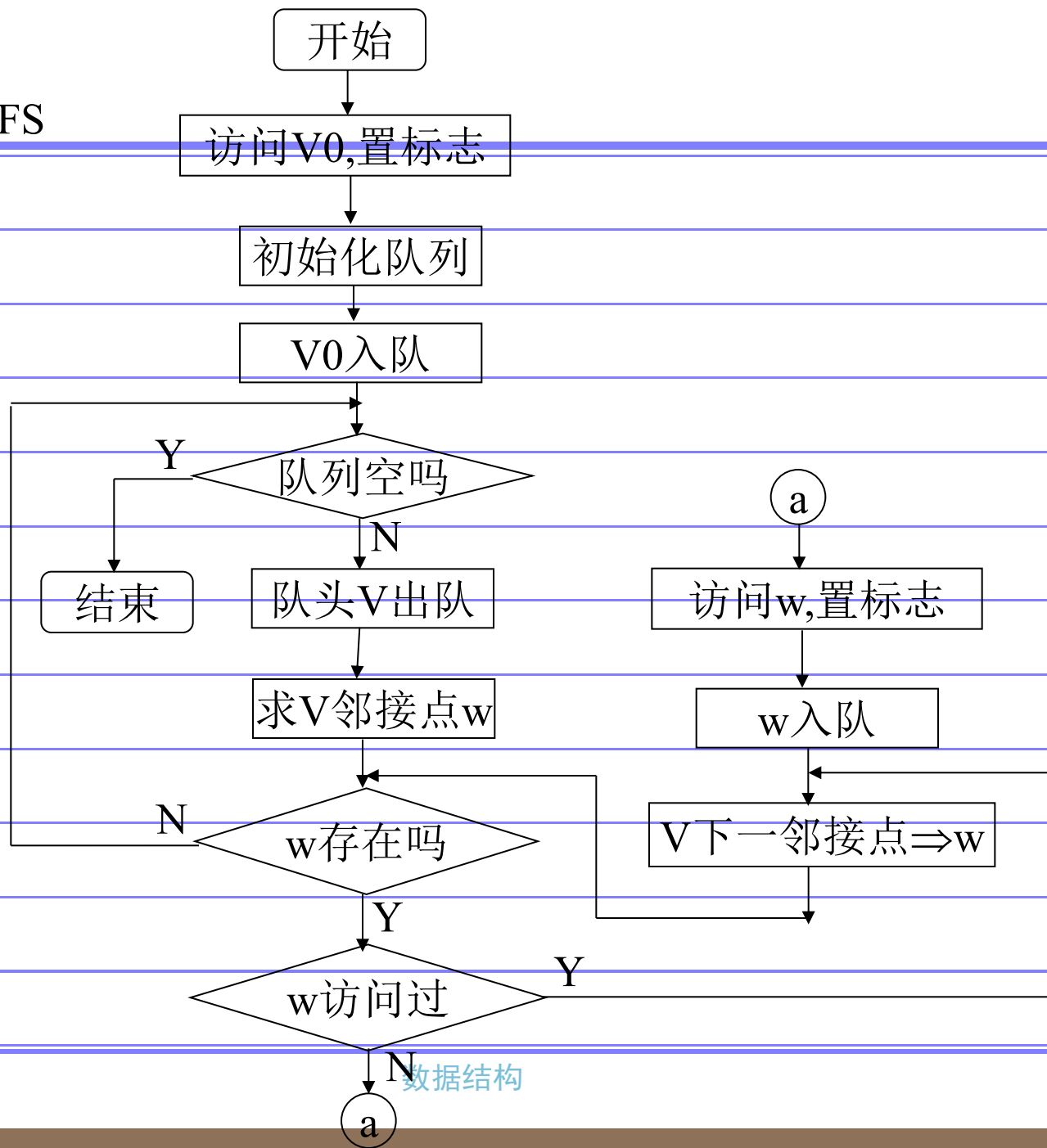


广度遍历: $V1 \Rightarrow V2 \Rightarrow V3 \Rightarrow V4 \Rightarrow V6 \Rightarrow V7 \Rightarrow V8 \Rightarrow V5$

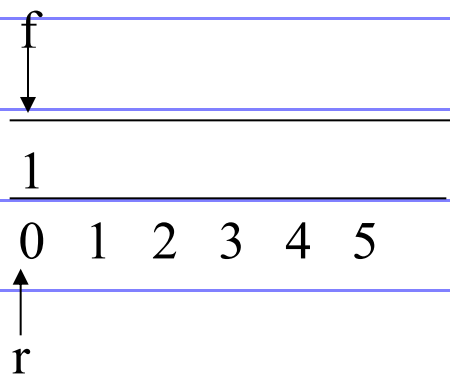
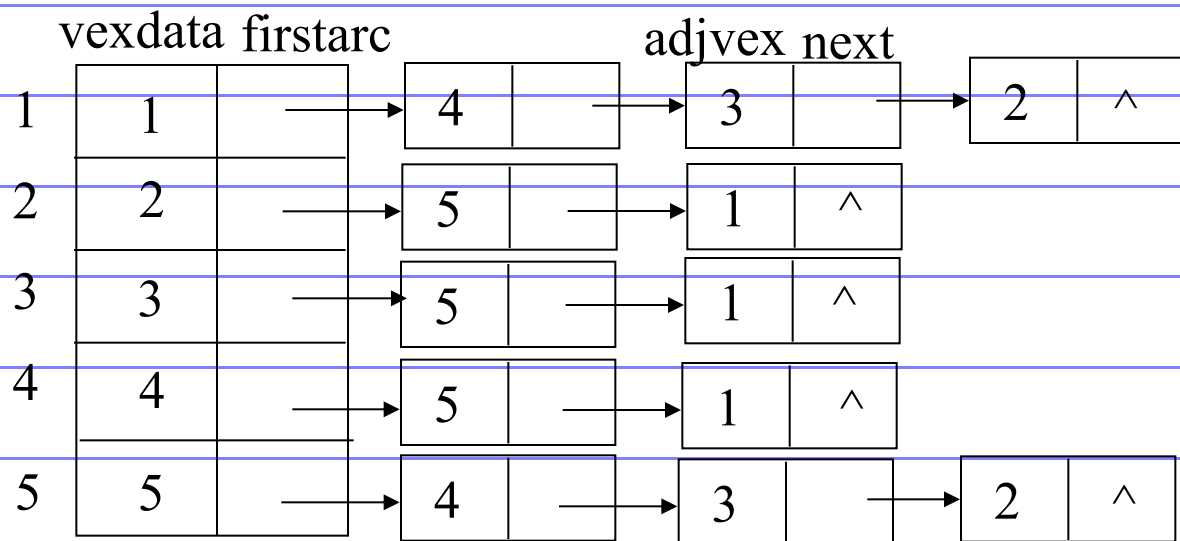
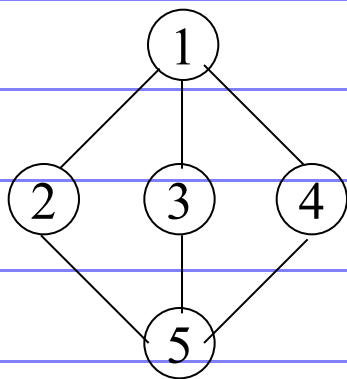
广度优先遍历算法



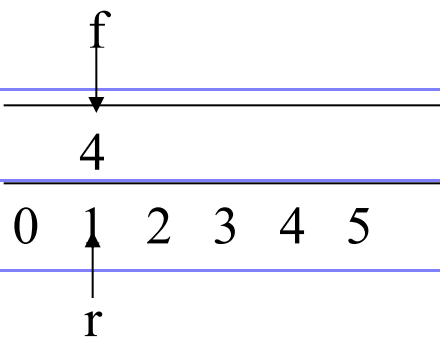
BFS



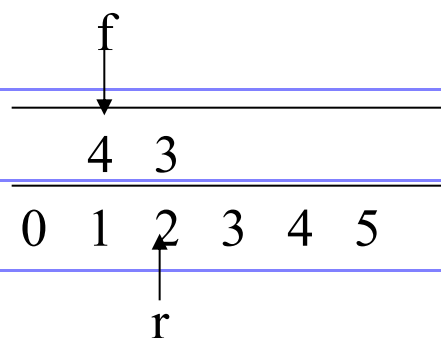
例



遍历序列: 1

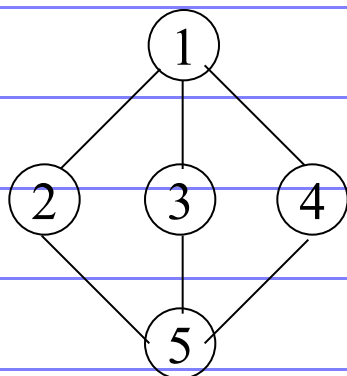


遍历序列: 1 4

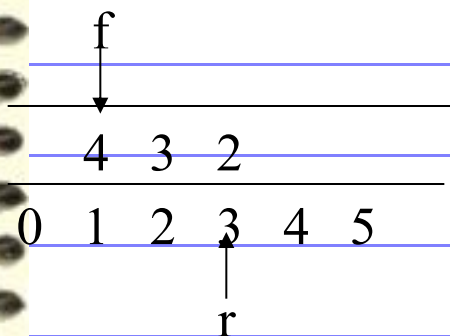


遍历序列: 1 4 3

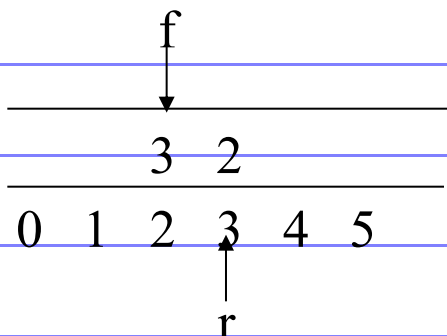
例



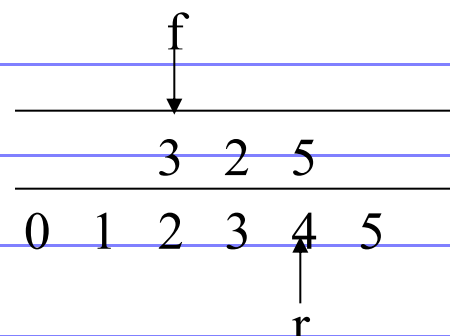
	vexdata	firstarc	adjvex	next
1	1	→ 4	→ 3	→ 2
2	2	→ 5	→ 1	→ ^
3	3	→ 5	→ 1	→ ^
4	4	→ 5	→ 1	→ ^
5	5	→ 4	→ 3	→ 2



遍历序列: 1 4 3 2

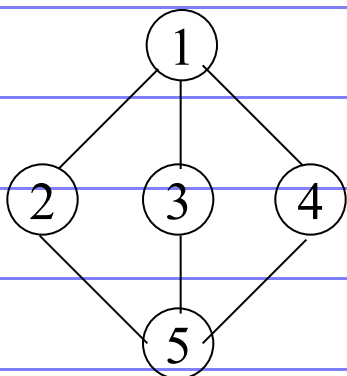


遍历序列: 1 4 3 2

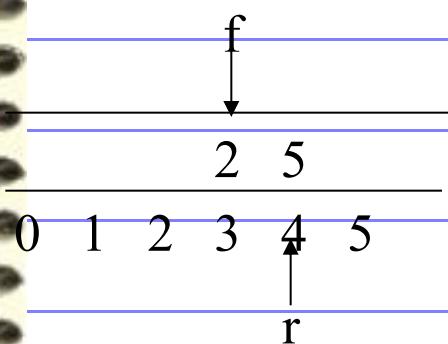


遍历序列: 1 4 3 2 5

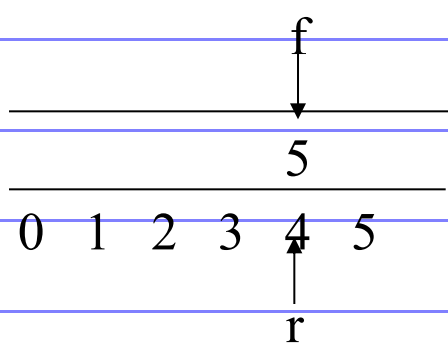
例



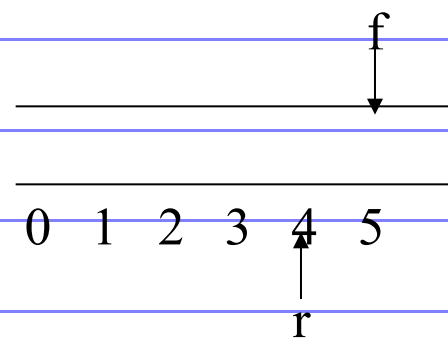
	vexdata	firstarc	adjvex	next
1	1	→ 4	3	→ 2
2	2	→ 5	1	→ ^
3	3	→ 5	1	→ ^
4	4	→ 5	1	→ ^
5	5	→ 4	3	→ 2



遍历序列: 1 4 3 2 5



遍历序列: 1 4 3 2 5



遍历序列: 1 4 3 2 5

基于邻接表的图的广度优先遍历

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTEX_NUM 100 // 图中最大节点数
typedef char VertexType; // 定义节点名为char型
// 边表节点
typedef struct node {
    VertexType adjvex; // 与顶点相连的邻接点下标(adjoin: 邻接)
    struct node* next; // 指向顶点的下一个邻接点
} EdgeNode;

// 顶点结构
typedef struct vnode {
    VertexType vex; // 存储顶点名
    EdgeNode* firstedge; // 边表头指针, 指向顶点第一个邻接点
} VertexNode, AdjList[MAX_VERTEX_NUM];

// 描述图结构的邻接表
typedef struct {
    AdjList adjlist;
    int vexnum; // 节点的数目
    int edgenum; // 边的数目
} ALGraph; // adjacency list: 邻接表

int visited[MAX_VERTEX_NUM]; // 标志数组

void CreateALG(ALGraph* ALG); // 邻接表法创建图
void TraverseALG(ALGraph ALG); // 输出图ALG的邻接表
void BFSTraverseALG(ALGraph ALG); // 广度优先遍历以邻接表存储的图ALG
// 定位节点vertex, 并将其下标赋给index
void LocateVex(ALGraph ALG, VertexType vertex, int* index);

/*-----定义一个循环队列-----*/
#define CQ_INIT_SIZE 100 // 队列初始容量
typedef int dataType;
typedef struct {
    dataType* data; // 存储队列元素
    int front; // 指向队列中第一个元素
    int rear; // 指向队列中最后一个元素下一位置
    int cqCapacity; // 最多能容纳的元素个数(队列容量)
} CQueue;

CQueue* initCQueue(); // 创建一个空循环队列
int push(CQueue* Q, dataType x); // 将元素x入队, 操作成功返回1, 失败返回0
int pop(CQueue* Q, dataType* x); // 队首元素出队, 并将其值赋给x, 操作成功返回1, 失败返回0
int isEmpty(CQueue* Q); // 队列空返回1, 否则返回0
int isFull(CQueue* Q); // 队列满返回1, 否则返回0

int main(void)
{
    ALGraph g;

    CreateALG(&g);
    printf("-----\n");
    printf("vexnum = %d ; edgenum = %d\n", g.vexnum, g.edgenum);
    printf("-----\n");
    TraverseALG(g);
    printf("-----\n");
    BFSTraverseALG(g);

    return 0;
}
```

```

// 建立空队列
CQueue* initCQueue()
{
    CQueue* Q = (CQueue*)malloc(sizeof(CQueue));
    Q->data = (dataType*)malloc(CQ_INIT_SIZE * sizeof(dataType));
    Q->front = 0;
    Q->rear = 0;
    Q->cqCapacity = CQ_INIT_SIZE;

    return Q;
}

int isFull(CQueue* Q)
{
    return (Q->rear + 1) % Q->cqCapacity == Q->front ? 1 : 0;
}

int isEmpty(CQueue* Q)
{
    return Q->front == Q->rear ? 1 : 0;
}

// 入队
int push(CQueue* Q, dataType x)
{
    if (isFull(Q)) {
        // 若达到最大容量，则将新容量扩大至旧容量 1.5 倍
        int increment = Q->cqCapacity / 2;
        Q->data = (dataType*)realloc(Q->data,
            (Q->cqCapacity + increment) * sizeof(dataType));

        if (!Q->data) {
            return 0;
        }
        Q->cqCapacity += increment;
    }
    Q->data[Q->rear] = x;
    Q->rear = (Q->rear + 1) % Q->cqCapacity;
    return 1;
}

```

```

// 出队
int pop(CQueue* Q, dataType* x)
{
    if (isEmpty(Q)) {
        return 0;
    } else {
        *x = Q->data[Q->front];
        Q->front = (Q->front + 1) % Q->cqCapacity;
        return 1;
    }
}

```

基于邻接表的图的广度优先遍历

```
void CreateALG(ALGraph* ALG)
{
    VertexType ch;
    int i = 0, count = 0;
    EdgeNode* temp;

    printf("请输入图的顶点: ");
    // 建立顶点表
    while ((ch = getchar()) != '\n') {
        ALG->adjlist[i].vex = ch;
        ALG->adjlist[i].firstedge = NULL;
        i++;
    }
    ALG->vexnum = i; // 顶点数

    // 头插法建立顶点的邻接边表
    for (i = 0; i < ALG->vexnum; i++) {
        printf("请输入顶点 %c 的邻接顶点: ", ALG->adjlist[i].vex);
        // 按下回车结束邻接点的创建
        while ((ch = getchar()) != '\n') {
            temp = (EdgeNode*)malloc(sizeof(EdgeNode));
            temp->adjvex = ch;
            temp->next = ALG->adjlist[i].firstedge;
            ALG->adjlist[i].firstedge = temp;
            count++;
        }
    }
    ALG->edgenum = count / 2;
    // 无向图中每条边连接两个顶点, 故: 节点总度数 = 边数 * 2
}
```

```
void TraverseALG(ALGraph ALG)
{
    int i;
    EdgeNode* index;

    // 若图为空, 则停止遍历
    if (ALG.vexnum == 0) {
        printf("图为空\n");
        return;
    }

    // 遍历图
    for (i = 0; i < ALG.vexnum; i++) {
        printf("顶点 %c : ", ALG.adjlist[i].vex);
        index = ALG.adjlist[i].firstedge;
        // 以邻接表形式输出图的信息
        while (index) {
            printf("[ %c ] -> ", index->adjvex);
            index = index->next;
        }
        printf("NULL\n");
    }
}
```

```

void BFSTraverseALG(ALGraph ALG)
{
    int i, index; // index为当前访问节点的索引
    char ch;      // 从节点ch开始对图进行BFS搜索
    EdgeNode* temp;
    CQueue* que = initCQueue();

    // 初始化标志数组
    for (i = 0; i < ALG.vexnum; i++) {
        visited[i] = 0;
    }

    printf("请输入开始节点: ");
    scanf("%c", &ch);
    LocateVex(ALG, ch, &index); // 将开始节点ch的下标赋给index

    printf("图的广度优先遍历序列: ");
    if (!visited[index]) {
        push(que, index); // 开始节点入队, 并修改visited数组
        visited[index] = 1;

        // 当队列不空时
        while (!isEmpty(que)) {
            pop(que, &index); // 队首元素出队并访问
            printf("%c, ", ALG.adjlist[index].vex);

            temp = ALG.adjlist[index].firstedge;
            // 将节点的所有邻接点入队
            while (temp) {
                LocateVex(ALG, temp->adjvex, &index);
                // 若节点未被遍历, 则入队并修改visited数组
                if (!visited[index]) {
                    push(que, index);
                    visited[index] = 1;
                }
                temp = temp->next;
            }
        }
    }
}

```

```

void LocateVex(ALGraph ALG, VertexType vertex, int* index)
{
    int i;

    for (i = 0; i < ALG.vexnum; i++) {
        if (ALG.adjlist[i].vex == vertex) {
            *index = i; // 将节点vertex的下标赋给index
            return;
        }
    }
    printf("节点 %c 定位失败! \n", vertex);
}

```

请输入图的顶点: 12345678

请输入顶点 1 的邻接顶点: 32

请输入顶点 2 的邻接顶点: 541

请输入顶点 3 的邻接顶点: 761

请输入顶点 4 的邻接顶点: 82

请输入顶点 5 的邻接顶点: 82

请输入顶点 6 的邻接顶点: 73

请输入顶点 7 的邻接顶点: 63

请输入顶点 8 的邻接顶点: 54

vexnum = 8 ; edgenum = 9

顶点 1 : [2] -> [3] -> NULL

顶点 2 : [1] -> [4] -> [5] -> NULL

顶点 3 : [1] -> [6] -> [7] -> NULL

顶点 4 : [2] -> [8] -> NULL

顶点 5 : [2] -> [8] -> NULL

顶点 6 : [3] -> [7] -> NULL

顶点 7 : [3] -> [6] -> NULL

顶点 8 : [4] -> [5] -> NULL

请输入开始节点: 1

图的广度优先遍历序列: 1, 2, 3, 4, 5, 6, 7, 8, Hit any key to close this window...

图的连通性问题

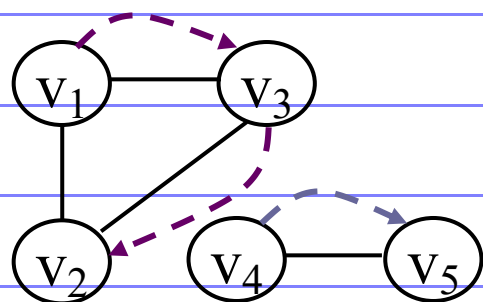
无向图的连通分量与生成树

对于无向图，对其进行遍历时：

- ◆ 若是**连通图**：仅需从图中**任一顶点出发**，就能访问图中的所有顶点；
- ◆ 若是**非连通图**：需从图中**多个顶点出发**。每次从一个新顶点出发所访问的顶点集序列恰好是各个连通分量的顶点集；

图的连通性问题

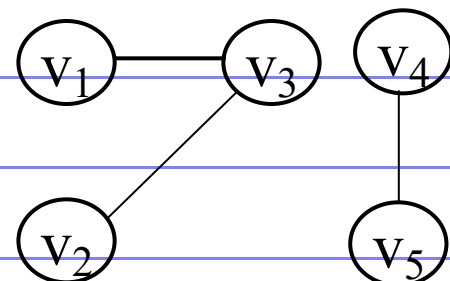
如下图所示的无向图是非连通图，按图中给定的邻接表进行深度优先搜索遍历，2次调用DFS所得到的顶点访问序列集是：{ v1 ,v3 ,v2 }和{ v4 ,v5 }



(a) 无向图G

0	v ₁	→	2	→	1	Λ
1	v ₂	→	2	→	0	Λ
2	v ₃	→	0	→	1	Λ
3	v ₄	→	4	Λ		
4	v ₅	→	3	Λ		
	⋮					
	MAX_VEX-1					

(b) G的邻接链表



(c) 深度优先生成森林

图 无向图及深度优先生成森林

图的连通性问题

(1) 若 $G=(V,E)$ 是无向连通图，顶点集和边集分别是 $V(G)$ ， $E(G)$ 。若从 G 中任意点出发遍历时， $E(G)$ 被分成两个互不相交的集合：

$T(G)$ ：遍历过程中所经过的边的集合；

$B(G)$ ：遍历过程中未经过的边的集合；

显然： $E(G)=T(G)\cup B(G)$ ， $T(G)\cap B(G)=\emptyset$

显然，图 $G'=(V, T(G))$ 是 G 的极小连通子图，且 G' 是一棵树。 G' 称为图 G 的一棵生成树。

从任意点出发按DFS算法得到生成树 G' 称为深度优先生成树；按BFS算法得到的 G' 称为广度优先生成树。

6.4 生成树

□ 所有顶点均由边连接在一起，但不存在回路的图

深度优先生成树与广度优先生成树

□ 生成森林：非连通图每个连通分量的生成树一起组成非连通图的~

◇ 说明 一个图可以有許多棵不同的生成树

■ 所有生成树具有以下共同特点：

◇ 生成树的顶点个数与图的顶点个数相同

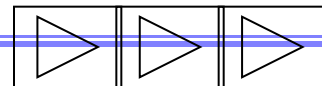
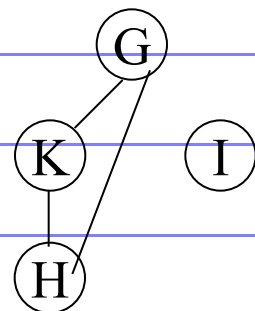
◇ 生成树是图的极小连通子图

◇ 一个有 n 个顶点的连通图的生成树有 $n-1$ 条边

◇ 生成树中任意两个顶点间的路径是唯一的

◇ 在生成树中再加一条边必然形成回路

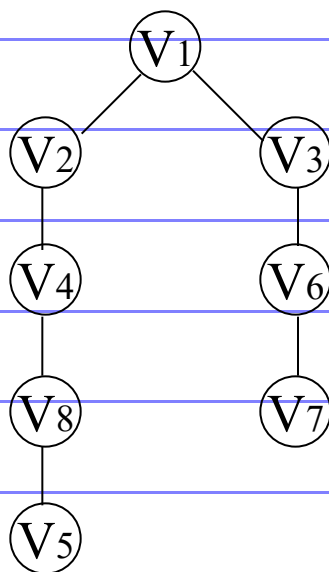
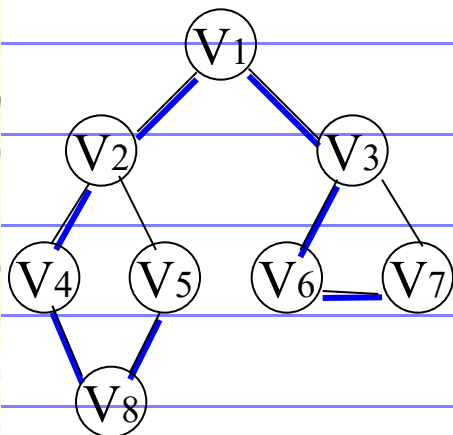
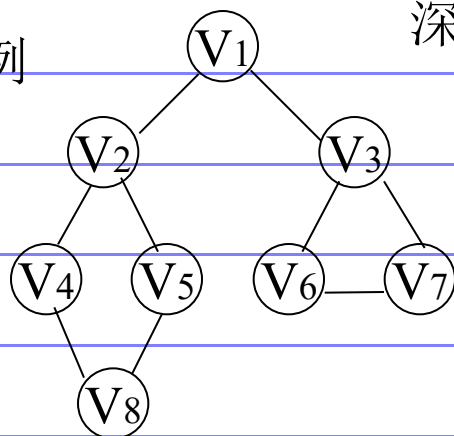
■ 含 n 个顶点 $n-1$ 条边的图不一定是生成树



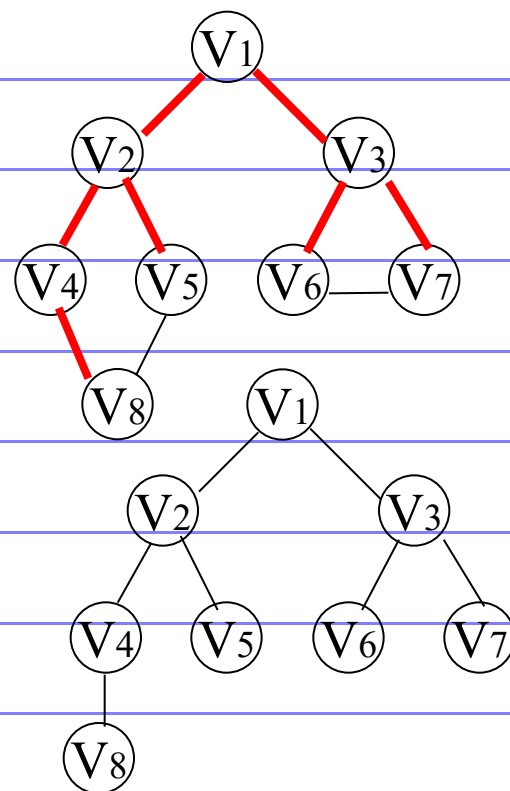
广度遍历: $V1 \Rightarrow V2 \Rightarrow V3 \Rightarrow V4 \Rightarrow V5 \Rightarrow V6 \Rightarrow V7 \Rightarrow V8$

深度遍历: $V1 \Rightarrow V2 \Rightarrow V4 \Rightarrow V8 \Rightarrow V5 \Rightarrow V3 \Rightarrow V6 \Rightarrow V7$

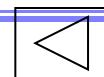
例



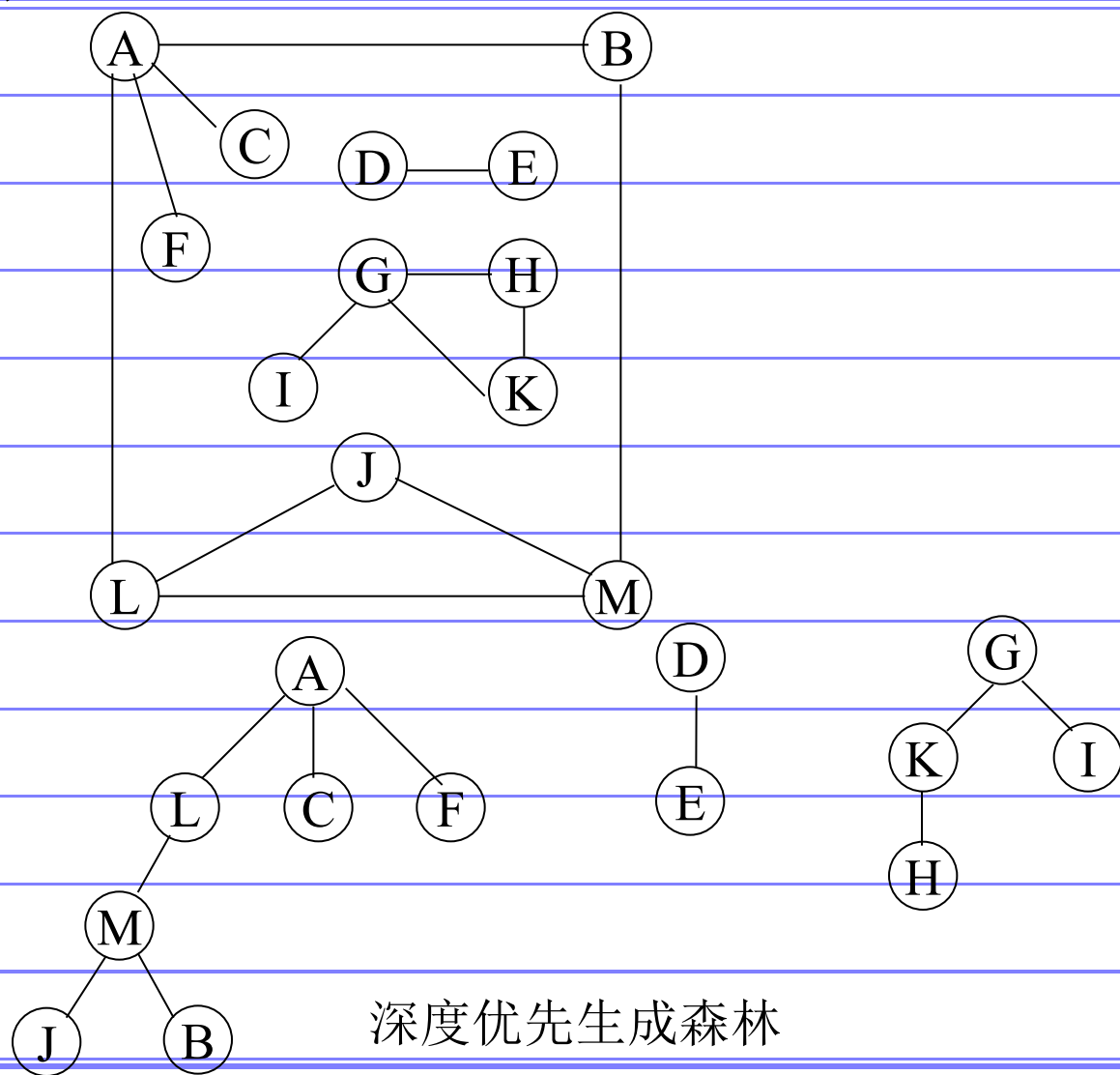
深度优先生成树



广度优先生成树

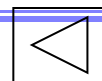


例



深度优先生成森林

数据结构



最小生成树

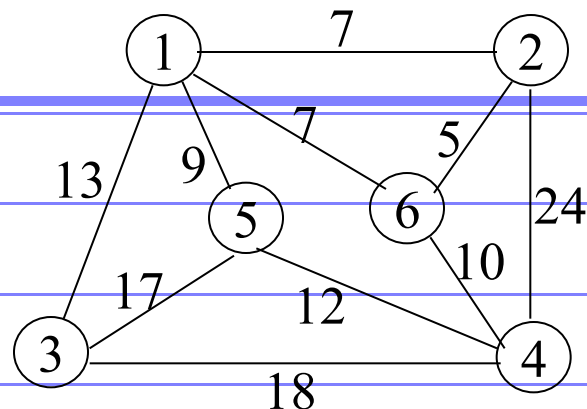
□ 问题提出

要在 n 个城市间建立通信联络网，

顶点——表示城市

权——城市间建立通信线路所需花费代价

希望找到一棵生成树，它的每条边上的权值之和（即建立该通信网所需花费的总代价）最小——最小代价生成树。



□ 问题分析

n 个城市间，最多可设置 $n(n-1)/2$ 条线路

n 个城市间建立通信网，只需 $n-1$ 条线路

问题转化为：如何在可能的线路中选择 $n-1$ 条，能把所有城市（顶点）均连起来，且总耗费（各边权值之和）最小。

□ 构造最小生成树方法

■ 方法一：普里姆(Prim)算法

✧ 算法思想：设 $N=(V, \{E\})$ 是连通网， TE 是 N 上最小生成树中边的集合

✧ 初始令 $U=\{u_0\}, (u_0 \in V), TE=\Phi$

✧ 在所有 $u \in U, v \in V-U$ 的边 $(u, v) \in E$ 中，找一条代价最小的边 (u_0, v_0)

✧ 将 (u_0, v_0) 并入集合 TE ，同时 v_0 并入 U

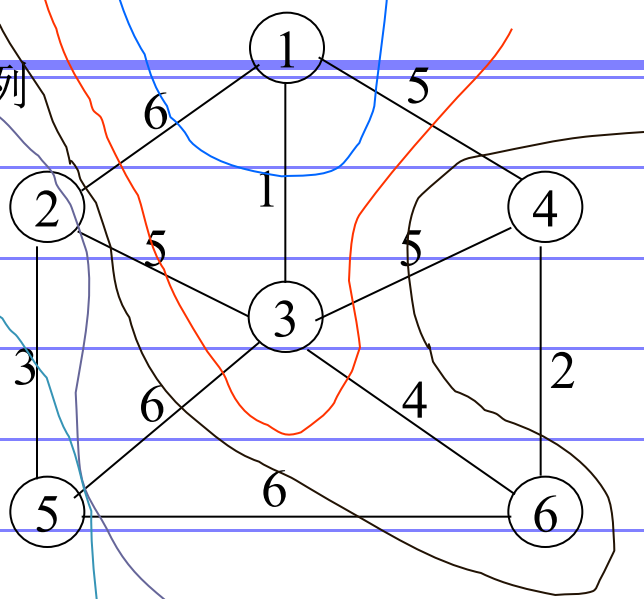
✧ 重复上述操作直至 $U=V$ 为止，则 $T=(V, \{TE\})$ 为 N 的最小生成树

✧ 算法实现：图用邻接矩阵表示

✧ 算法描述

✧ 算法评价： $T(n)=O(n^2)$

例



1

1



4



1

1

1



5

5



4

4

4



```
#include <iostream>
```

```
#define MaxVerTexNum 20
```

```
typedef char VertexType;
```

```
typedef int EdgeType;
```

```
// 邻接矩阵存储图
```

```
typedef struct {
```

```
    VertexType Vex[MaxVerTexNum];
```

```
    EdgeType Edge[MaxVerTexNum][MaxVerTexNum];
```

```
    int vexnum, arcnum;
```

```
} MGraph;
```

```
void MiniSpanTree_Prim(MGraph G, int u);
```

```
void Visit(const MGraph& G, int u, int v);
```

```
int main() {
```

```
    MGraph G = {
```

```
        {'1', '2', '3', '4', '5', '6'},
```

```
        {
```

```
            {-1, 6, 1, 5, -1, -1},
```

```
            {6, -1, 5, -1, 3, -1},
```

```
            {1, 5, -1, 5, 6, 4},
```

```
            {5, -1, 5, -1, -1, 2},
```

```
            {-1, 3, 6, -1, -1, 6},
```

```
            {-1, -1, 4, 2, 6, -1}
```

```
        },
```

```
        6, 10
```

```
    };
```

```
    MiniSpanTree_Prim(G, 0);
```

```
    system("pause");
```

```
    return 0;
```

```
}
```

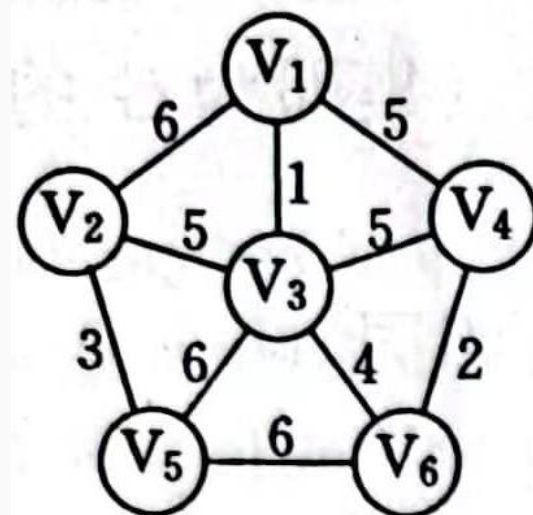
```
// 顶点的数据类型
```

```
// 带权图中边上权值的数据类型
```

```
// 顶点表
```

```
// 邻接矩阵，边表(-1代表无穷大)
```

```
// 图的当前顶点数和边数/弧数
```



```

// 从顶点u出发，构造最小生成树，并输出生成树的每条边
void MiniSpanTree_Prim(MGraph G, int u) {
    struct {
        int adjvex;           // 顶点编号
        int lowcost;          // 从顶点adjvex到当前顶点的权值（为0代表已加入集合U）
    } closedge[MaxVerTexNum]; // 求最小生成树时的辅助数组

    for (int i = 0; i < G.vexnum; ++i) {
        closedge[i].adjvex = u;
        closedge[i].lowcost = G.Edge[u][i];
    }

    closedge[u].lowcost = 0; // 初始化，U = {u}

    for (int i = 1; i < G.vexnum; ++i) { // 找n-1条边
        int min = -1; // closedge数组中的最小权值
        int v; // 下一个加入集合U的顶点

        for (int j = 0; j < G.vexnum; j++) { // 找到closedge数组中的最小边
            if (min == -1 && closedge[j].lowcost > 0) {
                min = closedge[j].lowcost;
                v = j;
            }
            if (min != -1 && closedge[j].lowcost > 0 && closedge[j].lowcost < min) {
                min = closedge[j].lowcost;
                v = j;
            }
        }

        closedge[v].lowcost = 0; // 将顶点v纳入集合U
        int u = closedge[v].adjvex;
        Visit(G, u, v);

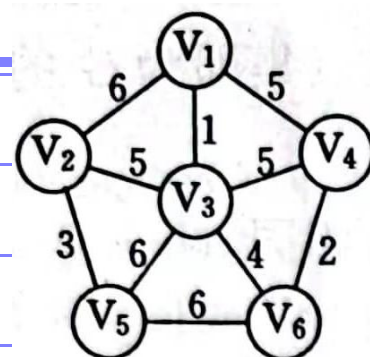
        for (int j = 0; j < G.vexnum; ++j) { // 在顶点v并入U之后，更新closedge[i]
            if (closedge[j].lowcost != 0 && G.Edge[v][j] > 0) {
                if (closedge[j].lowcost == -1) {
                    closedge[j].adjvex = v;
                    closedge[j].lowcost = G.Edge[v][j];
                }
                else if (closedge[j].lowcost != -1 && G.Edge[v][j] < closedge[j].lowcost) {
                    closedge[j].adjvex = v;
                    closedge[j].lowcost = G.Edge[v][j];
                }
            }
        }
    }
}

```

```

void Visit(const MGraph& G, int u, int v) {
    std::cout << G.Vex[u] << " -> " << G.Vex[v] << " : " << G.Edge[u][v] << std::endl;
}

```



i \ closedge	1	2	3	4	5	U	V-U	k
adjvex	v1	v1	v1			{v1}	{v2, v3, v4, v5, v6}	2
lowcost	6	1	5					
adjvex	v3		v1	v3	v3	{v1, v3}	{v2, v4, v5, v6}	5
lowcost	5	0	5	6	4			
adjvex	v3		v6	v3		{v1, v3, v6}	{v2, v4, v5}	3
lowcost	5	0	2	6	0			
adjvex	v3			v3		{v1, v3, v6, v4}	{v2, v5}	1
lowcost	5	0	0	6	0			
adjvex				v2		{v1, v3, v6, v4, v2}	{v5}	4
lowcost	0	0	0	3	0			
adjvex						{v1, v3, v6, v4, v2, v5}	{}	
lowcost	0	0	0	0	0			

□ 方法二：克鲁斯卡尔(Kruskal)算法

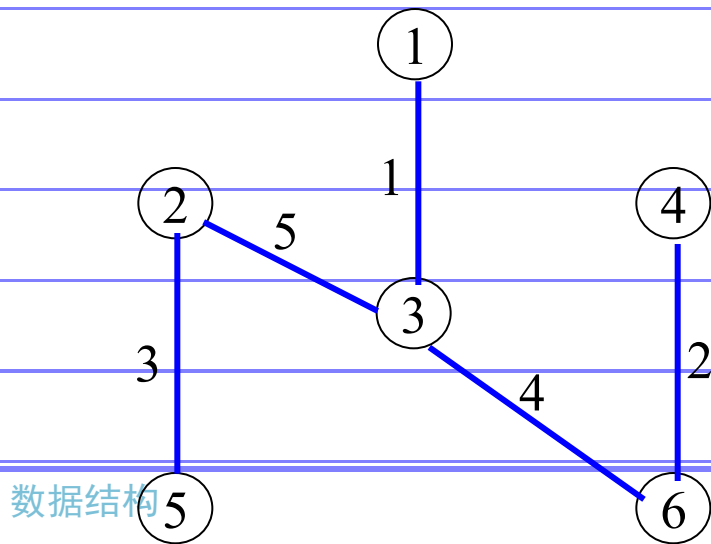
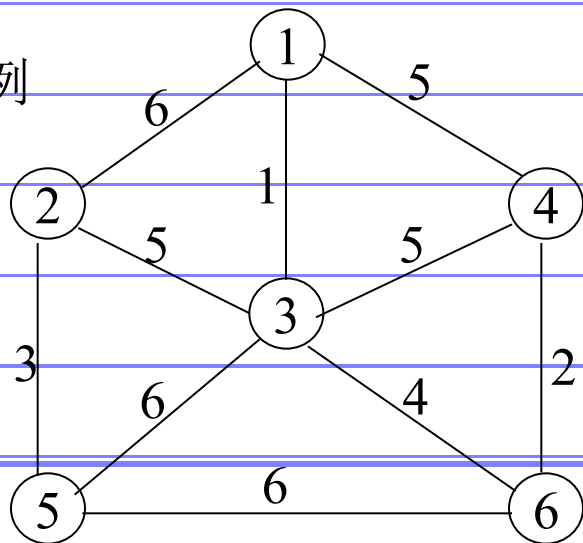
■ 算法思想：设连通网 $N=(V, \{E\})$ ，令最小生成树

☆ 初始状态为只有 n 个顶点而无边的非连通图 $T=(V, \{\Phi\})$ ，每个顶点自成一个连通分量

☆ 在 E 中选取代价最小的边，若该边依附的顶点落在 T 中不同的连通分量上，则将此边加入到 T 中；否则，舍去此边，选取下一条代价最小的边

☆ 依此类推，直至 T 中所有顶点都在同一连通分量上为止

例



✧算法实现:

顶点结点:

```
typedef struct
{   int data; //顶点信息
    int jihe;
}VEX;
```

边结点:

```
typedef struct
{   int vexh, vext; //边依附的两顶点
    int weight;     //边的权值
    int flag;       //标志域
}EDGE;
```

0) 用顶点数组和边数组存放顶点和边信息

1) 初始时, 令每个顶点的jihe互不相同; 每个边的flag为0

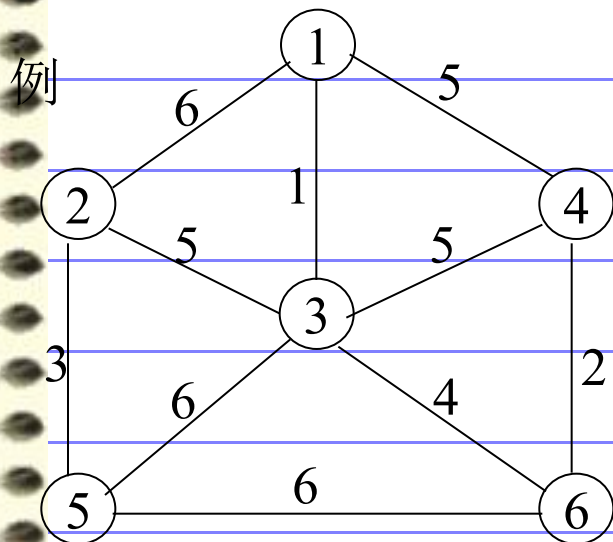
2) 选出权值最小且flag为0的边

3) 若该边依附的两个顶点的jihe值不同, 即非连通, 则令该边的flag=1, 选中该边; 再令该边依附的两顶点的jihe以及两集合中所有顶点的jihe 相同, 若该边依附的两个顶点的jihe值相同, 即连通, 则令该边的flag=2, 即舍去该边

4) 重复上述步骤, 直到选出n-1条边为止

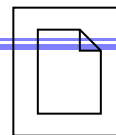
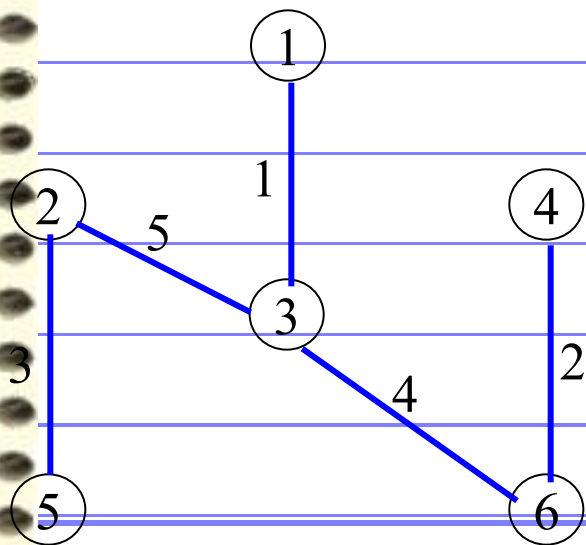
✧ 算法描述:

Ch6_30.txt



	data	jihe
1	1	2
2	2	2
3	3	1 2
4	4	1 2
5	5	3
6	6	4 1

	vexh	vext	weight	flag
0	1	2	6	0
1	1	3	1	0
2	1	4	5	0
3	2	3	5	0
4	2	5	3	0
5	3	4	5	0
6	3	5	6	0
7	3	6	4	0
8	4	6	2	0
9	5	6	6	0



```

#define MAX 0x7fffffff
#define M 30
typedef struct
{
    int data;
    int jihe;
}V;
typedef struct
{
    int vexh, vext;
    int weight;
    int flag;
}E;
void minitree_KRUSKAL(void)
{
    int n,i,m,min,k,j;
    V t[M];
    E e[M];
    printf("请输入顶点的个数:");
    scanf("%d",&n);
    printf("请输入data");
    for(i=0;i<n;i++)//建立点的集合
    {
        scanf("%d",&t[i].data);
        t[i].jihe=i;
    }
    printf("请输入边的条数:");
    scanf("%d",&m);
    printf("请输入vexh,vext,weight:");
    for(i=0;i<m;i++)//建立边的集合
    {
        scanf("%d%d%d",&e[i].vexh,&e[i].vext,&e[i].weight);
        e[i].flag=0;
    }
}

```

```

i=1;
while(i<n)
{
    min=MAX;
    for(j=0; j<m; j++)
    {
        if(e[j].weight<min && e[j].flag==0)
        {
            min=e[j].weight;
            k=j;
        }
    }
    if(t[e[k].vexh].jihe != t[e[k].vext].jihe)
    {
        e[k].flag=1;
        int temp = t[e[k].vext].jihe;
        for(j=0;j<n;j++)
            if(t[j].jihe == temp)
                t[j].jihe=t[e[k].vexh].jihe;
        i++;
    }
    else
        e[k].flag=2;
}
for(i=0;i<m;i++)
    if(e[i].flag==1)
        printf("%d,%d :%d",e[i].vexh,e[i].vext,e[i].weight);
}

```

有向无环图及其应用

有向无环图(Directed Acycling Graph): 是图中没有回路(环)的有向图。是一类具有代表性的图, 主要用于研究工程项目的工序问题、工程时间进度问题等。

一个工程(project)都可分为若干个称为活动(active)的子工程(或工序), 各个子工程受到一定的条件约束: 某个子工程必须开始于另一个子工程完成之后; 整个工程有一个开始点(起点)和一个终点。人们关心:

- ◆ 工程能否顺利完成? 影响工程的关键活动是什么?
- ◆ 估算整个工程完成所必须的最短时间是多少?

对工程的活动加以抽象: 图中顶点表示活动, 有向边表示活动之间的优先关系, 这样的有向图称为**顶点表示活动的网(Activity On Vertex Network, AOV网)**。

6.5 拓扑排序

□ 问题提出：学生选修课程问题

顶点——表示课程

有向弧——表示先决条件，若课程 i 是课程 j 的先决条件，则图中有弧 $\langle i, j \rangle$

学生应按怎样的顺序学习这些课程，才能无矛盾、顺利地完成学业——拓扑排序

□ 定义

■ AOV网——用顶点表示活动，用弧表示活动间优先关系的有向图称为顶点表示活动的网(Activity On Vertex network)，简称AOV网

■ 若 $\langle v_i, v_j \rangle$ 是图中有向边，则 v_i 是 v_j 的直接前驱； v_j 是 v_i 的直接后继

■ AOV网中不允许有回路，即某项活动以自己为先决条件

■ 拓扑排序——把AOV网络中各顶点按照它们相互之间的优先关系排列成一个线性序列的过程叫~

✧检测AOV网中是否存在环方法：对有向图构造其顶点的拓扑有序序列，若网中所有顶点都在它的拓扑有序序列中，则该AOV网必定不存在环

■ 拓扑排序的方法

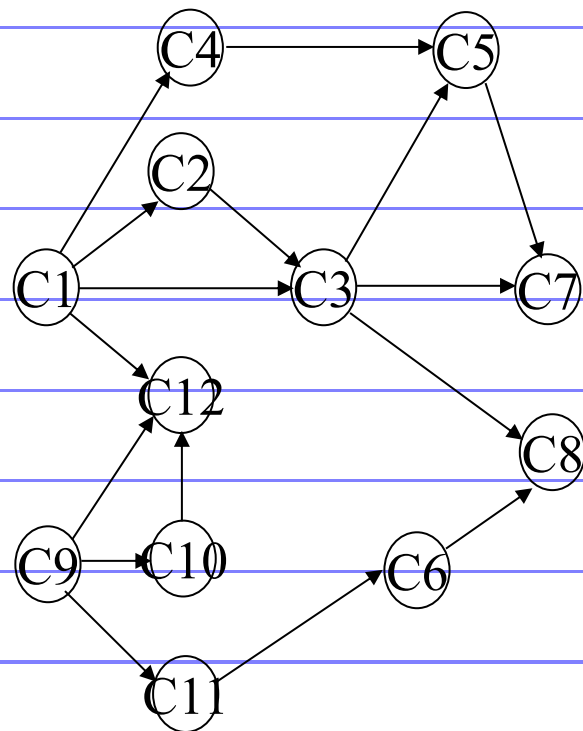
✧在有向图中选一个没有前驱的顶点且输出之

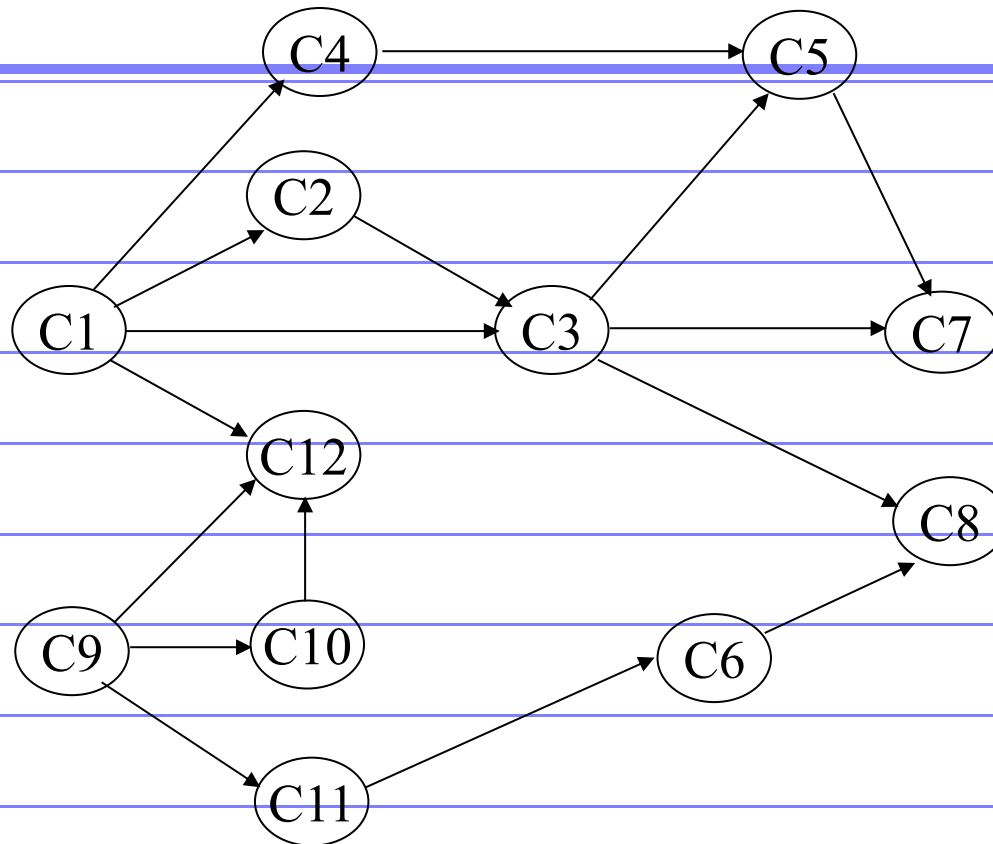
✧从图中删除该顶点和所有以它为尾的弧

✧重复上述两步，直至全部顶点均已输出；或者当图中不存在无前驱的顶点为止

例

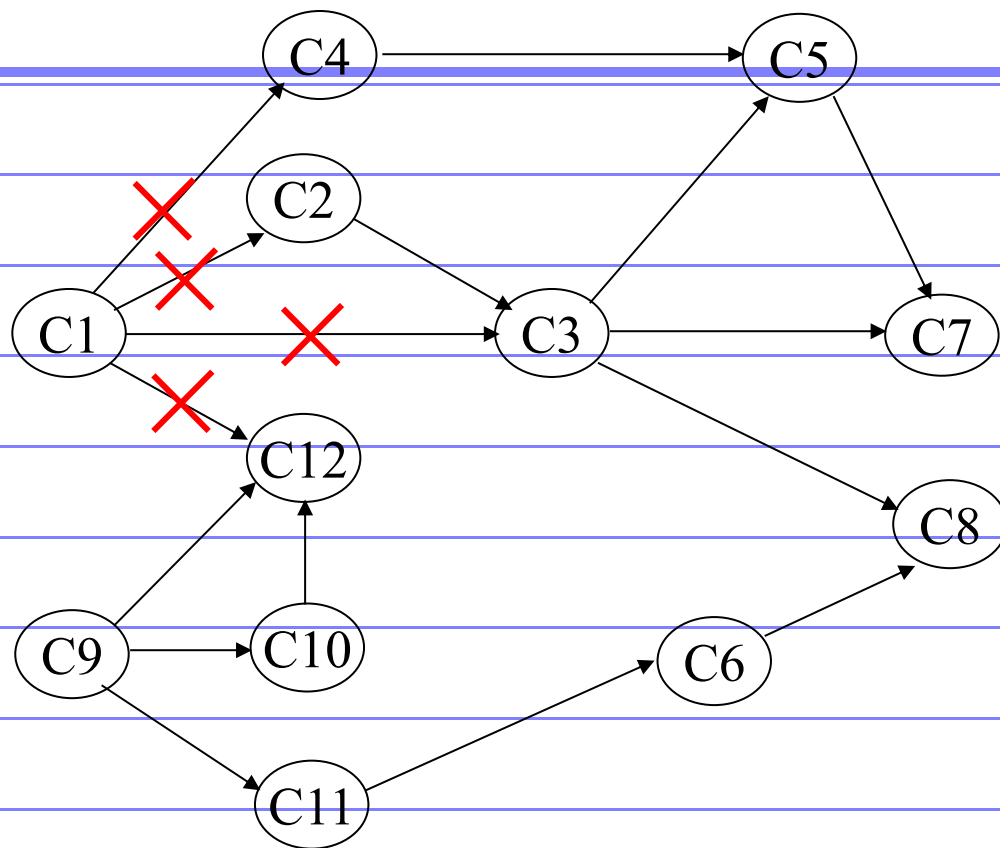
课程代号	课程名称	先修课
C1	程序设计基础	无
C2	离散数学	C1
C3	数据结构	C1,C2
C4	汇编语言	C1
C5	语言的设计和分析	C3,C4
C6	计算机原理	C11
C7	编译原理	C3,C5
C8	操作系统	C3,C6
C9	高等数学	无
C10	线性代数	C9
C11	普通物理	C9
C12	数值分析	C1,C9,C10

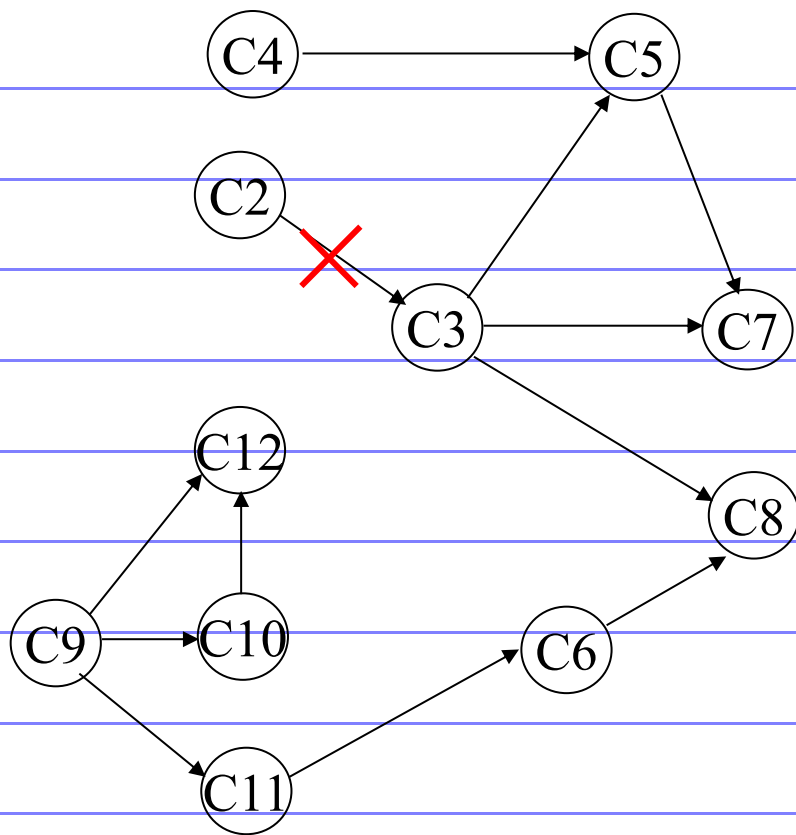




拓扑序列: C1--C2--C3--C4--C5--C7--C9--C10--C11--C6--C12--C8
或 : C9--C10--C11--C6--C1--C12--C4--C2--C3--C5--C7--C8

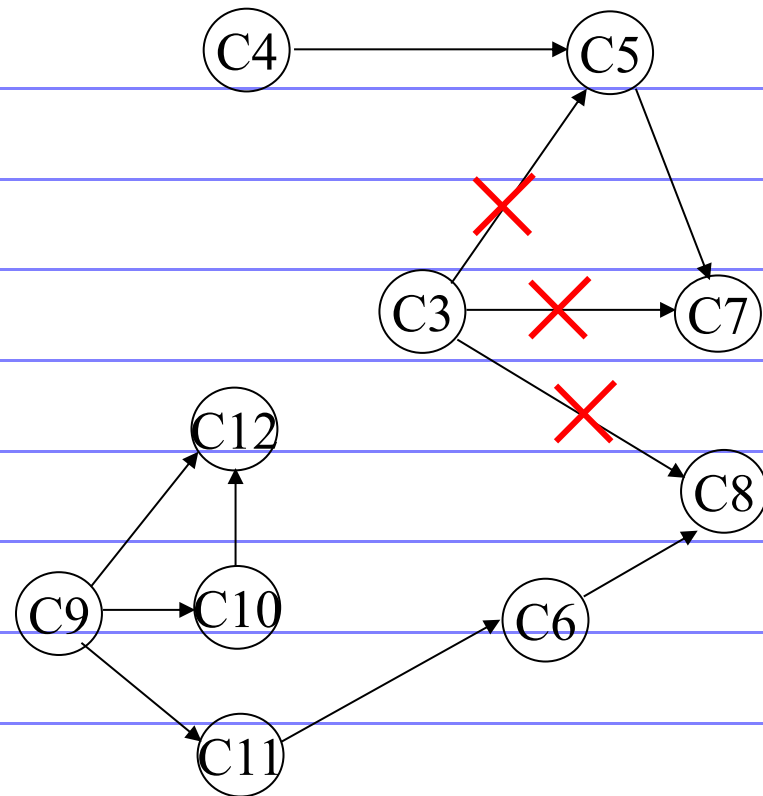
一个AOV网的拓扑序列不是唯一的





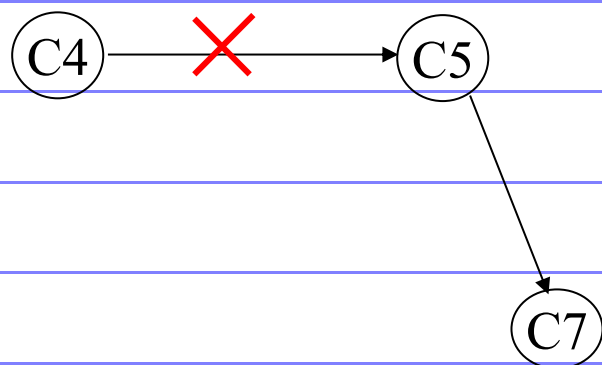
拓扑序列: C1

(1)

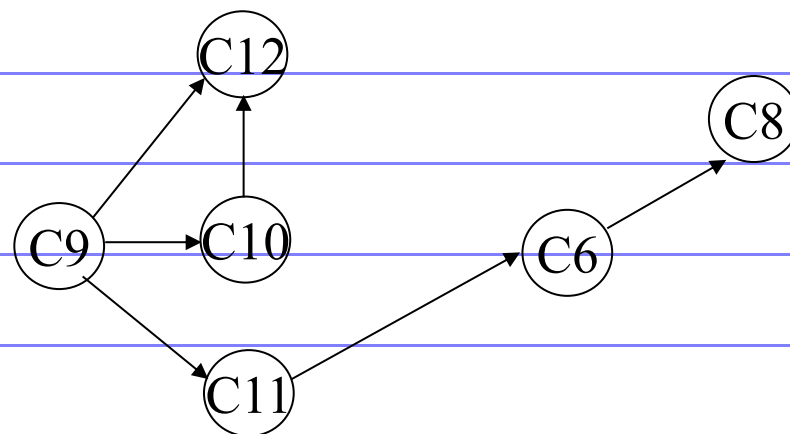
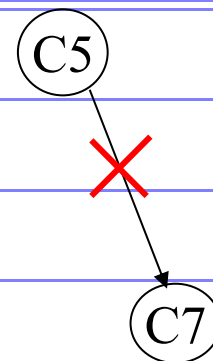


拓扑序列: C1--C2

(2)

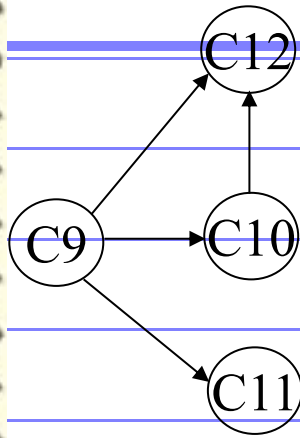


拓扑序列: C1--C2--C3
(3)

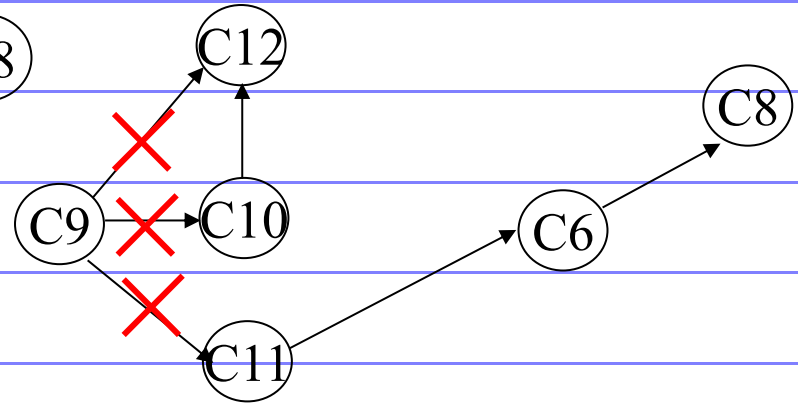


拓扑序列: C1--C2--C3--C4
(4)

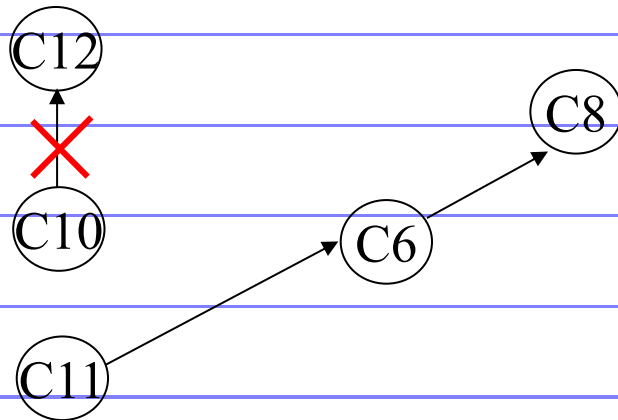
(C7)



拓扑序列: C1--C2--C3--C4--C5
(5)

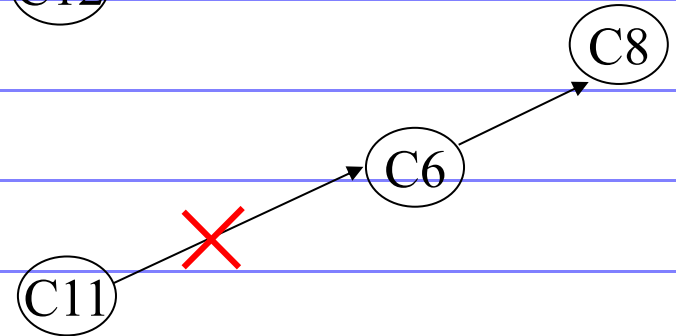


拓扑序列: C1--C2--C3--C4--C5--C7
(6)



拓扑序列: C1--C2--C3--C4--C5--C7--C9

(C12)



拓扑序列: C1--C2--C3--C4--C5--C7--C9
--C10

(8)

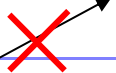
C12

C12

C8

C6

C8



(10)

(9)

拓扑序列: C1--C2--C3--C4--C5--C7--C9

拓扑序列: C1--C2--C3--C4--C5--C7--C9

--C10--C11--C6

--C10--C11

C8

拓扑序列: C1--C2--C3--C4--C5--C7--C9

(11)

--C10--C11--C6--C12--C8

拓扑序列: C1--C2--C3--C4--C5--C7--C9

--C10--C11--C6--C12

数据结构

(12)

■ 算法实现

◇以邻接表作存储结构

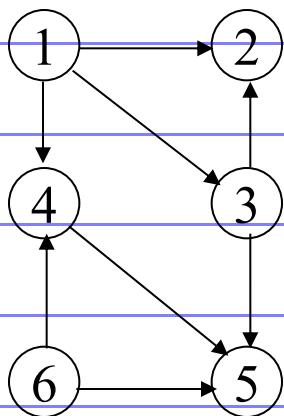
◇把邻接表中所有入度为0的顶点进栈

◇栈非空时，输出栈顶元素 V_j 并退栈；在邻接表中查找 V_j 的直接后继 V_k ，把 V_k 的入度减1；若 V_k 的入度为0则进栈

◇重复上述操作直至栈空为止。若栈空时输出的顶点个数不是 n ，则有向图有环；否则，拓扑排序完毕

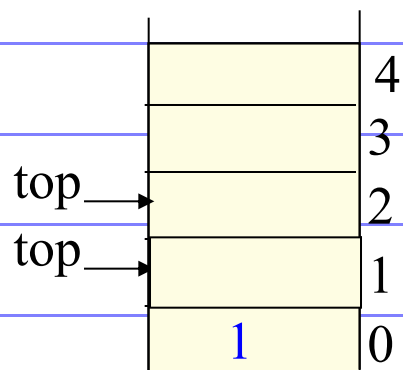
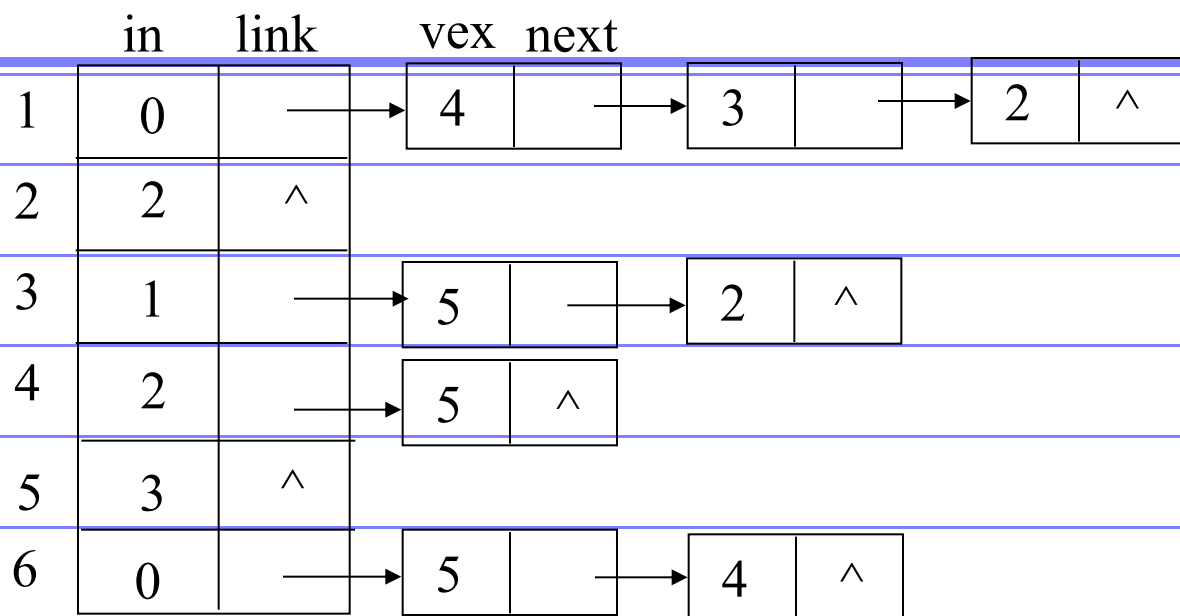
■ 算法描述

例



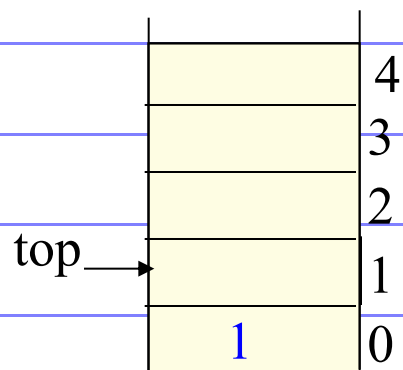
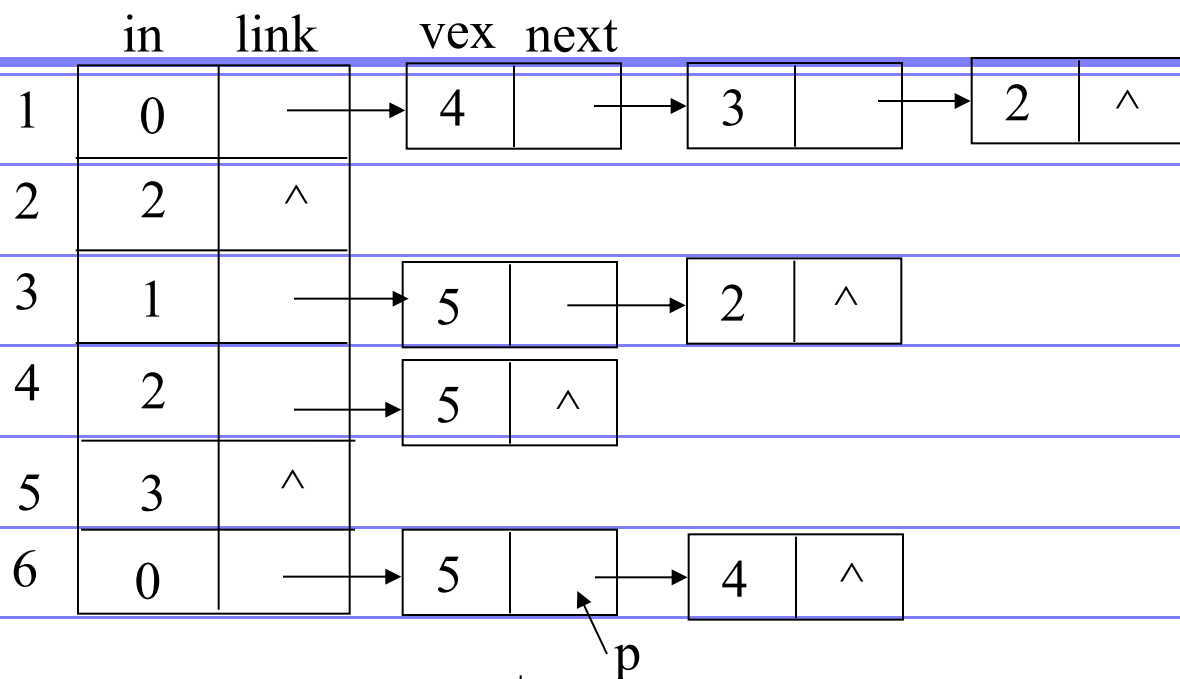
	in	link	vex	next
1	0	—	4	—
2	2	^	3	—
3	1	—	5	—
4	2	—	5	^
5	3	^	2	^
6	0	—	5	—

		4
		3
top	→	2
top	→	1
top	→	0



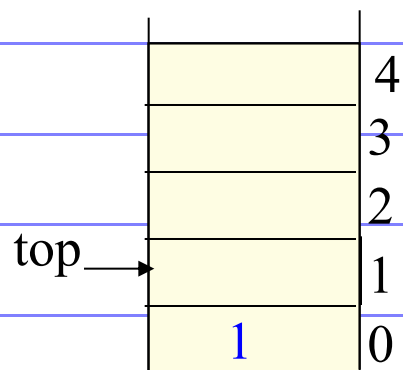
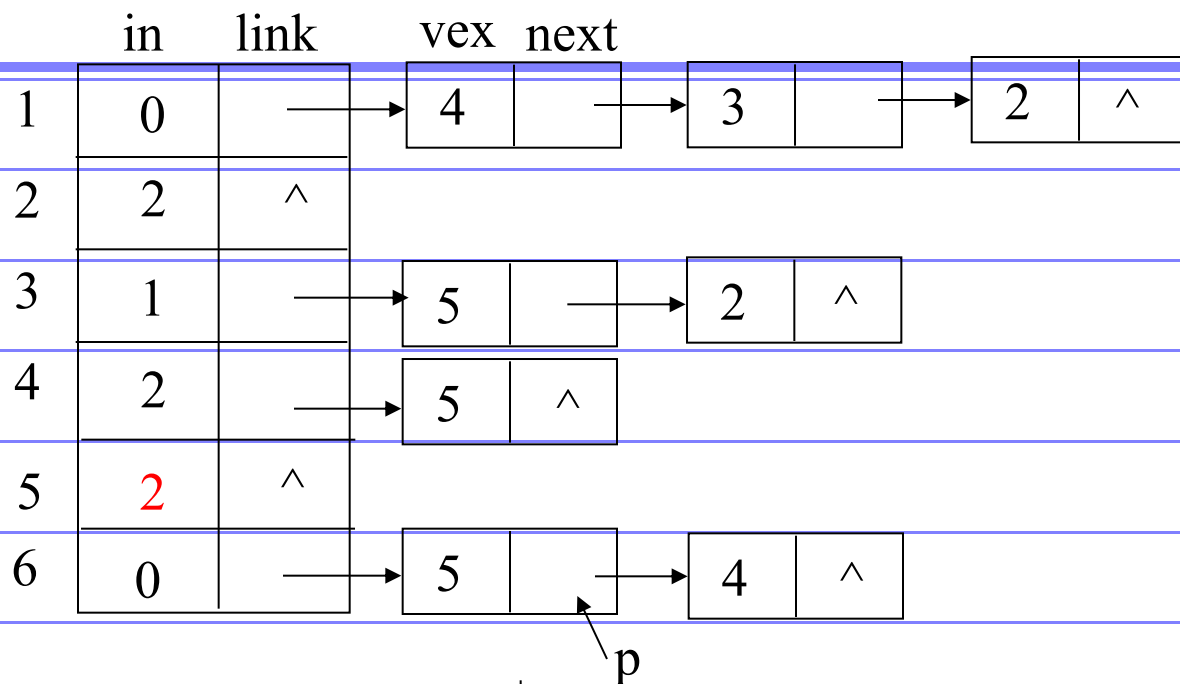
输出序列: 6

数据结构



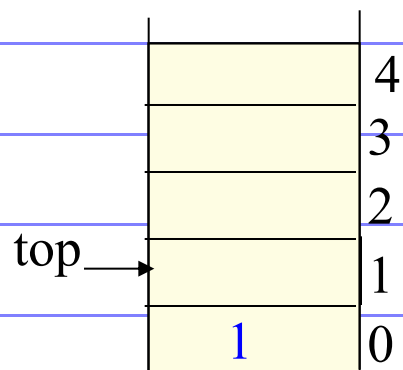
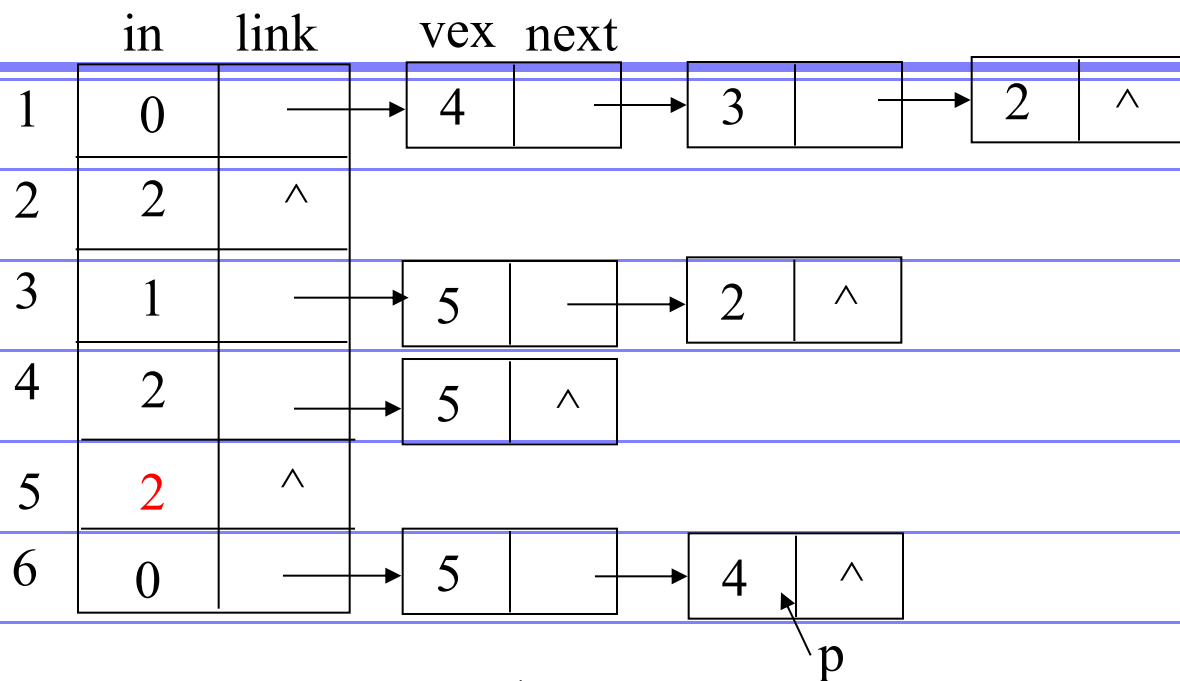
输出序列: 6

数据结构



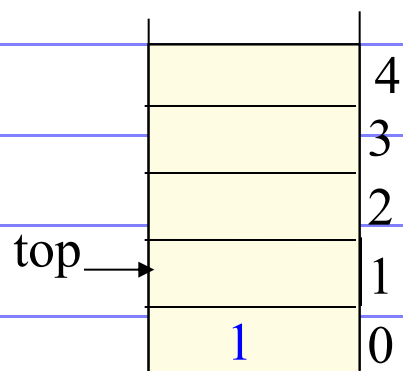
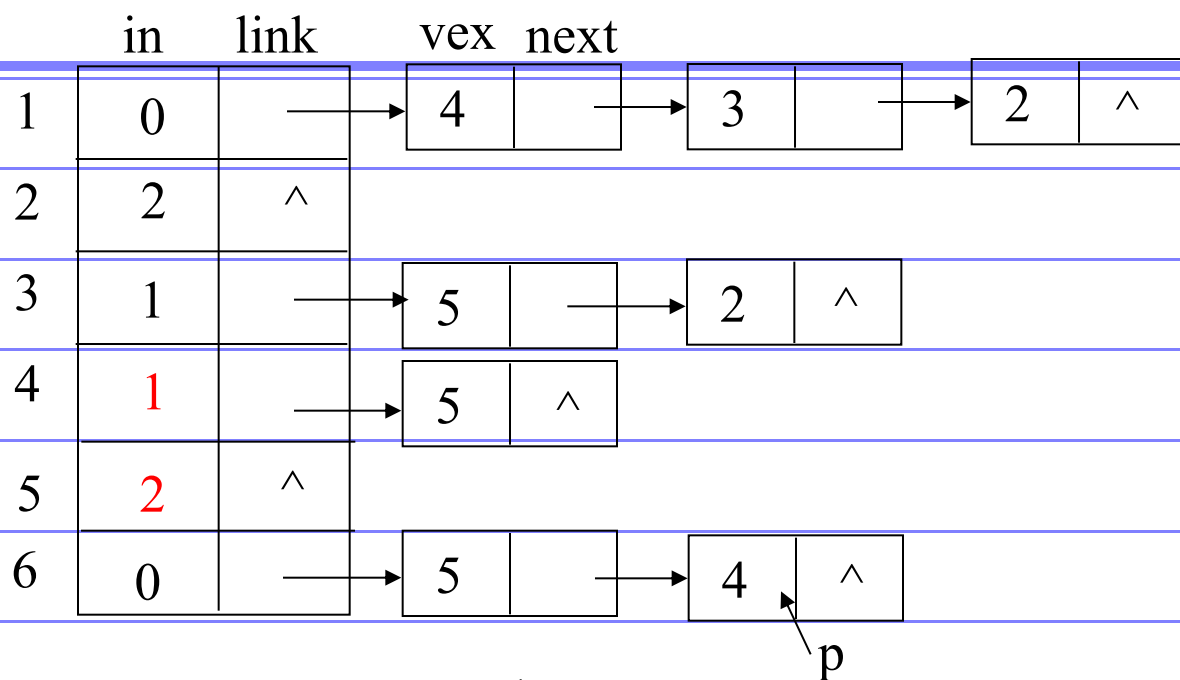
输出序列: 6

数据结构



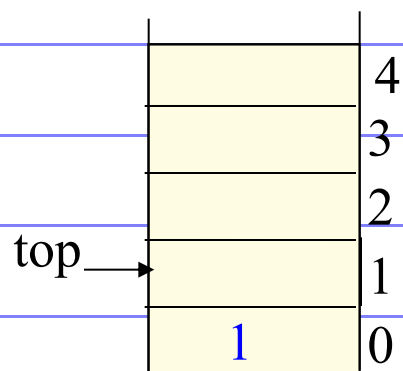
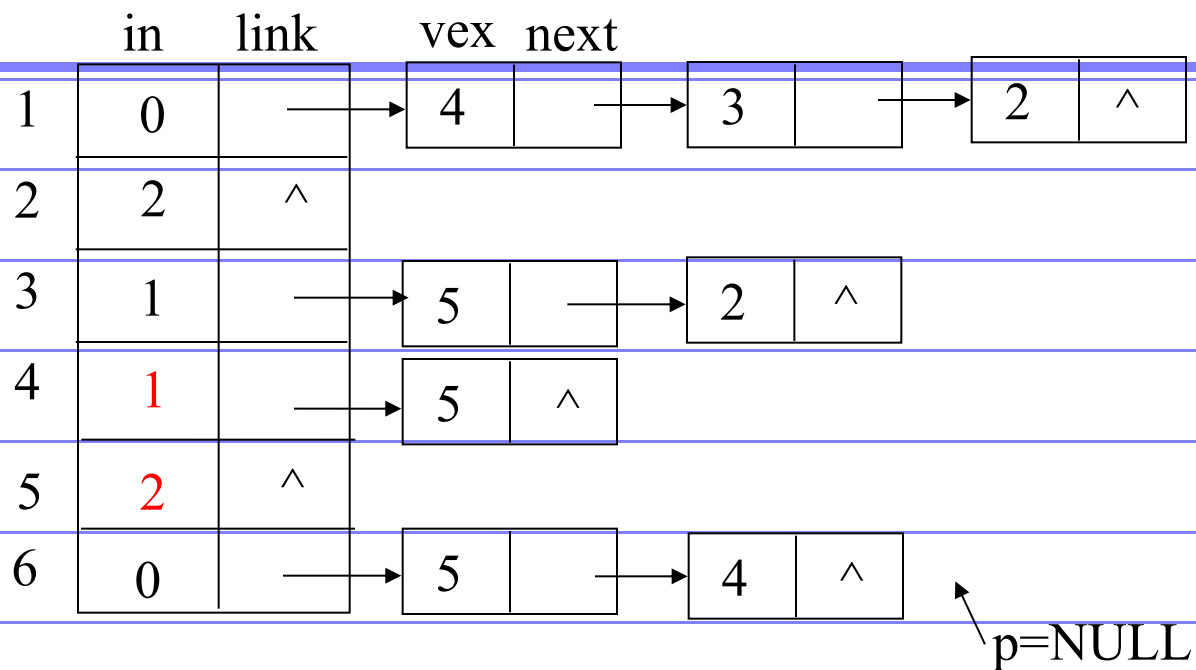
输出序列: 6

数据结构



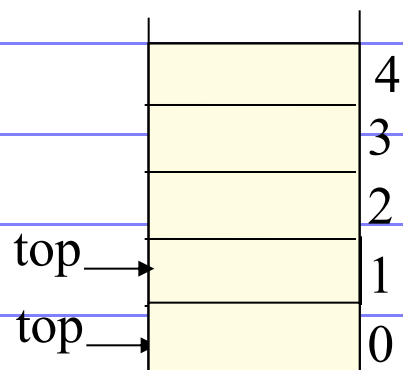
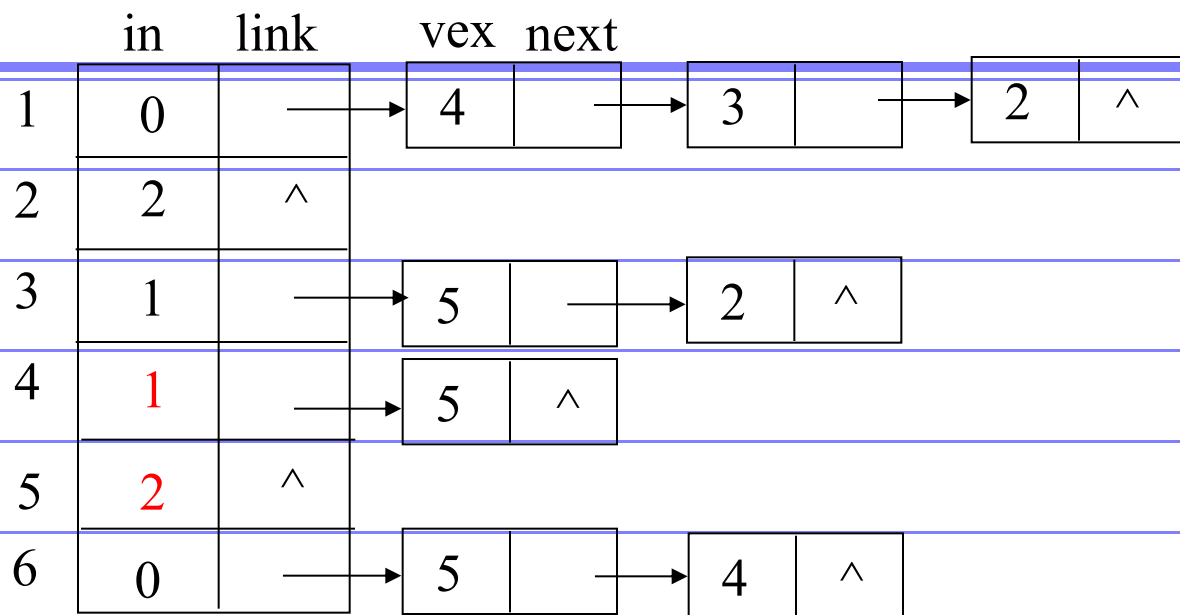
输出序列: 6

数据结构

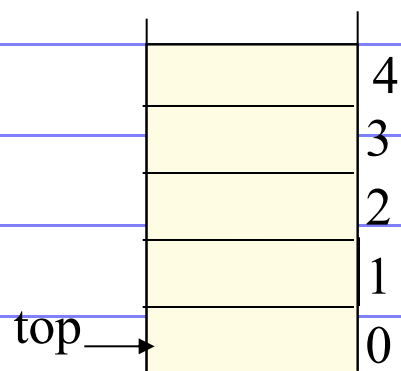
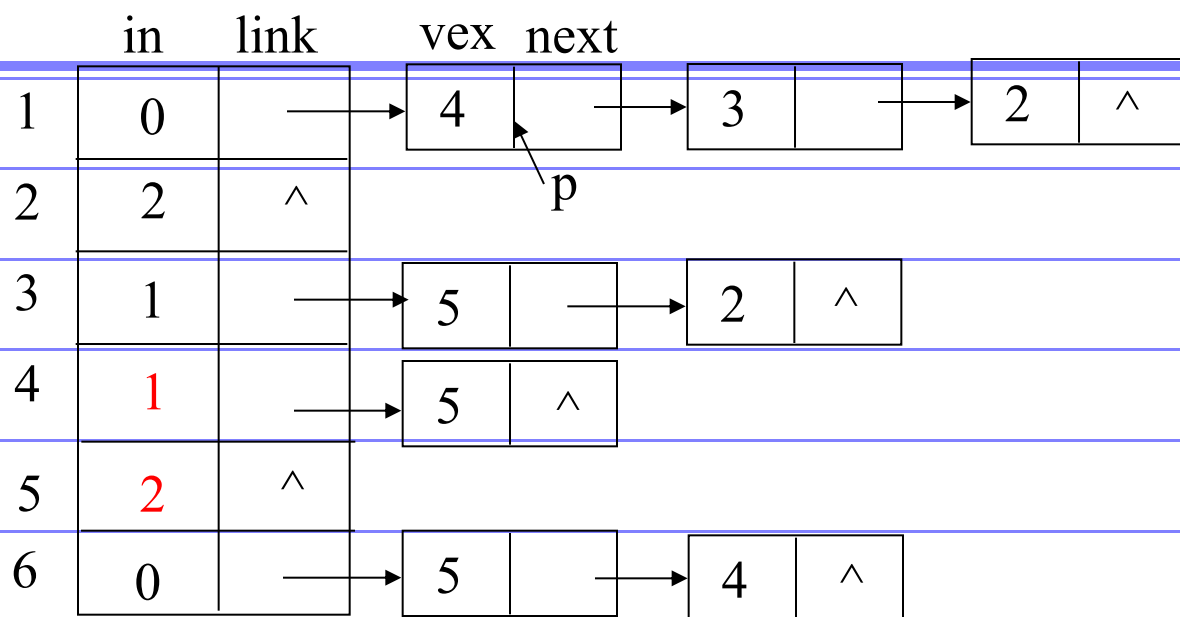


输出序列: 6

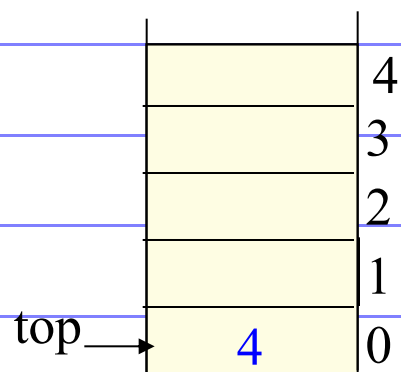
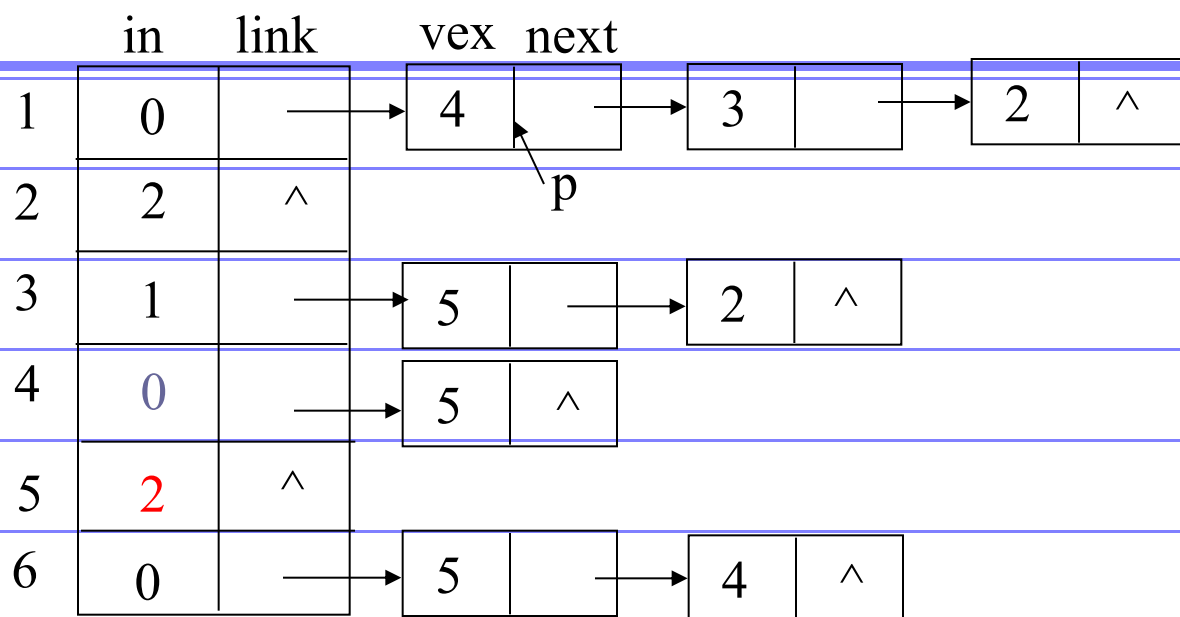
数据结构



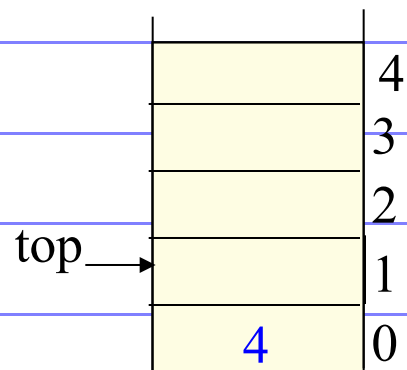
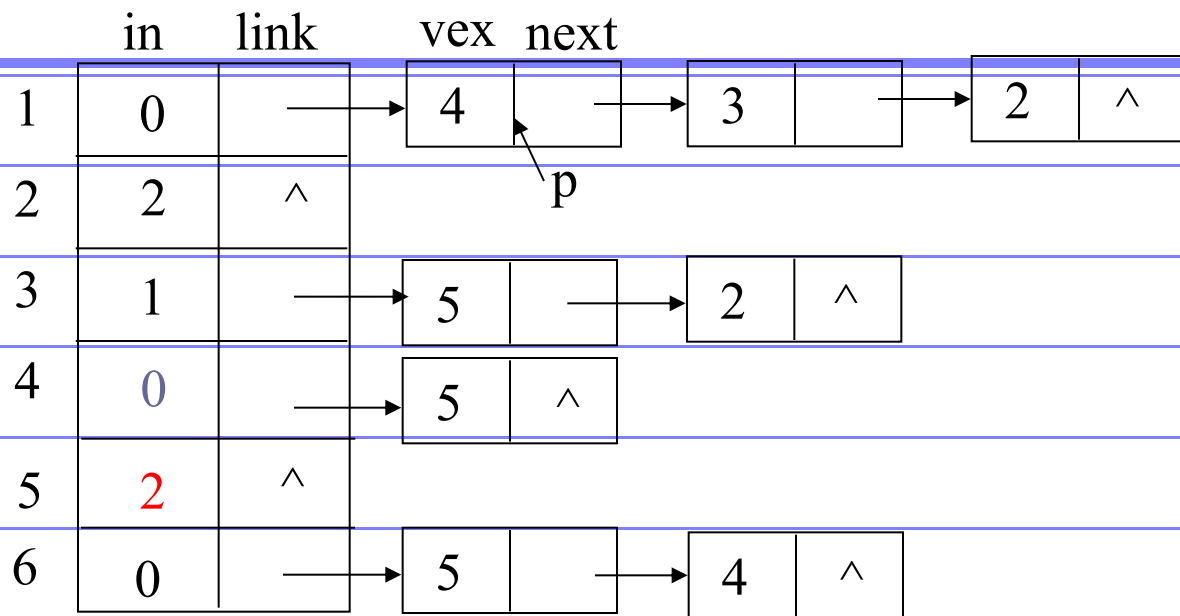
输出序列: 6 1 数据结构



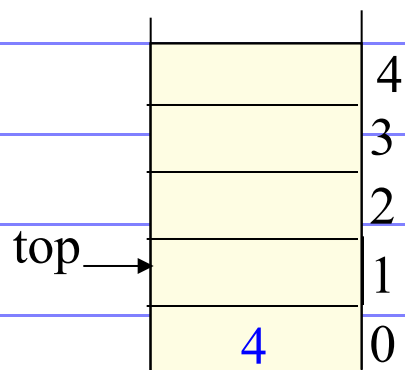
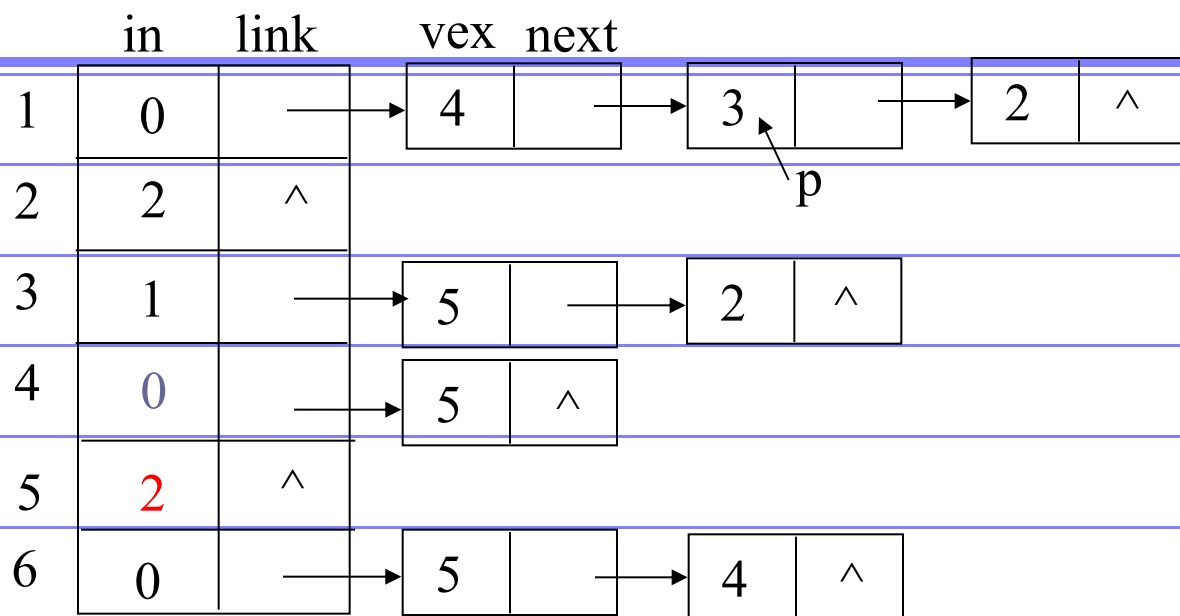
输出序列: 6 1 数据结构



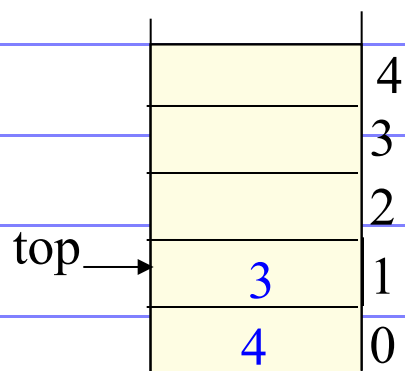
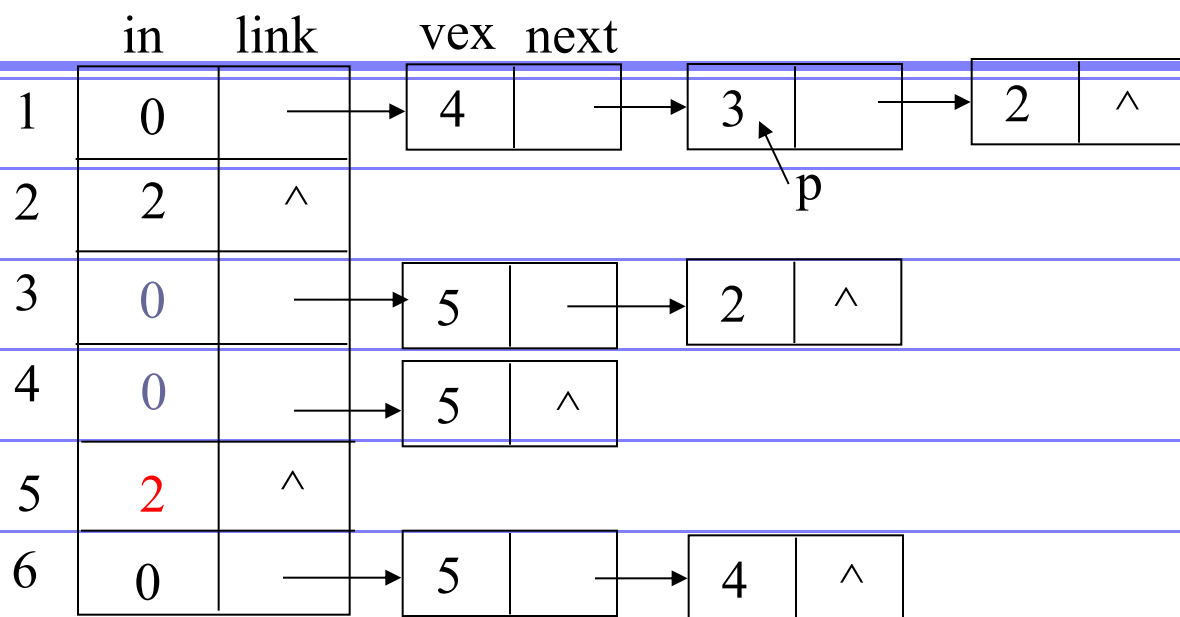
输出序列: 6 1 数据结构



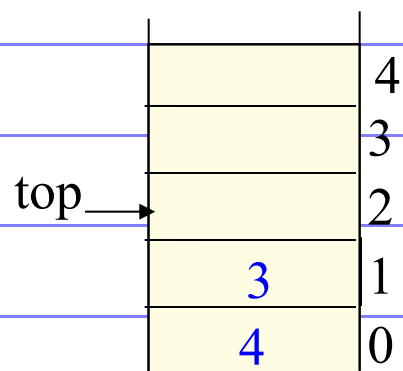
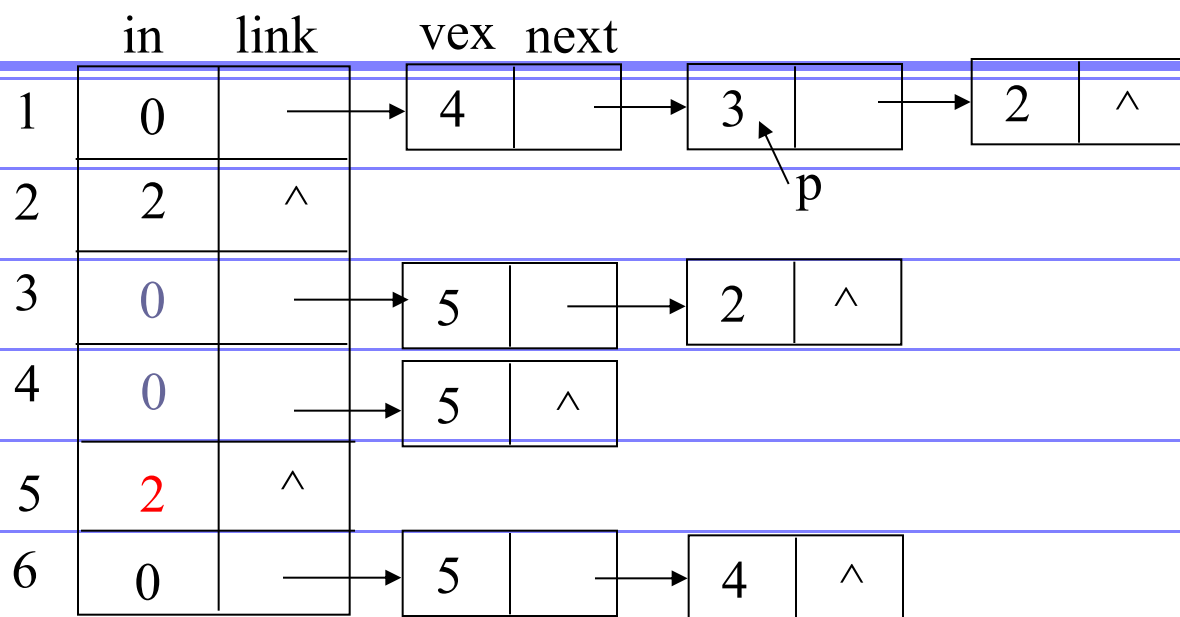
输出序列: 6 1 数据结构



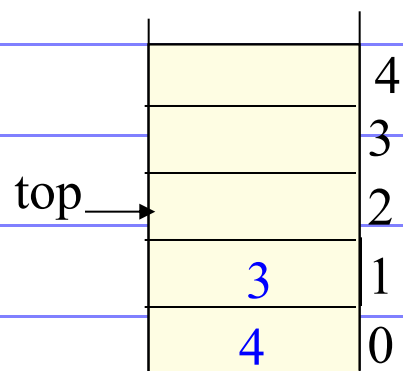
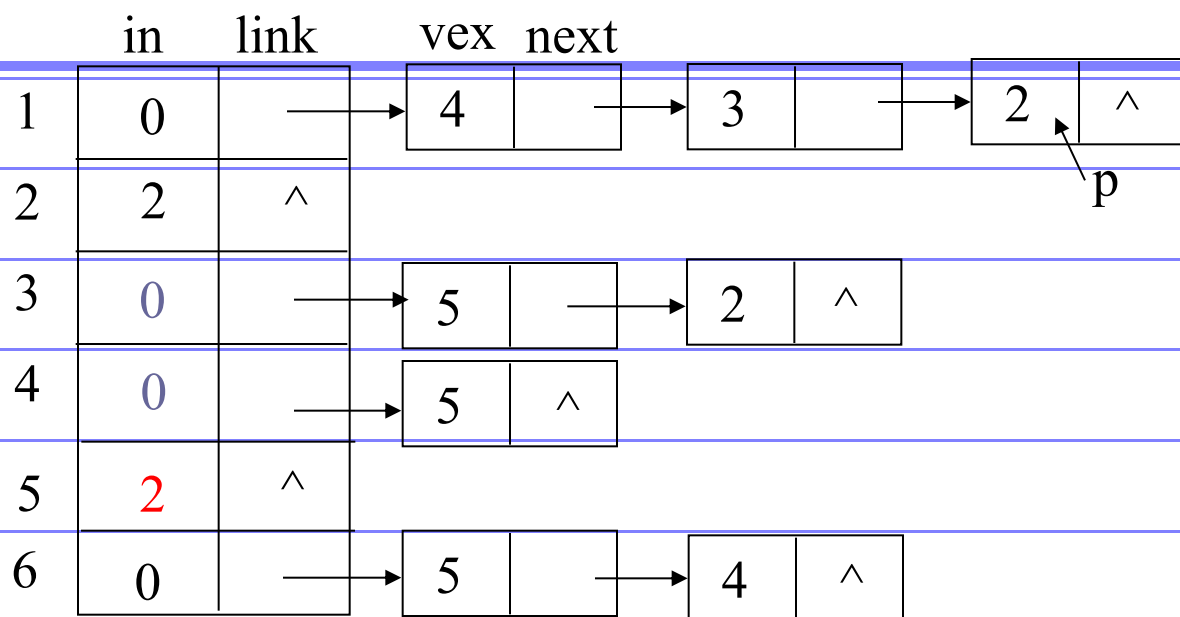
输出序列: 6 1 数据结构



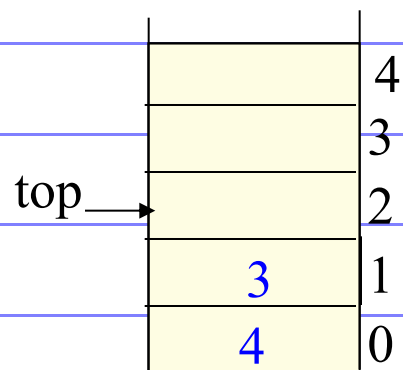
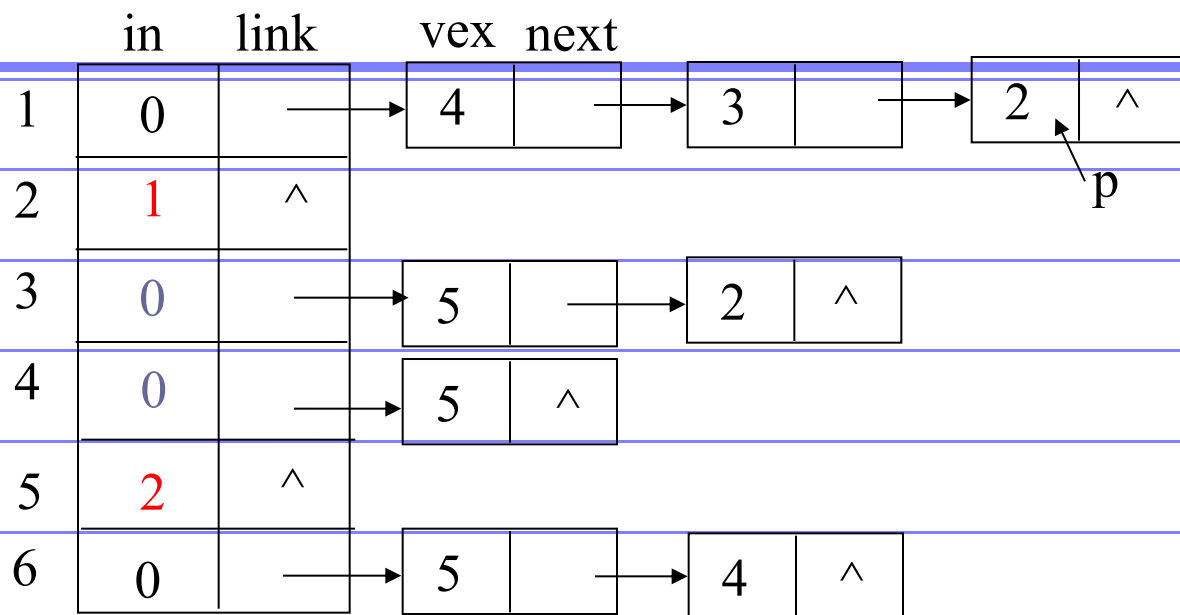
输出序列: 6 1 数据结构



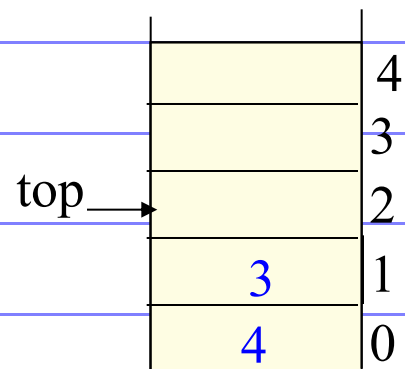
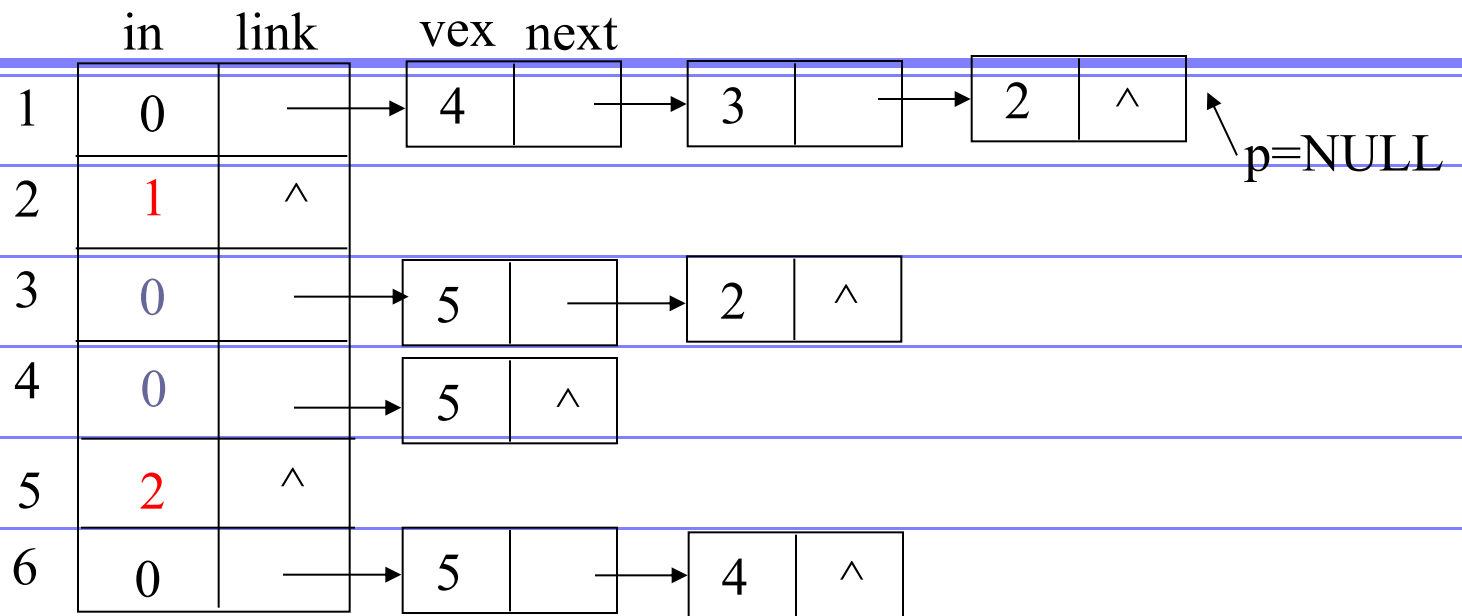
输出序列: 6 1 数据结构



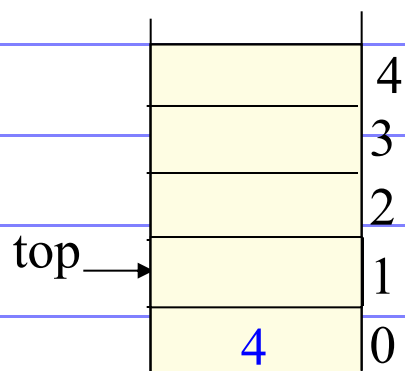
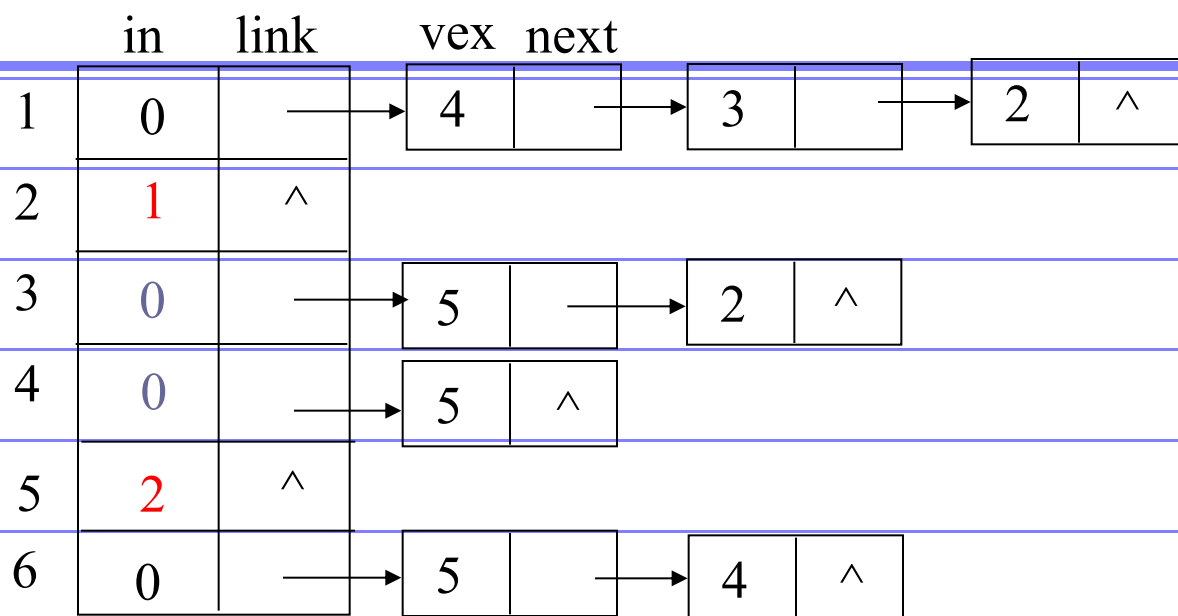
输出序列: 6 1 数据结构



输出序列: 6 1 数据结构

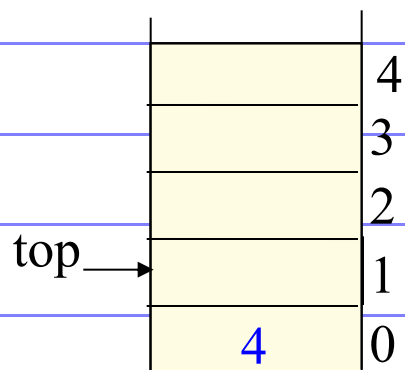
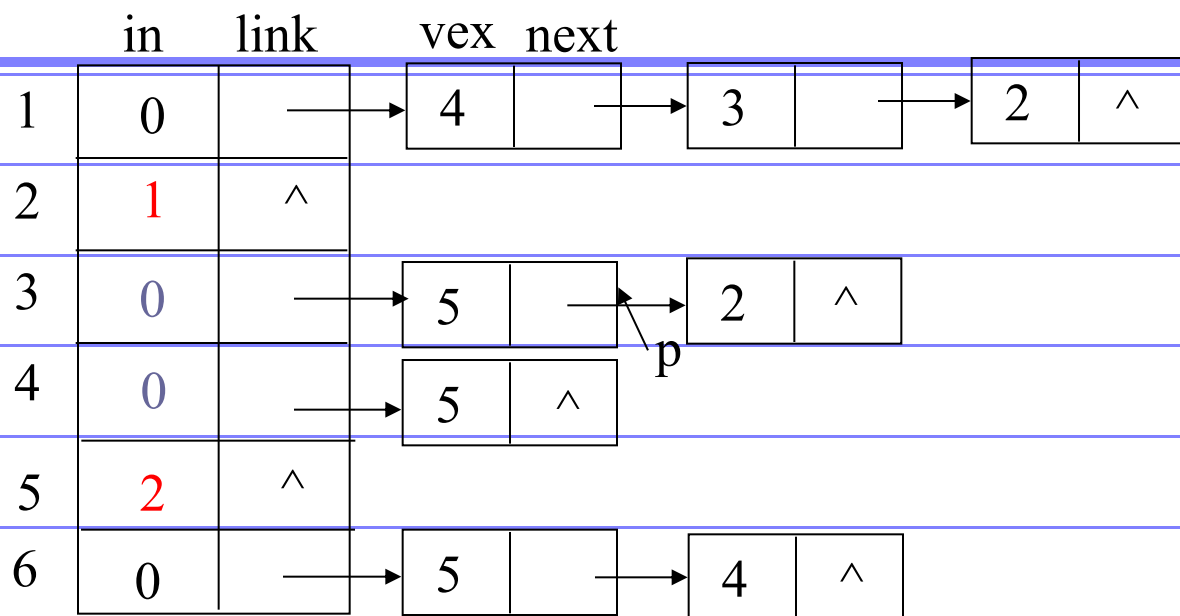


输出序列: 6 1 数据结构

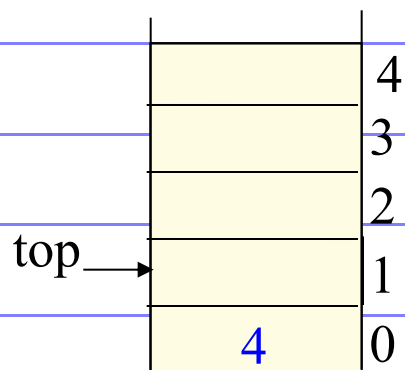
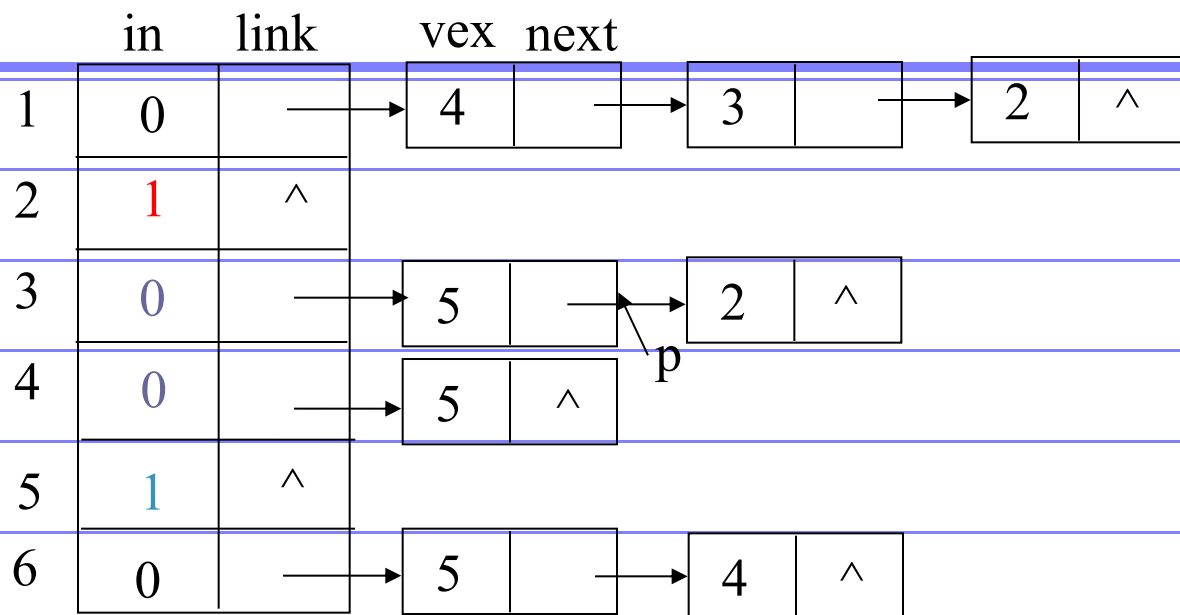


输出序列: 6 1 3

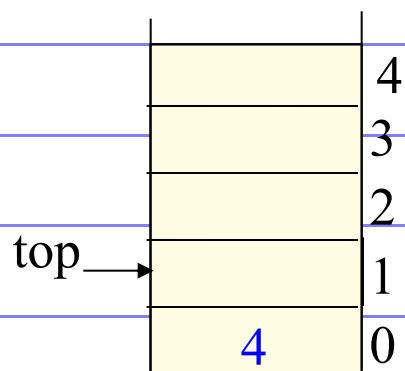
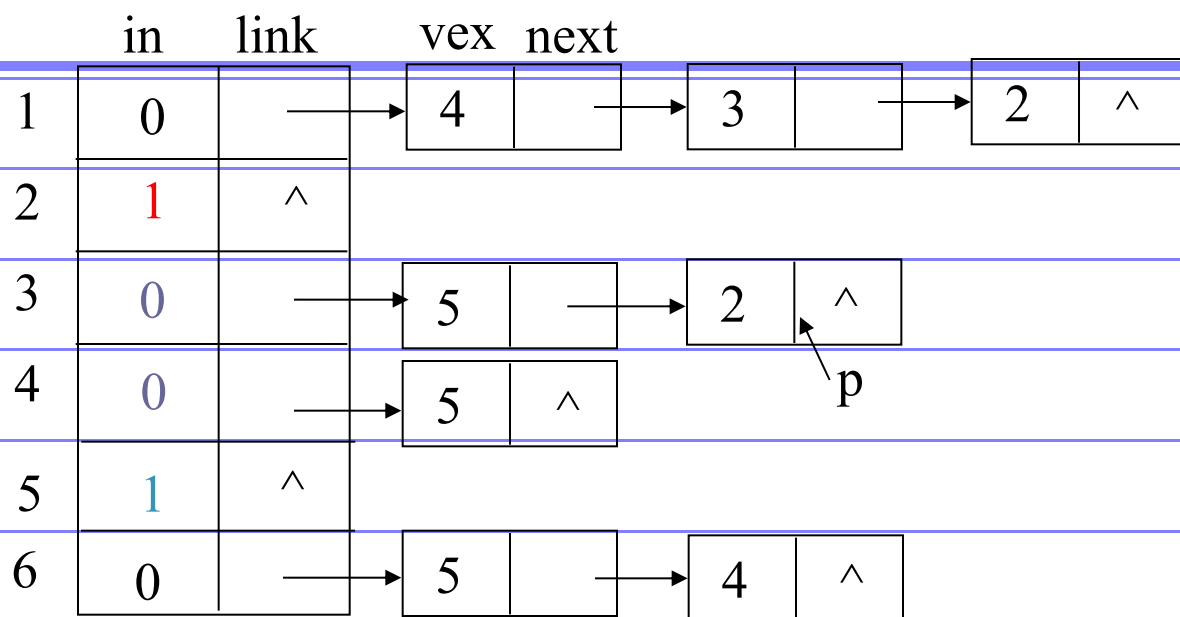
数据结构



输出序列: 6 1 3 数据结构

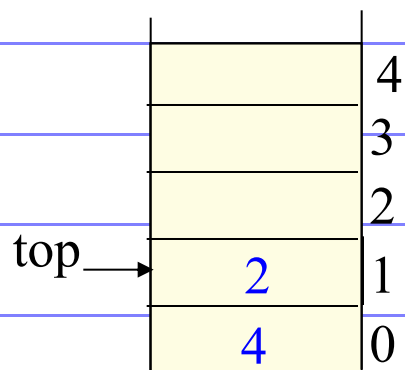
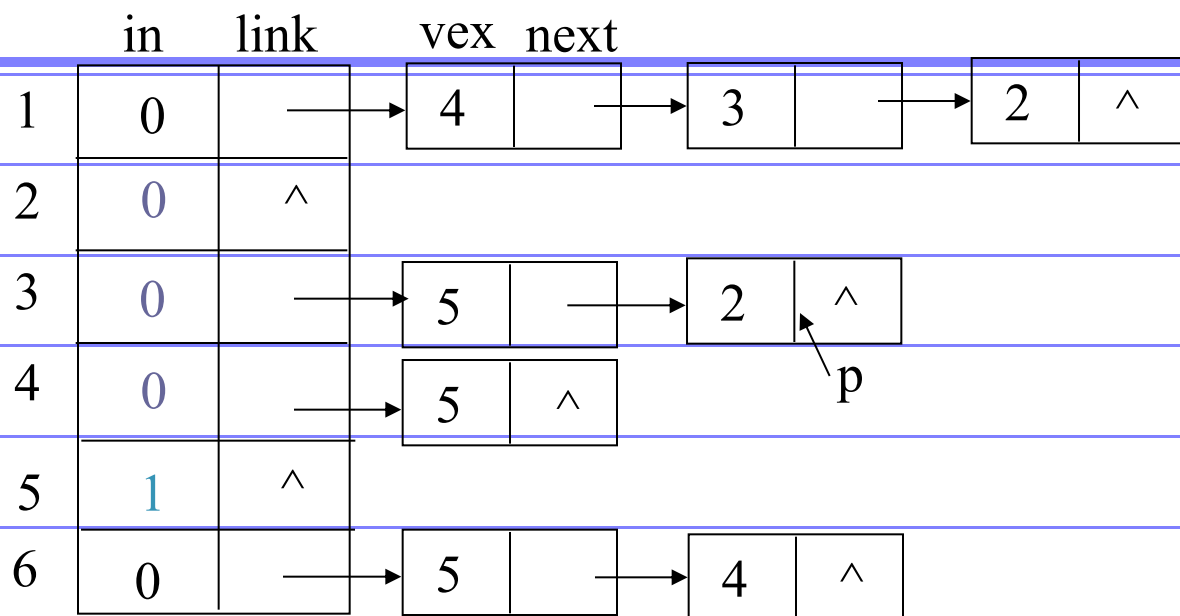


输出序列: 6 1 3 数据结构



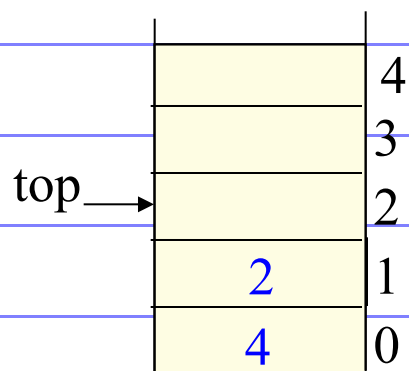
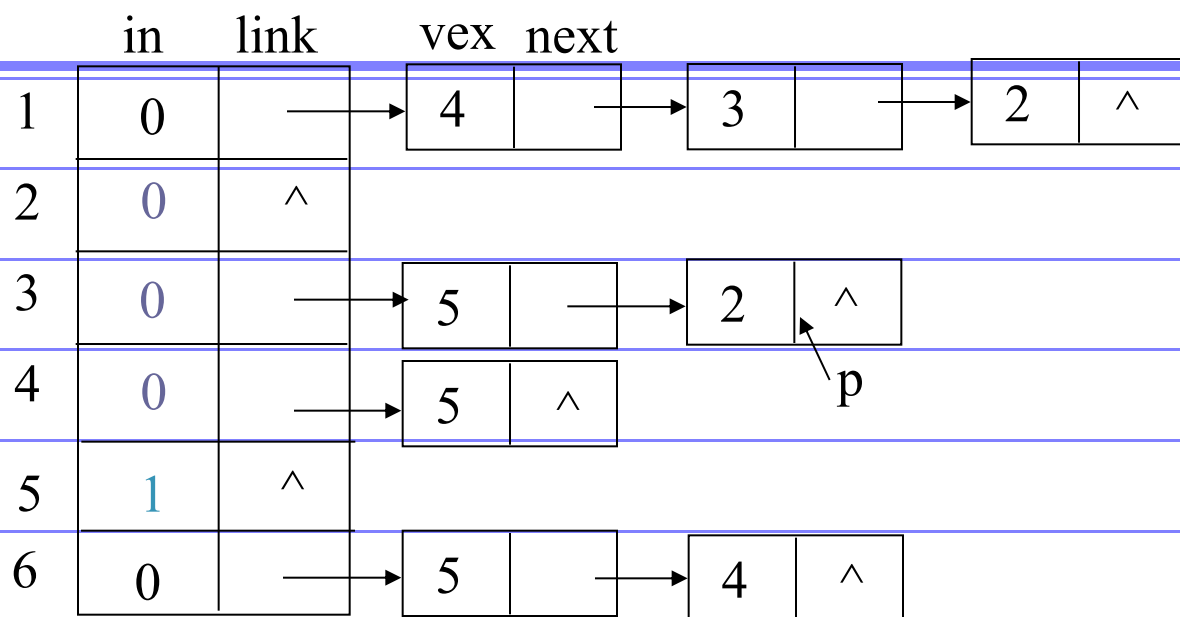
输出序列: 6 1 3

数据结构



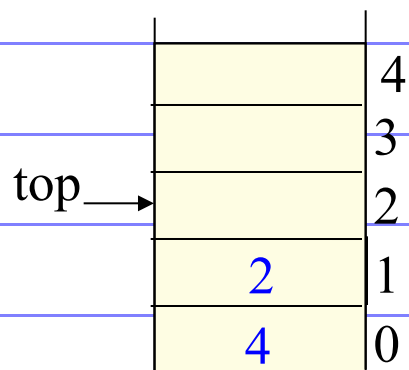
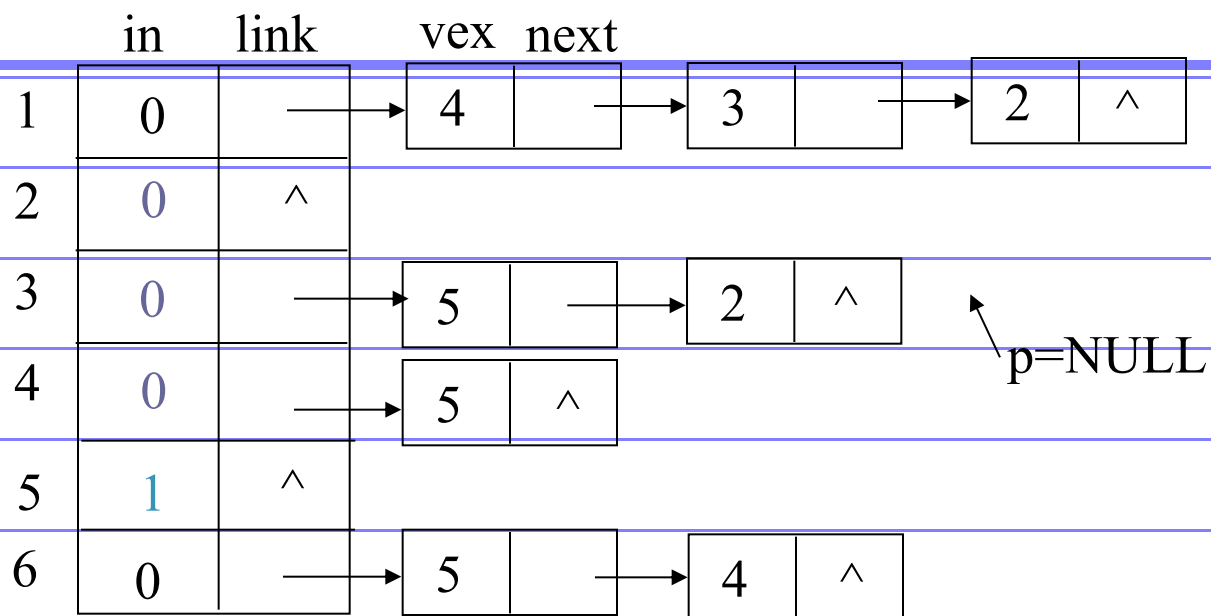
输出序列: 6 1 3

数据结构



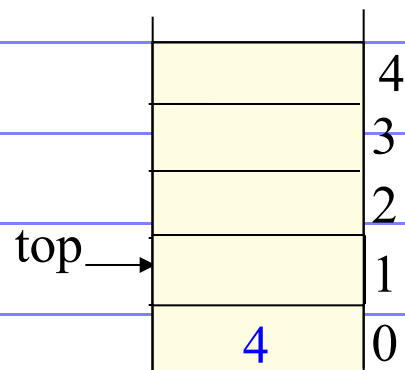
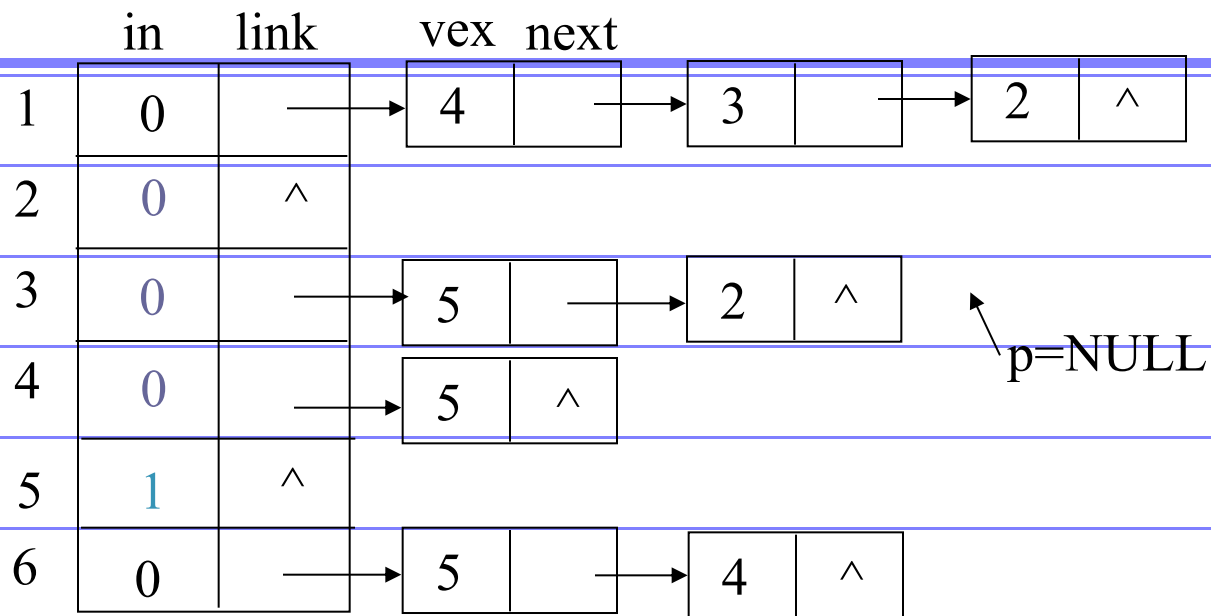
输出序列: 6 1 3

数据结构



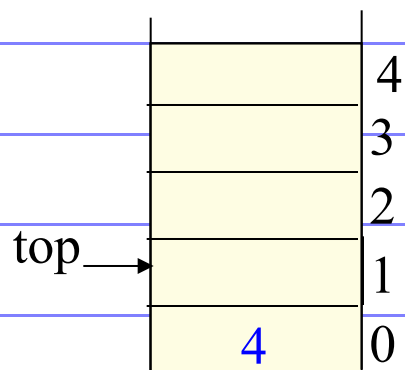
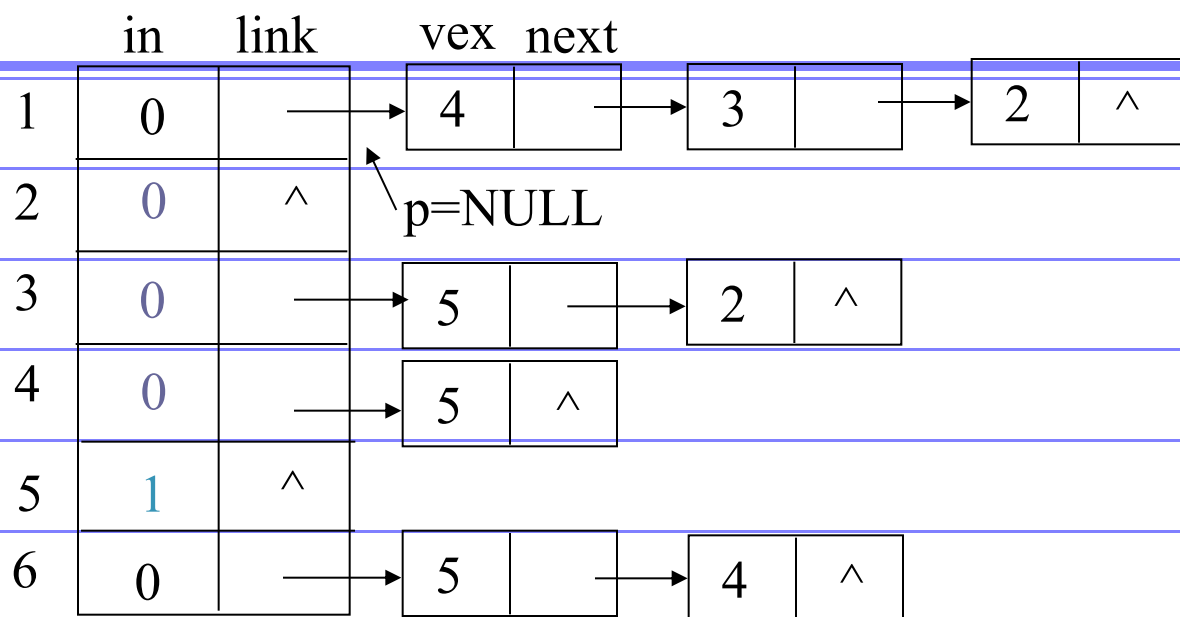
输出序列: 6 1 3

数据结构



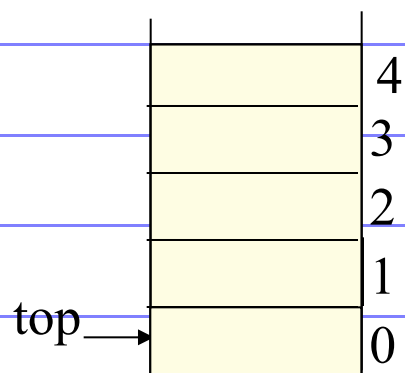
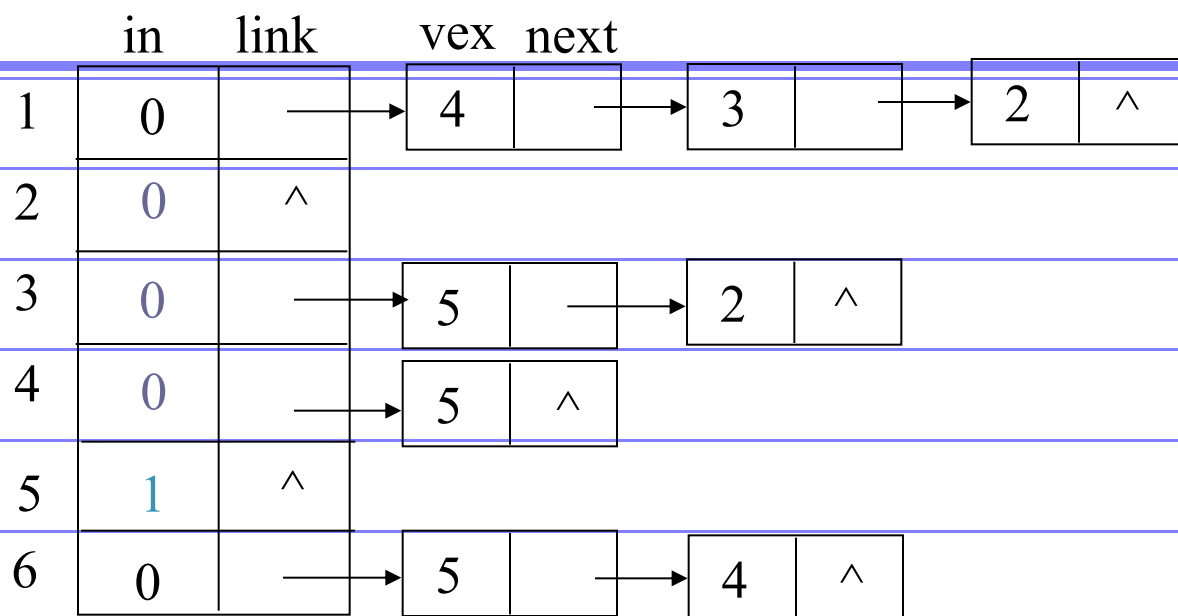
输出序列: 6 1 3 2

数据结构

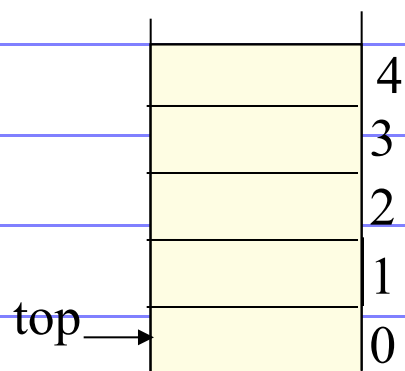
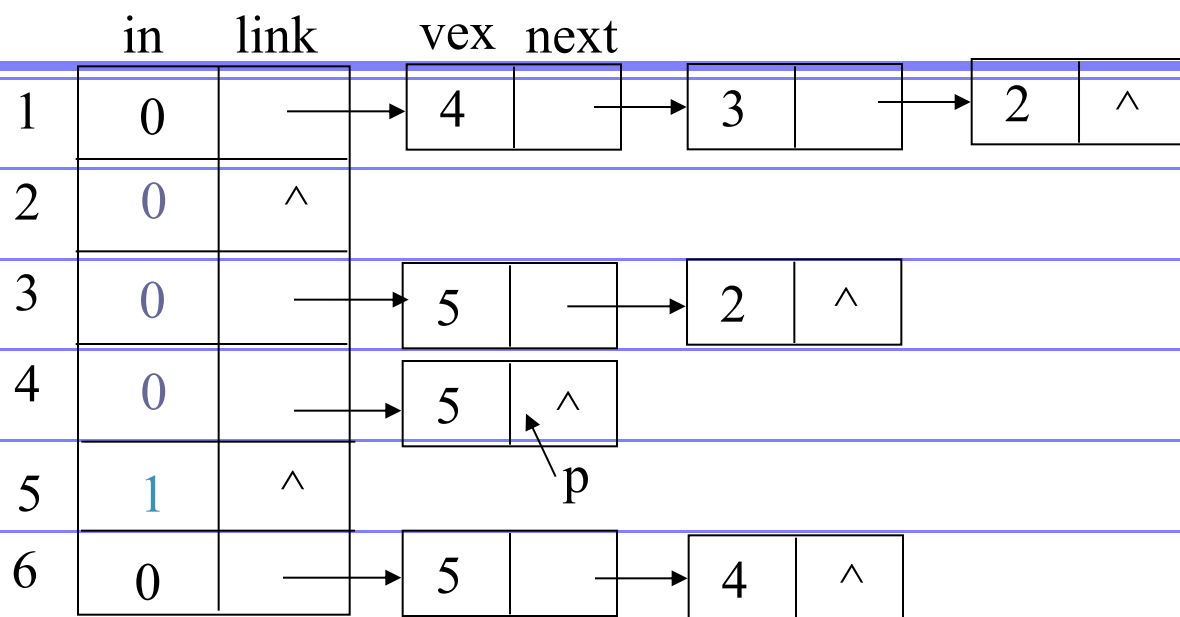


输出序列: 6 1 3 2

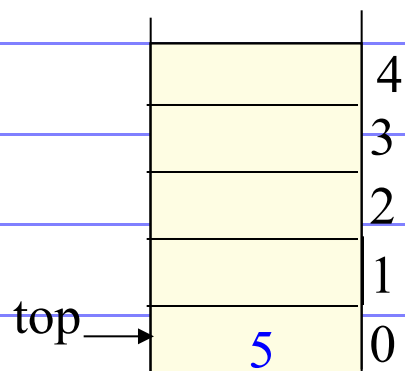
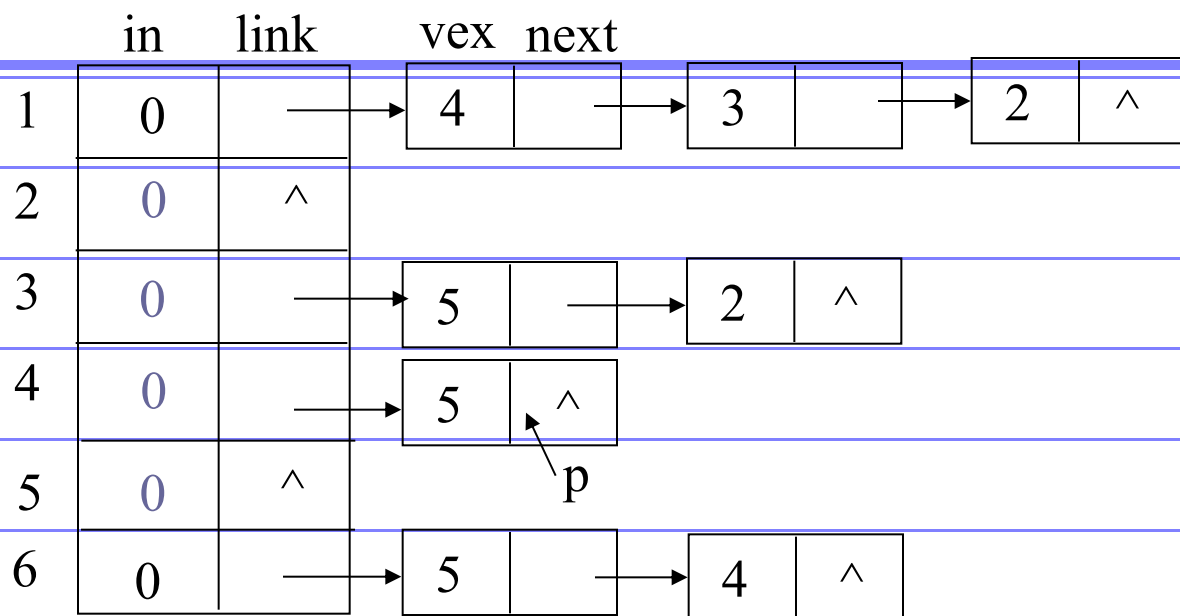
数据结构



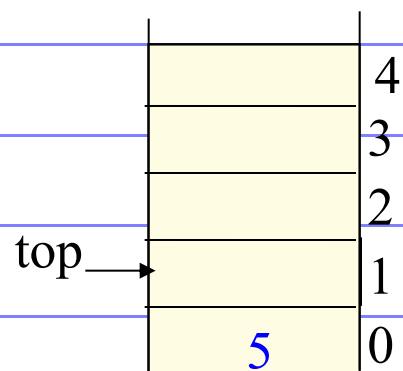
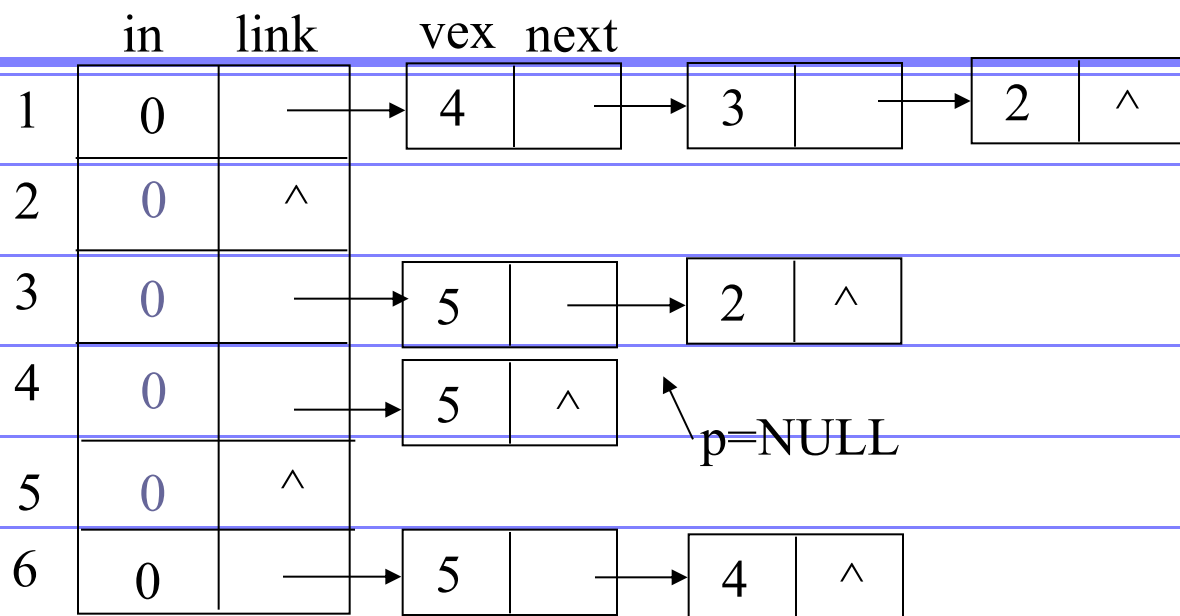
输出序列: 6 1 3 2 4



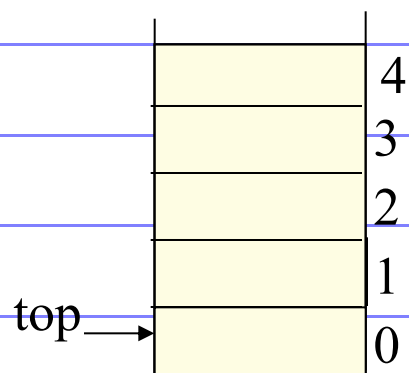
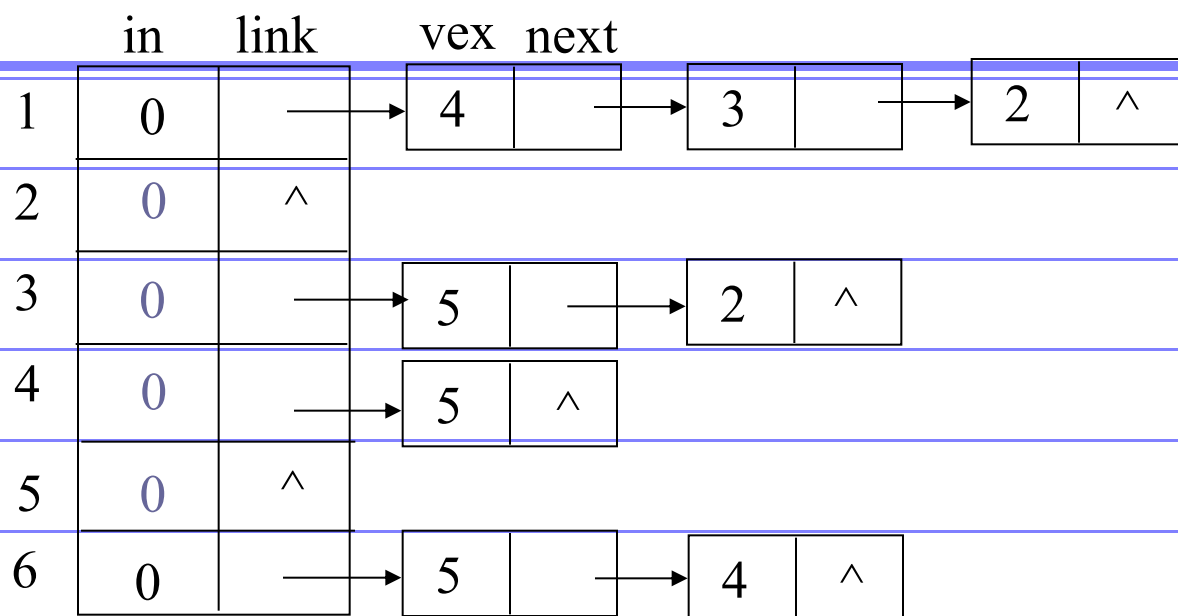
输出序列: 6 1 3 2 4



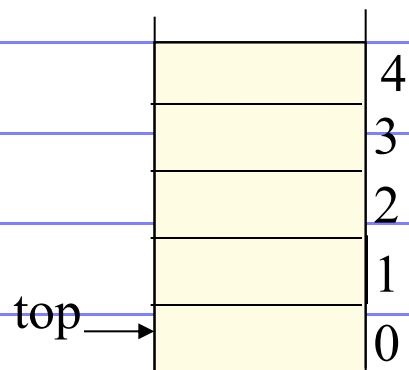
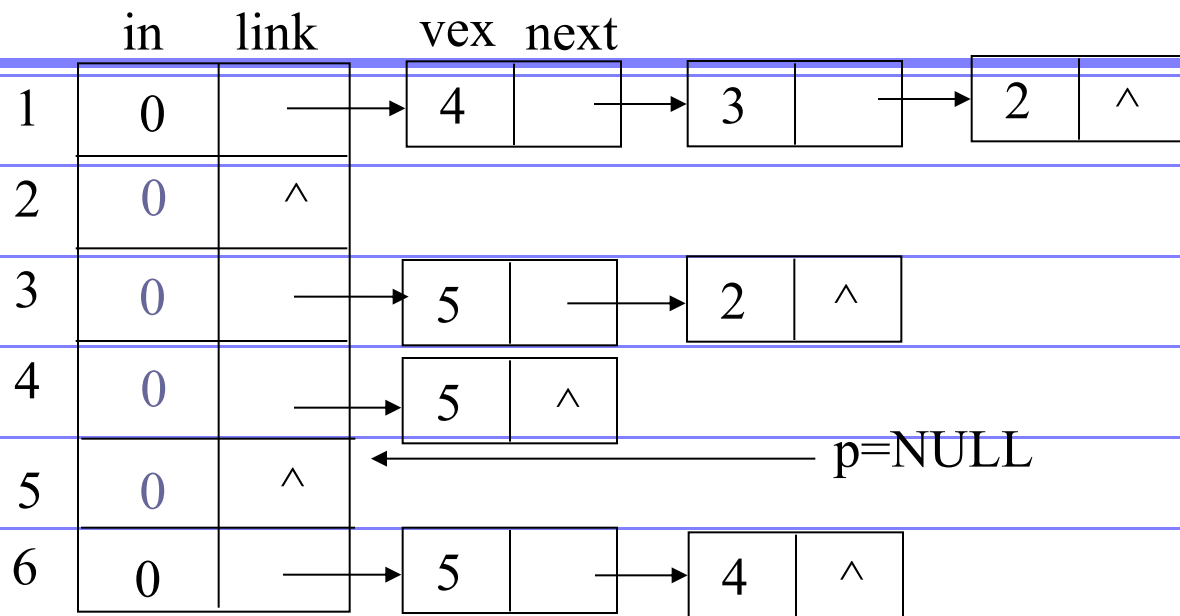
输出序列: 6 1 3 2 4



输出序列: 6 1 3 2 4



输出序列: 6 1 3 2 4 5



输出序列: 6 1 3 2 4 5

```

#include <iostream>

#define MaxVerTexNum 20

typedef int VertexType;           // 顶点的数据类型
typedef int InfoType;            // 带权图中边上权值的数据类型

// 用邻接表存储图
typedef struct ArcNode {          // 边表结点
    int adjvex;                  // 该弧所指向的顶点的位置 (存储下标)
    InfoType info;               // 边权值
    struct ArcNode* nextarc;     // 指向下一条弧的指针
}ArcNode;

typedef struct VNode {            // 顶点表结点
    VertexType data;             // 顶点信息
    ArcNode* firstarc;           // 指向第一条依附该顶点的弧的指针
}VNode, AdjList[MaxVerTexNum];

typedef struct {                  // 邻接表
    AdjList vertices;            // 邻接表
    int vexnum, arcnum;          // 图的顶点数和弧数
} Graph;

// 链栈 (带头结点)
typedef struct ListackNode {
    int data;                    // 存储图顶点的下标
    struct ListackNode* next;
} ListackNode, * Listack;

void initGraph(Graph& G);
bool InitStack(Listack& S);
bool StackEmpty(Listack S);
bool Push(Listack& S, int x);
bool Pop(Listack& S, int& x);
bool GetTop(Listack S, int& x);
void DestroyStack(Listack& S);
bool TopologicalSort(Graph G, int print[]);

```

```

int main() {
    Graph G;
    initGraph(G);
    int print[MaxVerTexNum];
    if (TopologicalSort(G, print)) {
        for (int i = 0; i < G.vexnum; ++i) {
            std::cout << print[i] << " ";
        }
        std::cout << std::endl;
    }

    system("pause");
    return 0;
}

// 第一行输入结点数和弧的个数, 第二行开始输入弧 (起点 终点 权值)
5 5
0 1 5
1 3 2
2 3 7
2 4 6
3 4 10
void initGraph(Graph& G) {        // 头插法建立邻接表
    std::cin >> G.vexnum >> G.arcnum;

    for (int i = 0; i < G.vexnum; ++i) {
        G.vertices[i].data = i;
        G.vertices[i].firstarc = NULL;
    }

    for (int i = 0; i < G.arcnum; ++i) {
        int u, v, val;
        std::cin >> u >> v >> val;
        ArcNode* p = (ArcNode*)malloc(sizeof(ArcNode));
        if (p == NULL)
            return;
        p->adjvex = v;
        p->info = val;
        p->nextarc = G.vertices[u].firstarc;
        G.vertices[u].firstarc = p;
    }
}

```

数据结

```

/*=====*/
// 初始化链栈（带头结点）
bool InitStack(Listack& S) {
    S = (ListackNode*)malloc(sizeof(ListackNode));
    if (S == NULL)          // 内存分配失败
        return false;

    S->next = NULL;
    return true;
}

// 判断栈空
bool StackEmpty(Listack S) {
    if (S->next == NULL)
        return true;
    else
        return false;
}

// 进栈操作—新元素入栈
bool Push(Listack& S, int x) {
    ListackNode* q = (ListackNode*)malloc(sizeof(ListackNode));
    if (q == NULL)          // 内存分配失败
        return false;

    q->data = x;
    q->next = S->next;
    S->next = q;
    return true;
}

// 出栈操作
bool Pop(Listack& S, int& x) {
    if (S->next == NULL)    // 栈空，报错
        return false;

    ListackNode* q = S->next;
    x = q->data;
    S->next = q->next;
    free(q);
    return true;
}

```

```

// 读栈顶操作
bool GetTop(Listack S, int& x) {
    if (S->next == NULL)    // 栈空，报错
        return false;

    x = S->next->data;
    return true;
}

// 销毁链栈
void DestroyStack(Listack& S) {
    ListackNode* p = S->next;
    ListackNode* q;
    while (p != NULL) {
        q = p;
        p = p->next;
        free(q);
    }
    free(S);
    S = NULL;
}

```

```

/*=====*/
// 求拓扑序列
bool TopologicalSort(Graph G, int print[]) {
    Liststack S;
    InitStack(S);

    int indegree[MaxVerTexNum] = {0};           // 存储顶点的入度
    for (int i = 0; i < G.vexnum; ++i) {         // 初始化入度数组
        ArcNode* p = G.vertices[i].firstarc;
        while (p) {
            indegree[p->adjvex]++;
            p = p->nextarc;
        }
    }

    for (int i = 0; i < G.vexnum; ++i) {
        if (indegree[i] == 0) {                   // 将所有入度为0的顶点进栈
            Push(S, i);
        }
    }

    int count = 0;                               // 计数，记录当前已经输出的顶点数
    while (!StackEmpty(S)) {                     // 栈不空，则存在入度为0的顶点
        int i;
        Pop(S, i);
        print[count++] = i;
        // 将所有i指向的顶点的入度减一，并且将入度减为0的顶点压入栈S
        for (ArcNode* p = G.vertices[i].firstarc; p != NULL; p = p->nextarc) {
            if (!(--indegree[p->adjvex])) {
                Push(S, p->adjvex);               // 入度为0，则入栈
            }
        }
    }

    DestroyStack(S);

    if (count < G.vexnum)
        return false;                           // 排序失败，有向图中有回路
    else
        return true;
}

```

■ 算法分析

建邻接表: $T(n)=O(e)$

搜索入度为0的顶点的时间: $T(n)=O(n)$

拓扑排序: $T(n)=O(n+e)$

求逆拓扑序列（邻接矩阵）

```
#include <iostream>

#define MaxVerTexNum 20

typedef char VertexType;           // 顶点的数据类型
typedef int EdgeType;              // 带权图中边上权值的数据类型

// 邻接矩阵存储图
typedef struct {
    VertexType Vex[MaxVerTexNum]; // 顶点表
    EdgeType Edge[MaxVerTexNum][MaxVerTexNum]; // 邻接矩阵，边表
    int vexnum, arcnum;           // 图的当前顶点数和边数/弧数
} MGraph;

// 链栈（带头结点）
typedef struct LiStackNode {
    int data;                     // 存储图顶点的下标
    struct LiStackNode* next;
} LiStackNode, * LiStack;

bool InitStack(LiStack& S);
bool StackEmpty(LiStack S);
bool Push(LiStack& S, int x);
bool Pop(LiStack& S, int& x);
bool GetTop(LiStack S, int& x);
void DestroyStack(LiStack& S);
bool ReverseTopologicalSort(MGraph G, int print[]);
```

// 求逆拓扑序列

```
bool ReverseTopologicalSort(MGraph G, int print[]) {
    LiStack S;
    InitStack(S);

    int outdegree[MaxVerTexNum] = { 0 };           // 存储顶点的出度
    for (int i = 0; i < G.vexnum; ++i) {
        for (int j = 0; j < G.vexnum; ++j) {
            if (G.Edge[i][j] != -1) {
                outdegree[i]++;
            }
        }
    }

    for (int i = 0; i < G.vexnum; ++i) {
        if (outdegree[i] == 0) {                   // 将所有入度为0的顶点进栈
            Push(S, i);
        }
    }

    int count = 0;                                // 计数, 记录当前已经输出的顶点数
    while (!StackEmpty(S)) {                       // 栈不空, 则存在入度为0的顶点
        int v;
        Pop(S, v);
        print[count++] = v;
        // 将所有i指向的顶点的入度减一, 并且将入度减为0的顶点压入栈S
        for (int u = 0; u < G.vexnum; ++u) {
            if (G.Edge[u][v] != -1) {
                if (--outdegree[u] == 0) {
                    Push(S, u);
                }
            }
        }
    }

    DestroyStack(S);

    if (count < G.vexnum)                          // 排序失败, 有向图中有回路
        return false;
    else
        return true;
}
```

求逆拓扑序列（邻接矩阵）

```
int main() {
    MGraph G = {
        {'0', '1', '2', '3', '4'},
        {
            {-1, 5, -1, -1, -1},
            {-1, -1, -1, 2, -1},
            {-1, -1, -1, 7, 6},
            {-1, -1, -1, -1, 10},
            {-1, -1, -1, -1, -1}
        },
        5, 5
    };

    int print[MaxVerTexNum];
    if (ReverseTopologicalSort(G, print)) {
        for (int i = 0; i < G.vexnum; ++i) {
            std::cout << print[i] << " ";
        }
        std::cout << std::endl;
    }

    system("pause");
    return 0;
}
```

6.6 关键路径

□ 问题提出

把工程计划表示为有向图，用顶点表示事件，弧表示活动；
每个事件表示在它之前的活动已完成，在它之后的活动可以开始

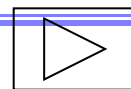
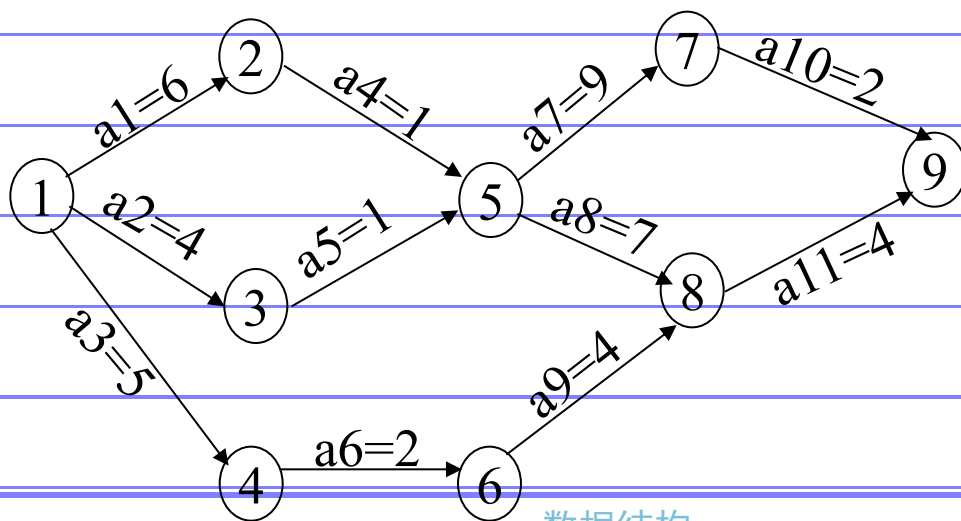
例 设一个工程有11项活动，9个事件

事件 V1——表示整个工程开始

事件V9——表示整个工程结束

问题：（1）完成整项工程至少需要多少时间？

（2）哪些活动是影响工程进度的关键？



相关定义与术语

- AOE网(Activity On Edge)——也叫边表示活动的网。
AOE网是一个带权的有向无环图，其中顶点表示事件，弧表示活动，权表示活动持续时间
- 路径长度——路径上各活动持续时间之和
- 关键路径——路径长度最长的路径叫~
- $Ve(j)$ ——表示事件 V_j 的最早发生时间
- $Vl(j)$ ——表示事件 V_j 的最迟发生时间
- $e(i)$ ——表示活动 a_i 的最早开始时间
- $l(i)$ ——表示活动 a_i 的最迟开始时间
- $l(i)-e(i)$ ——表示完成活动 a_i 的时间余量
- 关键活动——关键路径上的活动叫~，即 $l(i)=e(i)$ 的活动

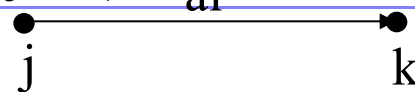
■ 问题分析

◇ 如何找 $e(i)=l(i)$ 的关键活动?

设活动 a_i 用弧 $\langle j, k \rangle$ 表示, 其持续时间记为: $dut(\langle j, k \rangle)$

则有: (1) $e(i)=Ve(j)$

(2) $l(i)=Vl(k) - dut(\langle j, k \rangle)$



◇ 如何求 $Ve(j)$ 和 $Vl(j)$?

(1) 从 $Ve(1)=0$ 开始向前递推

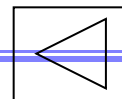
$$Ve(j) = \underset{i}{\text{Max}}\{Ve(i) + dut(\langle i, j \rangle)\}, \langle i, j \rangle \in T, 2 \leq j \leq n$$

其中 T 是所有以 j 为头的弧的集合

(2) 从 $Vl(n)=Ve(n)$ 开始向后递推

$$Vl(i) = \underset{j}{\text{Min}}\{Vl(j) - dut(\langle i, j \rangle)\}, \langle i, j \rangle \in S, 1 \leq i \leq n-1$$

其中 S 是所有以 i 为尾的弧的集合



■ 求关键路径步骤

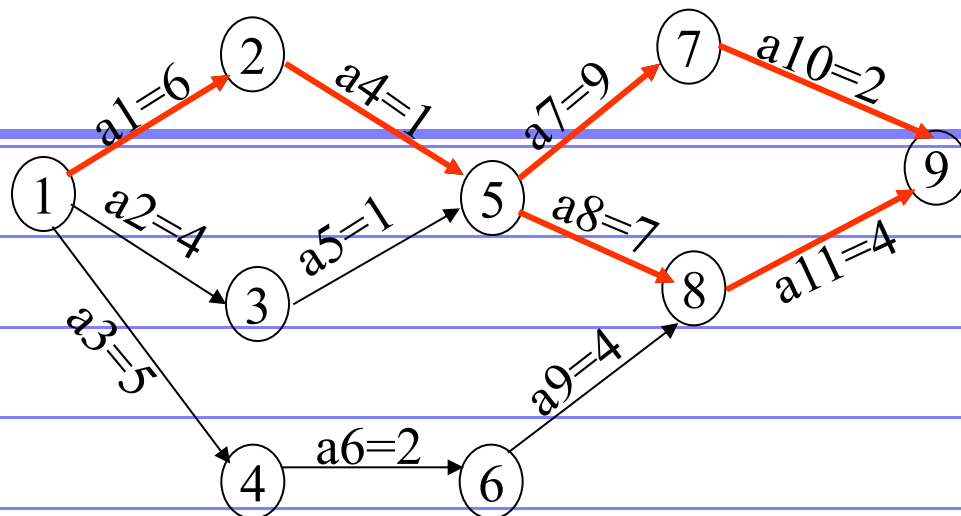
◇ 求 $Ve(i)$

◇ 求 $VI(j)$

◇ 求 $e(i)$

◇ 求 $l(i)$

◇ 计算 $l(i) - e(i)$



顶点	Ve	VI
V1	0	0
V2	6	6
V3	4	6
V4	5	8
V5	7	7
V6	7	10
V7	16	16
V8	14	14
V9	18	18

活动	e	l	$l - e$
a1	0	0	0 ✓
a2	0	2	2
a3	0	3	3
a4	6	6	0 ✓
a5	4	6	2
a6	5	8	3
a7	7	7	0 ✓
a8	7	7	0 ✓
a9	7	10	3
a10	16	16	0 ✓
a11	14	14	0 ✓

■ 算法实现

✧ 以邻接表作存储结构

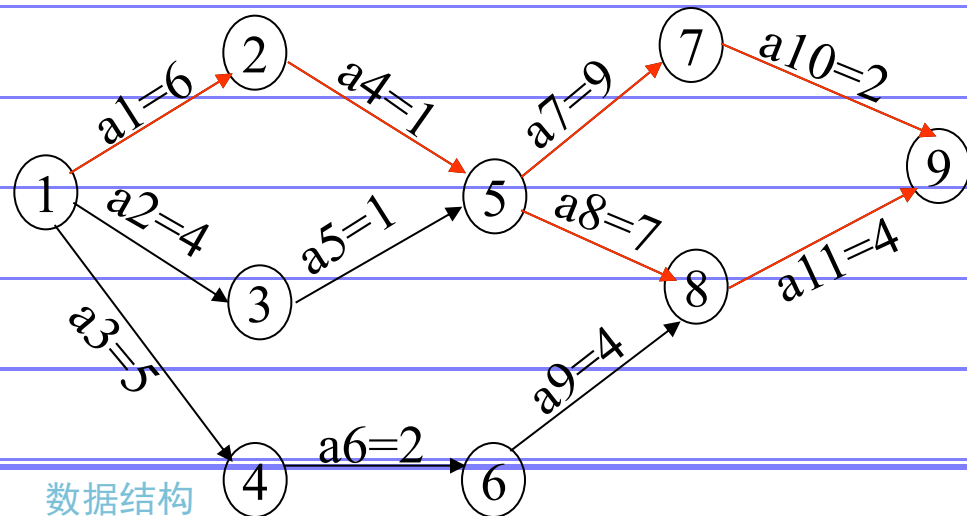
✧ 从源点 V_1 出发，令 $Ve[1]=0$ ，按拓扑序列求各顶点的 $Ve[i]$

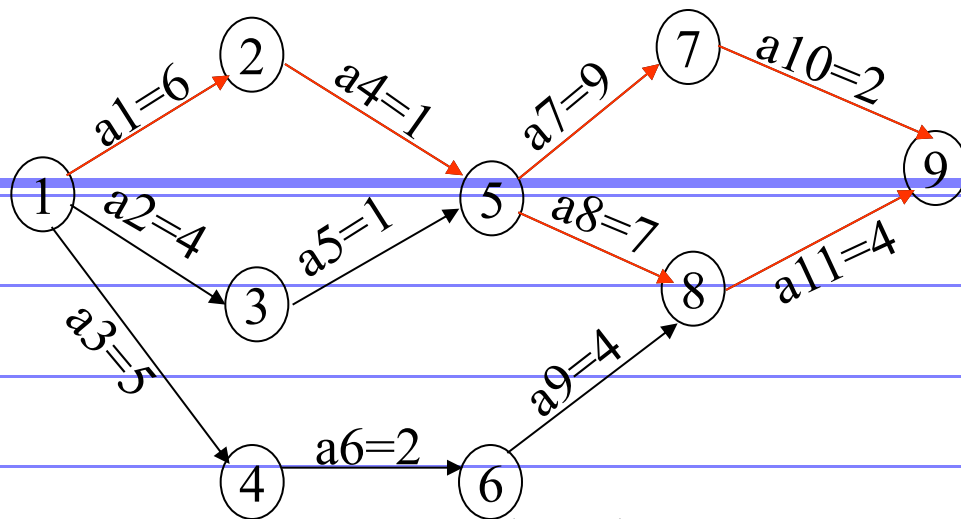
✧ 从汇点 V_n 出发，令 $VI[n]=Ve[n]$ ，按逆拓扑序列求其余各顶点的 $VI[i]$

✧ 根据各顶点的 Ve 和 VI 值，计算每条弧的 $e[i]$ 和 $l[i]$ ，找出 $e[i]=l[i]$ 的关键活动

■ 算法描述

- ◇ 输入顶点和弧信息，建立其邻接表
- ◇ 计算每个顶点的入度
- ◇ 对其进行拓扑排序
 - 排序过程中求顶点的 $Ve[i]$
 - 将得到的拓扑序列进栈
- ◇ 按逆拓扑序列求顶点的 $VI[i]$
- ◇ 计算每条弧的 $e[i]$ 和 $l[i]$,找出 $e[i]=l[i]$ 的关键活动





	vex	data	in	link	vex	length	next
1	1	0		→	2	6	→
2	2	1		→	5	1	^
3	3	1		→	5	1	^
4	4	1		→	6	2	^
5	5	2		→	7	9	→
6	6	1		→	8	4	^
7	7	1		→	9	10	^
8	8	2		→	9	4	^
9	9	2	^				

3	4	—	→	4	5	^
---	---	---	---	---	---	---

8	7	^
---	---	---

数据结构

(2) 算法实现

```
void critical_path(ALGraph *G)
{
    int j, k, m ; LinkNode *p ;

    if (Topologic_Sort(G)==-1)
        printf("\nAOE网中存在回路， 错误!!\n\n");
    else
    {
        for ( j=0; j<G->vexnum; j++)
            ve[j]=0 ;    /* 事件最早发生时间初始化 */

        for (m=0 ; m<G->vexnum; m++)
        {
            j=topol[m] ;

            p=G->adjlist[j].firstarc ;

            for (; p!=NULL; p=p->nextarc )
```

```
{ k=p->adjvex ;
```

```
    if (ve[j]+p->weight>ve[k])
```

```
        ve[k]=ve[j]+p->weight ;
```

```
    }
```

```
} /* 计算每个事件的最早发生时间ve值 */
```

```
for ( j=0; j<G->vexnum; j++)
```

```
    vl[j]=ve[j] ; /* 事件最晚发生时间初始化 */
```

```
for (m=G->vexnum-1; m>=0; m--)
```

```
{ j=topol[m] ; p=G->adjlist[j].firstarc ;
```

```
    for (; p!=NULL; p=p->nextarc )
```

```
        { k=p->adjvex ;
```

```
            if (vl[k]-p->weight<vl[j])
```

```
                vl[j]=vl[k]-p->weight ;
```

```
    }  
    } /* 计算每个事件的最晚发生时间vl值 */  
    for (m=0 ; m<G->vexnum; m++)  
    { p=G->adjlist[m].firstarc ;  
      for (; p!=NULL; p=p->nextarc )  
      { k=p->adjvex ;  
        if ( (ve[m]+p->weight)==vl[k])  
          printf("<%d, %d>, m, k") ;  
      }  
    } /* 输出所有的关键活动 */  
  } /* end of else */  
}
```

(3) 算法分析

设AOE网有 n 个事件， e 个活动，则算法的主要执行是：

- ◆ 进行拓扑排序：时间复杂度是 $O(n+e)$ ；
 - ◆ 求每个事件的 ve 值和 vl 值：时间复杂度是 $O(n+e)$ ；
 - ◆ 根据 ve 值和 vl 值找关键活动：时间复杂度是 $O(n+e)$ ；
- 因此，整个算法的时间复杂度是 $O(n+e)$ 。

6.7 最短路径

□ 问题提出

用带权的有向图表示一个交通运输网，图中：

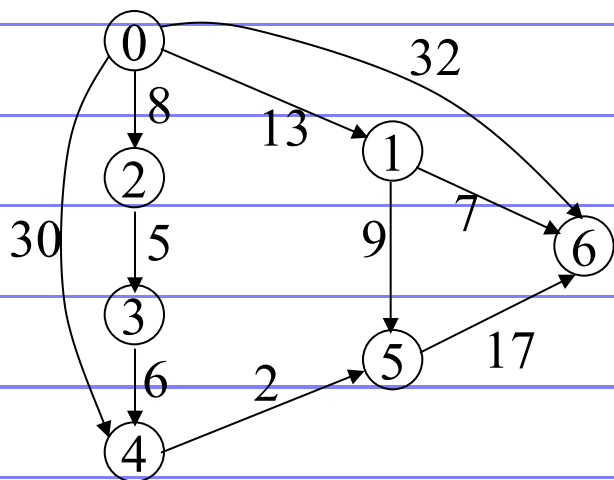
顶点——表示城市

边——表示城市间的交通联系

权——表示此线路的长度或沿此线路运输所花的时间或费用等

问题：从某顶点出发，沿图的边到达另一顶点所经过的路径中，各边上权值之和最小的一条路径——最短路径

□ 从某个源点到其余各顶点的最短路径



最短路径	长度
$\langle V_0, V_1 \rangle$	13
$\langle V_0, V_2 \rangle$	8
$\langle V_0, V_2, V_3 \rangle$	13
$\langle V_0, V_2, V_3, V_4 \rangle$	19
$\langle V_0, V_2, V_3, V_4, V_5 \rangle$	21
$\langle V_0, V_1, V_6 \rangle$	20

□ 迪杰斯特拉(Dijkstra)算法思想

按路径长度递增次序产生最短路径算法：

把V分成两组：

(1) S：已求出最短路径的顶点的集合

(2) V-S=T：尚未确定最短路径的顶点集合

将T中顶点按最短路径递增的次序加入到S中，

保证：(1) 从源点V0到S中各顶点的最短路径长度都不大于
从V0到T中任何顶点的最短路径长度

(2) 每个顶点对应一个距离值

S中顶点：从V0到此顶点的最短路径长度

T中顶点：从V0到此顶点的只包括S中顶点作中间
顶点的最短路径长度

依据：可以证明V0到T中顶点Vk的最短路径，或是从V0到Vk的
直接路径的权值；或是从V0经S中顶点到Vk的路径权值之和
(反证法可证)

◇求最短路径步骤

◇初使时令 $S=\{V_0\}$, $T=\{\text{其余顶点}\}$, T 中顶点对应的距离值

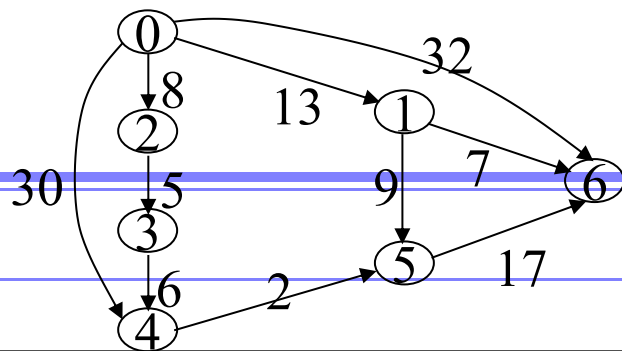
◇若存在 $\langle V_0, V_i \rangle$, 为 $\langle V_0, V_i \rangle$ 弧上的权值

◇若不存在 $\langle V_0, V_i \rangle$, 为 ∞

◇从 T 中选取一个其距离值为最小的顶点 W , 加入 S

◇对 T 中顶点的距离值进行修改: 若加进 W 作中间顶点, 从 V_0 到 V_i 的距离值比不加 W 的路径要短, 则修改此距离值

◇重复上述步骤, 直到 S 中包含所有顶点, 即 $S=V$ 为止

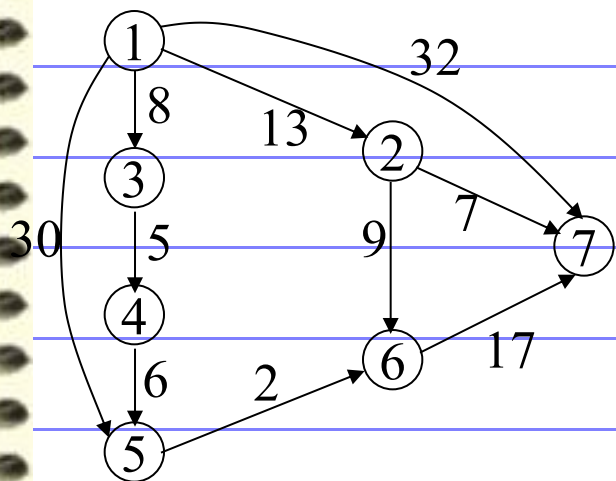


终点	从V0到各终点的最短路径及其长度				
V1	13 <V0,V1>	13 <V0,V1>	-----	-----	-----
V2	8 <V0,V2>	-----	-----	-----	-----
V3	∞	13 <V0,V2,V3>	13 <V0,V2,V3>	-----	-----
V4	30 <V0,V4>	30 <V0,V4>	30 <V0,V4>	19 <V0,V2,V3,V4>	-----
V5	∞	∞	22 <V0,V1,V5>	22 <V0,V1,V5>	21 <V0,V2,V3,V4,V5>
V6	32 <V0,V6>	32 <V0,V6>	20 <V0,V1,V6>	20 <V0,V1,V6>	20 <V0,V1,V6>
Vj	V2:8 <V0,V2>	V1:13 <V0,V1>	V3:13 <V0,V2,V3>	V4:19 <V0,V2,V3,V4>	V6:20 <V0,V1,V6>

✧ 算法实现

- ✧ 图用带权邻接矩阵存储 $ad[][]$
- ✧ 数组 $dist[]$ 存放当前找到的从源点 V_0 到每个终点的最短路径长度，其初态为图中直接路径权值
- ✧ 数组 $pre[]$ 表示从 V_0 到各终点的最短路径上，此顶点的前一顶点的序号；若从 V_0 到某终点无路径，则用0作为其前一顶点的序号

✧ 算法描述



$$ad[][] = \begin{bmatrix} 1 & 13 & 8 & \infty & 30 & \infty & 32 \\ \infty & 0 & \infty & \infty & \infty & 9 & 7 \\ \infty & \infty & 0 & 5 & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 & 6 & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & \infty & \infty & 0 & 17 \\ \infty & \infty & \infty & \infty & \infty & \infty & 0 \end{bmatrix}$$

最短路径	长度
<V1,V2>	13
<V1,V3>	8
<V1,V3,V4>	13
<V1,V3,V4,V5>	19
<V1,V3,V4,V5,V6>	21
<V1,V2,V7>	20

	0	1	2	3	4	5	6
dist	0	13	8	13	19	21	20
	0	1	2	3	4	5	6
pre	0	1	1	3	4	5	2
		k=1		(1)			

✧ 算法分析: $T(n)=O(n^2)$

//设数组final[n], 标识一个顶点是否已加入S中

BOOLEAN final[MAX_VEX] ;

int pre[MAX_VEX] , dist[MAX_VEX] ;

void Dijkstra_path (AdjGraph *G, int v)

/* 从图G中的顶点v出发到其余各顶点的最短路径 */

{ int j, k, m, min ;

for (j=0; j<G->vexnum; j++)

{ pre[j]=v ; final[j]=FALSE ;

dist[j]=G->adj[v][j] ;

} /* 各数组的初始化 */

dist[v]=0 ; final[v]=TRUE ; /* 设置S={v} */

for (j=0; j<G->vexnum-1; j++) /* 其余n-1个顶点 */

{

```
min=INFINITY ;
for ( k=0; k<G->vexnum; k++)
    { if (!final[k]&&dist[k]<min)
        { min=dist[k] ; m=k ; }
    } /* 求出当前最小的dist[k]值 */
final[m]=TRUE ; /* 将第m个顶点并入S中 */
for ( j=0; j<G->vexnum; j++)
    { if (!final[j]&&(dist[m]+G->adj[m][j]<dist[j]))
        { dist[j]=dist[m]+G->adj[m][j] ;
          pre[j]=m ;
        }
    } /* 修改dist和pre数组的值 */
} /* 找到最短路径 */
}
```

算法分析

Dijkstra算法的主要执行是：

- ◆ 数组变量的初始化：时间复杂度是 $O(n)$ ；
- ◆ 求最短路径的二重循环：时间复杂度是 $O(n^2)$ ；

因此，整个算法的时间复杂度是 $O(n^2)$ 。

■ 每一对顶点之间的最短路径

◇方法一：每次以一个顶点为源点，重复执行Dijkstra算法
n次—— $T(n)=O(n^3)$

◇方法二：弗洛伊德(Floyd)算法

□算法思想：逐个顶点试探法

□求最短路径步骤

◇初始时设置一个n阶方阵，令其对角线元素为0，
若存在弧 $\langle V_i, V_j \rangle$ ，则对应元素为权值；否则为 ∞

◇逐步试着在原直接路径中增加中间顶点，若加入
中间点后路径变短，则修改之；否则，维持原值

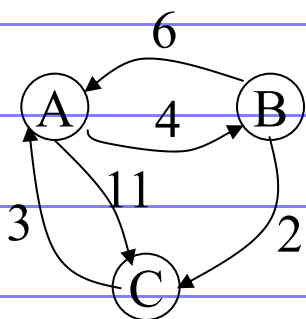
◇所有顶点试探完毕，算法结束

初始:
$$\begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix}$$

路径:

	AB	AC
BA		BC
CA		

例



加入V1:
$$\begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

路径:

	AB	AC
BA		BC
CA	CAB	

加入V2:
$$\begin{bmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

路径:

	AB	ABC
BA		BC
CA	CAB	

加入V3:
$$\begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

路径:

	AB	ABC
BCA		BC
CA	CAB	

✧ 算法实现

✧ 图用邻接矩阵存储

✧ length[][] 存放最短路径长度

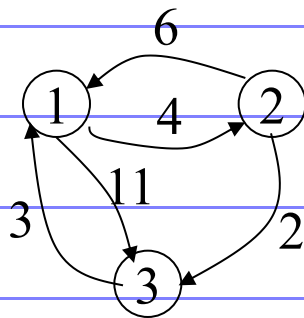
✧ path[i][j] 是从 V_i 到 V_j 的最短路径上 V_j 前一顶点序号

✧ 算法描述

初始:

$$\text{length} = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix} \quad \text{path} = \begin{bmatrix} 0 & 1 & 1 \\ 2 & 0 & 2 \\ 3 & 0 & 0 \end{bmatrix}$$

例



加入 V_1 :

$$\text{length} = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix} \quad \text{path} = \begin{bmatrix} 0 & 1 & 1 \\ 2 & 0 & 2 \\ 3 & 1 & 0 \end{bmatrix}$$

加入 V_2 :

$$\text{length} = \begin{bmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix} \quad \text{path} = \begin{bmatrix} 0 & 1 & 2 \\ 2 & 0 & 2 \\ 3 & 1 & 0 \end{bmatrix}$$

加入 V_3 :

✧ 算法分析: $T(n) = O(n^3)$

$$\text{length} = \begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix} \quad \text{path} = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 0 & 2 \\ 3 & 1 & 0 \end{bmatrix}$$

算法实现

```
int length[MAX_VEX][MAX_VEX] ;
```

```
int Path[MAX_VEX][MAX_VEX] ;
```

```
void Floyd_path (AdjGraph *G)
```

```
{ int j, k, m ;
```

```
  for ( j=0; j<G->vexnum; j++)
```

```
    for ( k=0; k<G->vexnum; k++)
```

```
      { /* 各数组的初始化 */
```

```
        length[j][k]=G->adj[j][k] ;
```

```
        Path[j][k]=-1 ;
```

```
        if(length[j][k] > 0) Path[j][k] = j;
```

```
      } // -1==length[j][k] 表示无穷大
```

```

for ( m=0; m<G->vexnum; m++)
    for ( j=0; j<G->vexnum; j++)
        for ( k=0; k<G->vexnum; k++)
            if ((length[j][m]+length [m][k])<length [j][k])
                { length[j][k] = length[j][m] + length[m][k] ;
                  Path[j][k]=m ;
                } /* 修改数组 length 和 Path 的元素值 */
for ( j=0; j<G->vexnum; j++)
    for ( k=0; k<G->vexnum; k++)
        if (j!=k)
            { printf(“%d到%d的最短路径为:\n”, j, k) ;
              printf(“%d ”,j) ;
              prn_pass(j, k) ;
              printf(“%d ”, k) ;
              printf(“最短路径长度为: %d\n”, length[j][k]) ;
            }
} /* end of Floyd */

void prn_pass(int j , int k)
{ if (Path[j][k]!=-1)
    { prn_pass(j, Path[j][k]) ;
      printf(“, %d” , Path[j][k]) ;
      prn_pass(Path[j][k], k) ;
    }
}

```

习题七

(1) 分析并回答下列问题：

① 图中顶点的度之和与边数之和的关系？

② 有向图中顶点的入度之和与出度之和的关系？

③ 具有 n 个顶点的无向图，至少应有多少条边才能确保是一个连通图？若采用邻接矩阵表示，则该矩阵的大小是多少？

④ 具有 n 个顶点的有向图，至少应有多少条弧才能确保是强连通图的？为什么？

(2) 设一有向图 $G=(V,E)$ ，其中 $V=\{a,b,c,d,e\}$ ， $E=\{<a,b>, <a,d>, <b,a>, <c,b>, <c,d>, <d,e>, <e,a>, <e,b>, <e,c>\}$

① 请画出该有向图，并求各顶点的入度和出度。

② 分别画出有向图的正邻接链表和逆邻接链表。

(3) 对图7-27所示的带权无向图。

① 写出相应的邻接矩阵表示。

② 写出相应的边表表示。

③ 求出各顶点的度。

(4) 已知有向图的逆邻接链表如图7-28所示。

① 画出该有向图。

② 写出相应的邻接矩阵表示。

③ 写出从顶点a开始的深度优先和广度优先遍历序列。

④ 画出从顶点a开始的深度优先和广度优先生成树。

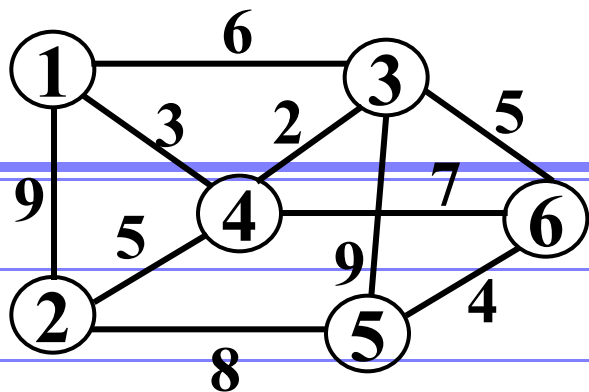


图7-27 带权无向图

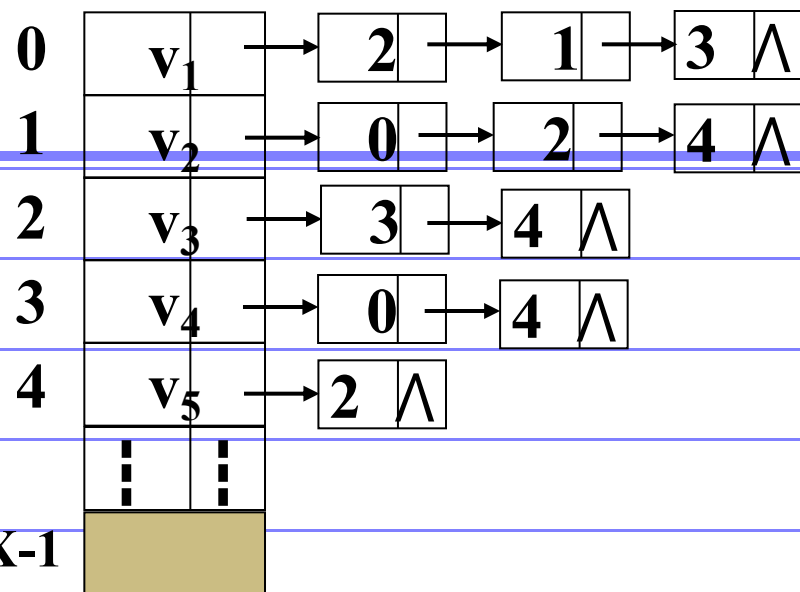


图7-28 有向图的逆邻接链表

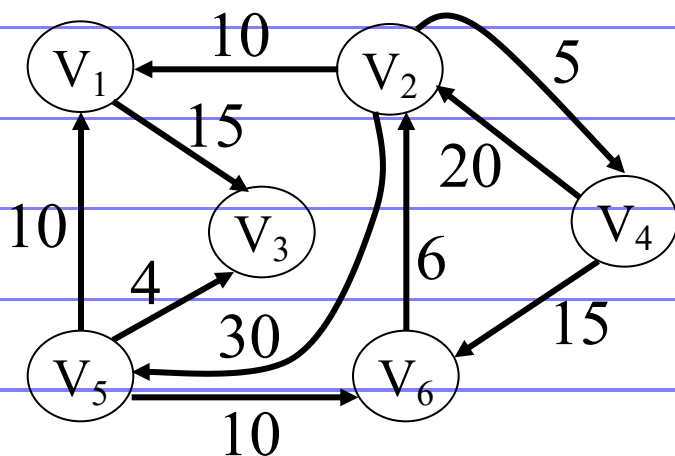


图7-29 带权有向图

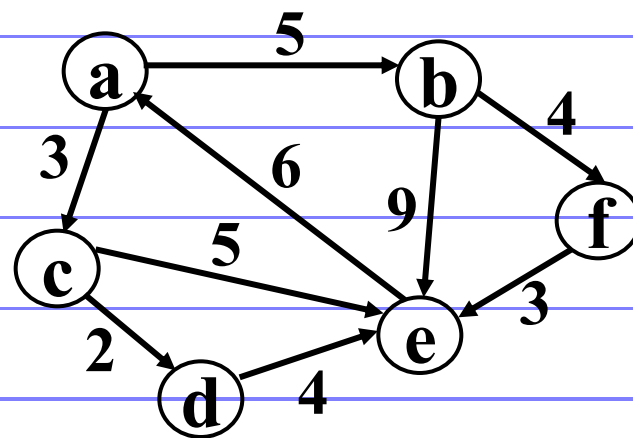


图7-30 带权有向图

- (5) 一个带权连通图的最小生成树是否唯一?在什么情况下可能不唯一?
- (6) 对于图7-27所示的带权无向图。
- ① 按照**Prime**算法给出从顶点2开始构造最小生成树的过程。
 - ② 按照**Kruskal**算法给出最小生成树的过程。
- (7) 已知带权有向图如图7-29所示, 请利用**Dijkstra**算法从顶点 V_4 出发到其余顶点的最短路径及长度, 给出相应的求解步骤。
- (8) 一个**AOV**网用邻接矩阵表示, 如图7-31。用拓扑排序求该**AOV**网的一个拓扑序列, 给出相应的步骤。

(10) 拓扑排序的结果不是唯一的，请给出如图7-32所示的有向图的所有可能的拓扑序列。

(11) 请在深度优先搜索算法的基础上设计一个对有向无环图进行拓扑排序的算法。

(12) 设计一个算法利用图的遍历方法输出一个无向图G中从顶点 V_i 到 V_j 的长度为S的简单路径，设图采用邻接链表作为存储结构。

(13) 假设一个工程的进度计划用AOE网表示，如图7-33所示。

① 求出每个事件的最早发生时间和最晚发生时间。

② 该工程完工至少需要多少时间？

③ 求出所有关键路径和关键活动。

$$\begin{array}{c}
 V_0 \\
 V_1 \\
 V_2 \\
 V_3 \\
 V_4 \\
 V_5 \\
 V_6
 \end{array}
 \begin{pmatrix}
 0 & 1 & 1 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 1 & 1 & 1 & 0 \\
 0 & 0 & 0 & 0 & 1 & 0 & 1 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0
 \end{pmatrix}$$

图7-31 一个AOV网的邻接矩阵

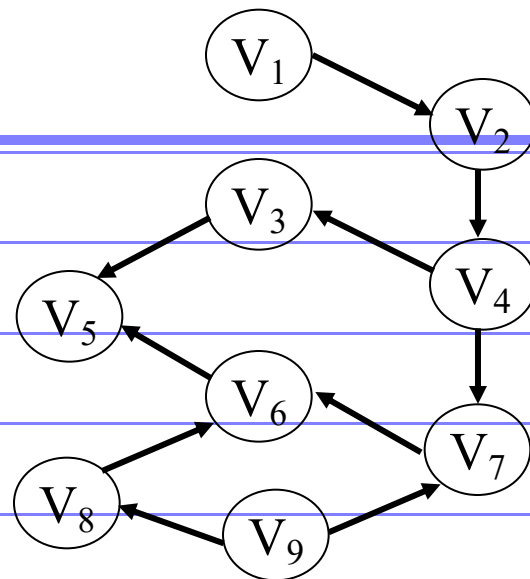


图7-32 有向图

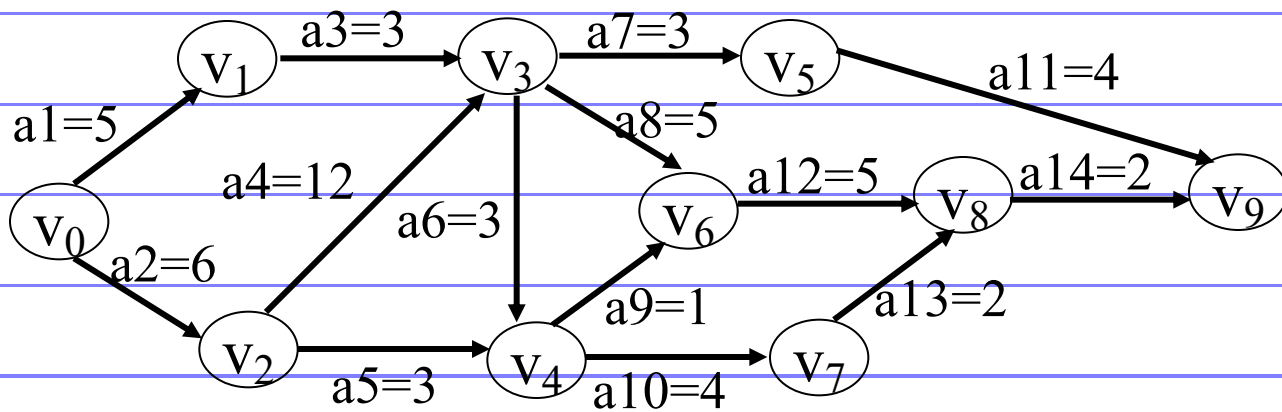


图7-33 一个AOE网